FishBot v0.4: Observer-Conditioned Deal Inference and Value-Based Declaration in Six-Player Canadian Fish

An exact reference posterior, a deployed approximation,
and a duplicate-block evaluation protocol

Dylan Nguyen
FishLab Research Project

Technical report — August 21, 2026
Repository: `https://github.com/dylann29/fish-optimization`
Experiments E1–E14 were produced at commit `bb3bc8a`; E15–E17 are newer and are regenerated by the commands in Appendix G. Artifact digests are in `research/v04/results/MANIFEST.json`.

Abstract

Canadian Fish, also called Literature, is a six-player card game of imperfect information played by two teams of three who may not communicate outside the public record of play. A team scores a half-suit only by declaring an exact allocation of its six cards among its own members, and any error concedes the half-suit to the opponents, so what matters is the probability of one complete six-card assignment rather than per-card confidence. Every card movement in the game is publicly observed, so a card whose location has never been revealed is still held by whoever was dealt it: the hidden state is exactly the initial deal, and conditioning on the public record reduces to counting the deals consistent with it under degree constraints. That count, the ownership marginals it induces and the probability that a named allocation is correct all admit closed-form evaluation. FishBot v0.4 contains an exact observer-conditioned deal-posterior engine, validated against exhaustive enumeration on small reachable states and used as a reference and validation oracle; the deployed and primarily evaluated v0.4-Fast policy runs a fitted approximate inference path instead. On the primary evaluation bank of 700 held-out deals played in six seat rotations for 4,200 games, v0.4-Fast defeated FishBot v0.3 in 3,153 games — a 75.07% win rate with a deal-clustered 95% interval of 73.71–76.40%. The result establishes strong performance within the tested simulator, rules dialect, opponent panel and deal distribution; it does not establish optimal play, equilibrium, generalisation to unseen opponent populations, or a causal advantage for exact inference.

1 Introduction

Canadian Fish, also called Literature, is a six-player card game of imperfect information: two teams of three, the whole deck dealt out. On turn a player names a card and an opponent; if the opponent holds it the card transfers and the asker continues, otherwise the turn passes to the opponent asked. Cards taken this way score nothing. A team scores only by declaring — naming one half-suit and stating which of its own members holds each of the six cards — a correct declaration awarding the half-suit to the declaring team and an incorrect one to the opponents. Teammates may not communicate outside the record of play, so a declaration must be assembled from what asking has revealed. §2 specifies every consequential rule choice in the dialect studied: the 54-card deck of nine six-card half-suits, the requirement that an asker hold another card of the half-suit asked in, and declaration at any moment, including out of turn. Partial knowledge of where cards sit is worth nothing at the moment

of declaration: what a team needs is the probability that one complete six-card assignment is correct, and a probability that is high but wrong transfers the half-suit to the other side.

This paper asks how close to exact that quantity can be brought, what it costs to compute, and what a policy built on it gains. Cards in Fish move only through publicly observed transfers, so a card whose location has never been revealed is still held by the player who was dealt it, and the hidden state of the game is exactly the initial deal (Proposition 1). Conditioning on the public record is then a counting problem over deals, and that count is what an observer-conditioned deal posterior requires (§5).

FishBot v0.3 [1] approached the same game with a one-ply utility policy over an approximate posterior. It established that tracking the public ask record and reconciling it against hand sizes dominated the other features it tested, and left three things open: its posterior was approximate and was not validated against any exact reference; its declaration decision was a fixed confidence threshold applied to an approximate allocation score; and its simulator did not implement declarations made outside the declarer's turn, which changes both the tempo of the game and what a player can wait for. This paper makes four contributions.

1. **A formalised rules dialect, an information-safe engine, and the constraints the record imposes.** We specify the dialect (§2) and implement it in a simulator whose agent interface exposes only what a seated player may observe, audited at runtime. We state the constraint system C1–C5 — hand sizes, cards whose holder is known, voids established by failed asks, and the certificate an ask carries about the asker's own hand — that the public record places on the initial deal.
2. **An exact observer-conditioned deal posterior, together with the approximation the deployed policy runs.** We evaluate in closed form the number of deals consistent with C1–C5, the per-card ownership marginals, the probability that a team holds a half-suit outright and the probability that a named allocation is correct, validated against a brute-force enumeration oracle on small reachable states (§10.1). The deployed v0.4-Fast policy runs a cheaper fitted approximation instead; the exact engine serves as a reference and validation oracle, and v0.4-Block denotes the same fitted policy with the reference belief substituted.
3. **An analysis of locked half-suits and a value-based declaration rule.** We characterise half-suits one team already holds in full and derive what ask legality implies about them, then build a declaration timing rule on the allocation probability and a fitted value model rather than a fixed confidence threshold (§6).
4. **A duplicate-controlled empirical evaluation.** Every deal on the primary bank is played in all six seat rotations, with intervals clustered on the deal; the frozen-policy ablations use a weaker two-rotation block for budget reasons, and say so (§9). The protocol also covers calibration of the ask and allocation forecasts and a local response probe fitted against the frozen policy. On that bank — 700 held-out deals in six rotations, 4,200 games (E3) — v0.4-Fast defeated FishBot v0.3 in 3,153 games, a 75.07% win rate with a deal-clustered 95% interval of 73.71–76.40%.

Termination is a secondary empirical observation, not a theorem: ownership of a half-suit freezes once one team holds all six of its cards, but asks inside it stay legal and still carry ask-legality certificates, so allocation information can change while ownership cannot, and ownership monotonicity alone proves nothing about termination. In mirror self-play an earlier fitted configuration with no forcing rule reached the action cap without finishing in 21.0% of games, at both the 400-ask and the 1000-ask cap (E11), against zero under the graduated escalation rule the deployed policy uses; §6 states what a general result would require.

This paper claims no Nash or team equilibrium, reports no exploitability bound, and does not claim that Fish is solved. The term "held-out" refers to deals except where §8 applies it to opponent identities: the evaluation deals were withheld from fitting and from configuration selection, but opponent identities largely were not. Six of the eight evaluation-panel policies were also in the fitting panel, and only the misdirection artist and the random legal control were withheld, so the experiments measure

generalisation to new deals rather than to new strategic populations. §12 organises these and the remaining threats to validity.

2 The game and the rules dialect studied

Canadian Fish (also called Literature) is played by six players in two teams of three, seated so that the teams alternate around the table. The deck is 54 cards, partitioned into nine *half-suits* of six cards each: the low band $\{2, 3, 4, 5, 6, 7\}$ and the high band $\{9, 10, \mathrm{J}, \mathrm{Q}, \mathrm{K}, \mathrm{A}\}$ of each of the four suits, together with a ninth half-suit containing the four eights and the two jokers. Each player is dealt nine cards. A half-suit is awarded to one team or the other when it is claimed; the team that takes at least five of the nine half-suits wins the game.

Asking. On their turn a player asks one opponent for one named card. The ask is legal only if the named card lies in a half-suit that is still live, the asker holds at least one *other* card of that half-suit, the asker does not hold the named card, the target sits on the opposing team, and the target still holds at least one card. If the target holds the named card they must surrender it and the asker retains the turn; otherwise the turn passes to the target. Every ask, its answer, every resulting transfer, and every player's card count are public.

Declaring. A claim on a half-suit is a *declaration*: an exact allocation naming, for each of the six cards, which member of the declaring player's own team holds it. If every one of the six assignments is correct the declaring team is awarded the half-suit; any error at all awards it to the opposing team. Either way the six cards leave play, the half-suit becomes inactive, and the turn returns to the player who held it before the declaration.

Running out of cards. A player with no cards leaves the asking cycle: they can neither ask nor be asked, though they may still be named as the holder of a card in a teammate's declaration and, in this dialect, may still declare. If the turn reaches a cardless player — possible only after a declaration — that player chooses a teammate who still holds cards to receive it. When an entire team is cardless, the other team must declare every remaining half-suit with no further asking, and may misdeclare and thereby hand half-suits to the cardless team. In both situations teammates negotiate openly but exchange nothing beyond willingness (§A.7).

2.1 The dialect studied

Published descriptions of the game document several mutually incompatible variants: deck size, whether declarations are confined to the declarer's turn, what becomes of a misdeclared half-suit, and how the endgame is run all differ between sources [2]. What follows is one explicit selection among those variants, not a uniquely canonical rule set. It matches the strategy corpus this work draws on [3], and every contested choice is exposed as an engine switch, so its effect can be measured rather than assumed (§10.5, §E). The consequential choices, each with the switch that changes it, are:

- **Deck:** 54 cards, nine half-suits, nine per player (`--sets=8` gives the 48-card, eight-half-suit form).
- **Declaration timing:** any seat may declare at any moment, in or out of turn, holding cards or not (`--no-out-of-turn`, `--no-cardless-declare`).
- **Misdeclaration:** the half-suit is awarded to the opposing team; published variants differ here [2].
- **Simultaneous declarations:** the lowest seat index that offers a declaration acts first — a modelling choice, not a rule (`--arb=low|high|turn`).
- **Forced endgame:** an eight-level willingness ladder (`--legacy` gives the two-level v0.3 ladder).

- **Action cap:** 400 asks, 360 under `--legacy`, any residue adjudicated neutrally by majority physical holding (`--maxasks`).

No prearranged signalling conventions are available to any policy, and team communication is limited to the willingness bit above and the successor choice made by a cardless turn-holder. Table 5 in §A gives the full list against the underlying `Rules` fields, and the rest of that appendix states the dialect at the length needed to re-implement it. Unless a result says otherwise, every number in this paper was produced under these settings.

Simultaneous declarations. None of the statements of the rules we reviewed says who acts first when two seats offer a declaration at the same instant, so the arbitration order is a modelling choice rather than a rule. Ranking candidates by stated confidence is deliberately excluded, because it would compare private confidences across seats; E17 reports the sensitivity of the primary head-to-head result to the three orders implemented (§A.5, §E).

The action cap. The cap is a backstop against unbounded play, not part of the rules: two sufficiently patient policies could in principle stall, and §6.3 treats termination as an empirical property of the fitted policies rather than a guarantee. Its incidence is reported with every experiment, and is 0.000% in E1, E3, E7, E10 and E13; the exception is the deliberately unforced mirror configuration of E11 (§6.3), which is measured precisely because it does reach the cap.

2.2 What the v0.3 simulator modelled and what it did not

The v0.3 engine [1] implemented the deck, the ask-legality conditions, transfers, exact-allocation declarations and the cardless-team endgame, and is the direct ancestor of this work. It differs from the dialect above in four respects, each of which changes the strategic problem rather than only its presentation. Two of them bear on the results reported here:

1. **Declarations were restricted to the declarer's own turn.** With out-of-turn declarations allowed, declaration timing becomes a continuous decision rather than a once-per-turn one, and a team can claim a half-suit immediately before an opponent acts.
2. **Games were adjudicated after an action cap** of 360 asks. A cap remains necessary here, but in v0.4 it is a backstop with the measured incidence given above, and the residue is split neutrally rather than awarded to one side.

The other two — the successor rule applied when the turn reaches a cardless seat, and the confidence comparison by which the v0.3 forced endgame selected a declarer — are stated in §A.8.

Each of the four is a switch in the v0.4 engine, so the difference each makes is measurable: E7 reports play under the v0.3 dialect, under the 48-card eight-half-suit deck, and with out-of-turn declarations disabled (§10.5, §E). Setting `--legacy` restores the v0.3 dialect in full — turn-only declarations, no declarations by cardless players, the two-level willingness ladder and the 360-ask cap — and is also the configuration in which the v0.3 port was validated (§10.1).

3 Related work

This section covers only the work the rest of the report builds on. The background it does not depend on is collected in §I, and the evaluation-methodology literature is taken up in §9.

3.1 Fish and Literature

The published material on this game is human and informal, and neither of its two substantial sources is quantitative or describes a program. McLeod's Pagat entry [2] is the canonical rules source and

records several points on which dialects differ, including the variant in which a declaration error awards the half-suit to the opposing team; ours is specified in §2 and listed in full in §A. Develin's chapter [3] is the most detailed strategy writing we located, describes the same dialect, and supplies two observations our design uses: an ask teaches the opponents as much as it teaches a teammate, which the information-leak features of §7 price, and a player may not declare a half-suit they hold entirely, a rules-level constraint rather than a tactical one.

Computational prior art is close to absent among the sources we reviewed: arXiv searches on 21 August 2026 for "Canadian Fish", "Literature card game" and "half-suit" returned nothing, a sweep of public GitHub repositories found none containing search, planning or a working learner for this game, and Literature is not among the environments packaged with RLCard [10]. That search was not exhaustive, so every novelty statement in this report takes the form "we found no prior published work that ..." and should be read against this scope rather than as a categorical claim. The one agent we found, Somani's `literature` [7, 8], plays the four-player, 48-card variant, which does not raise the six-player allocation problem this report addresses, and no strength numbers for it have been published, so it cannot serve as a baseline; a closed-source Android application advertises four- and six-player bots with no stated methodology [9], which we were not able to verify. §I describes both, the informal strategy corpus, and where Develin and Pagat disagree.

3.2 Determinization, and why it is not the foundation used here

Determinization, or Perfect Information Monte Carlo — sample hidden cards consistently with the public history, solve the resulting perfect-information game, and play the move with the best average value [12] — is the standard recipe for hidden-hand card games, and is unsound however many worlds are drawn [13]. It is a poor foundation for Fish for a structural reason as well as that one: the perfect-information game is degenerate, because a clairvoyant player names only cards the target actually holds, so never fails an ask, never loses the turn, and declares each half-suit correctly as soon as their team holds all of it. Almost every reachable position therefore carries the same perfect-information value, and averaging that value over deals sampled from the public history collapses to choosing the ask most likely to hit: a one-ply heuristic blind to the information an ask leaks, the information a refusal supplies and the option value of postponing a declaration. §I gives that collapse in full, together with the unsoundness diagnosis, the conditions under which determinization performs well anyway, and the proposed repairs. We therefore treat the hit probability as one feature among several (§7) rather than as an engine, and put the modelling effort into the posterior (§5) and the declaration-timing rule (§6).

3.3 Policy-aware inference over deals

Skat engines enumerate the deals consistent with the public record and reweight them by a learned model of what each player would have done — a supervised per-card location model [24], or reach probabilities taken from a policy [25] — and a dedicated inference module was reported to be worth +217 tournament points per 36 games to the Skat engine Kermit on its own [23]. The policy-aware prior of §5.7 is the same idea in a much simpler form, a fitted soft reweighting of the rules posterior rather than a learned opponent model, and §10.4 reports its frozen-policy ablation rather than claiming it is necessary.

3.4 Counting deals under constraints

Counting the deals consistent with a public history is a permanent computation; the exact formulas and the randomised approximation schemes are recorded in §I, and none is practical at $n = 54$. Matrix scaling is the usual practical approximation [60], and a Sinkhorn iteration of this kind is what the Fast path uses (§5.6). The tractable case that matters here comes from the contingency-table literature,

where counting tables with a *constant* number of rows is polynomial [63]. Fish lands in that regime because the hidden state is exactly the initial deal (Proposition 1) and each seat's share of it is a known, constant number of cards, so the counting matrix has one column type per seat, with capacities that are small integers rather than binary-encoded margins. Ask legality is disjunctive, which would normally destroy the product form, but every legality certificate is confined to a single half-suit, so exhaustive enumeration within a block keeps the computation exact; §5 develops this into the reference engine and §5.6 into the deployed approximation.

3.5 Team games and common information

Three teammates share a payoff, hold different information and cannot communicate, so the target concept is neither a single player with pooled knowledge nor a Nash equilibrium over individual strategies, but a team-maxmin equilibrium with correlation: a correlated joint strategy agreed before the deal and then executed without communicating [36]. We do not compute it, and none of the machinery collected in §I for computing it is applied here, because one seat's information set at deal time contains $45!/(9!)^5 \approx 1.9 \times 10^{28}$ deals and a prescription would have to name an action for each teammate at each of their possible hands. Nothing in this report is an equilibrium result: win rates are measured against a fixed opponent panel, and §10.6 reports a fitted-response probe whose achieved score is a lower bound within its search class.

4 The simulator: implementation, information safety, verification

4.1 Implementation

The engine is a single-header C++20 core in `engine/src/`. A hand is one `uint64_t` over the 54 card indices $c = 6h + i$, where h numbers the nine half-suits and i the six cards within one, so a half-suit mask is `0x3F << 6h` and the six hands together with the deal they came from occupy under a hundred bytes. Where the policy explores a hypothetical — the two-ply refinement of §7.3 — it copies the constraint state and recomputes on the copy rather than making and unmaking moves on the live game. There are no rollouts anywhere in this agent: every quantity the policy consumes is either a closed-form posterior query (§5) or a fitted linear score (§7).

Games are driven by a loop that, after every public event, (i) offers every player the opportunity to declare, (ii) resolves a cardless team into the forced endgame, (iii) resolves a cardless turn-holder into a chosen successor, and (iv) otherwise asks the turn-holder for a move. All randomness comes from counter-based streams keyed by *(seed, seat, purpose)*, so a deal and its entire play are reproducible from the seed alone. The duplicate-block protocol of §9 depends on that property.

4.2 Information safety

The acting policy never receives the game state. It receives its own `Knowledge` object — the constraint bookkeeping of §5 — and the public record, in which each event carries only the actor, the target, the named card, the outcome, any declared allocation, and the resulting public hand counts. There is no interface through which a policy can read another player's cards. A leak of the kind the FishBot v0.3 audit reported in its predecessor [1], where a reply-risk feature estimated a target's follow-up options by invoking the legal-move generator with that target as actor, is therefore excluded structurally rather than by inspection.

To confirm that no leak remains, and more importantly that the deductions the belief machinery makes are *sound*, the engine has an audit mode that runs after every public event and checks, for every player, four properties against the true deal:

- every card the constraint state records as resolved is in fact held by the seat it is resolved to, and no card of a live half-suit is marked out of play;
- for every unresolved card, the true holder lies in the surviving support M_c (§5);
- the derived capacities (1) equal the true counts of unresolved cards per player;
- every live ask-legality certificate (2) is satisfied by the true deal.

A single violation would mean the belief engine had excluded the truth. In experiment E1 the engine performed **23,594,580** such checks with **0** violations under the full dialect, and **10,910,844** checks with no violations under the FishBot v0.3 dialect (§10.1). The audit is run inside `fish verify`, which plays the 9×9 ordered cross-product of nine policies (v0.4-Fast, FishBot v0.3, FishBot v0.2, turn-starvation lockout, posterior detective, focused hunter, adaptive diversifier, misdirection artist, random legal control), so that both orderings of every pair and every self-play pairing are exercised.

4.3 Verification suite

Beyond the soundness audit the suite checks deck integrity and card conservation; that exactly nine half-suits are resolved in every completed game; that a declaration only ever assigns cards to the declarer's own teammates; that ask legality is enforced independently on the move generator and on the applier, so a policy cannot submit an illegal ask even if its generator is wrong; that replaying a seed at a fixed thread count reproduces the game bit-identically; and that the action cap of §2 — a backstop whose residue adjudication is a convention of the simulator rather than a rule of the game — is not reached in ordinary play, which it was not (0.000%, E1 and E3). The in-policy mechanism that keeps games terminating is the subject of §6.3. The random streams are keyed so that thread count cannot affect a game's outcome, but the suite tests replay identity only at a fixed thread count and does not test that claim directly.

5 The observer-conditioned deal posterior

5.1 The hidden state is the initial deal

In the dialect of §2 a card changes hands in exactly one way, a successful ask, which every player observes; declarations remove cards from play, and that too is public. Hence:

Proposition 1 (Public transfer)**.** *Assume the dialect of §2, in which the only card movements are successful asks and declarations and both are announced. Let c be a card whose location has never been publicly revealed. Then c is still held by the player to whom it was dealt, the joint hidden state of the game at any time is exactly the initial deal, and the current holder of every card is a deterministic function of the initial deal and the public history.*

Proof. A card changes hands only through a successful ask, and both the ask and its outcome are announced, so every change of holder is a public event and a card with no such event is where it was dealt. Replaying the public history from the deal then determines current holdings, since each announced transfer moves a named card between named seats and each declaration removes a named half-suit. □

Proposition 1 eliminates the filtering recursion that dominates inference in most imperfect-information card games: there is no belief that decays and no dynamics to approximate, only a fixed hidden object and an accumulating set of logical constraints on it. What follows is an *observer-conditioned deal posterior*: every player computes it from the same public history but conditions additionally on their own private hand, so the six posteriors differ and no single distribution is shared. It is exact with respect to the rules-conditioned uniform prior of §5.3, and it is not a complete Bayesian opponent

model, since it conditions on nothing about how opponents choose among legal moves (§5.7). Two interchangeable engines compute it, contrasted in Figure 1: the exact reference engine of §5.5, and the approximate Fast path of §5.6 that the deployed v0.4-Fast policy runs.

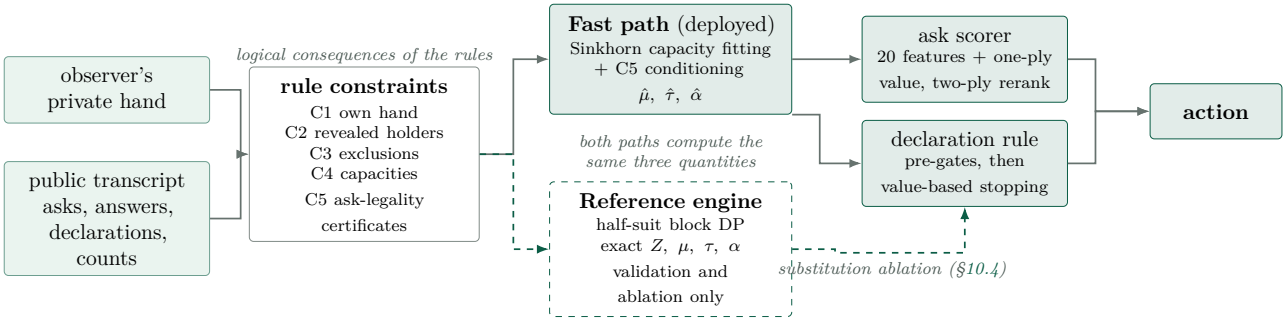

Figure 1: Inference and decision pipeline. The observer's private hand and the public transcript determine the rule constraints C1–C5, from which two interchangeable engines compute the same three quantities: per-card ownership marginals, team-ownership probabilities, and named-allocation probabilities. The **Fast path** (Sinkhorn capacity fitting with C5 conditioning) is what the deployed v0.4-Fast policy runs, and produced every performance number reported in this paper. The **reference engine** (exact half-suit block dynamic program) is used for validation and for the substitution ablation only, and is not on the deployed decision path.

5.2 Notation

These symbols are used throughout the paper and are not varied. $\mathcal{H}$ is the set of live half-suits and $H \in \mathcal{H}$ one of them. Fix an observer i. U is the set of cards in live half-suits whose holder is not publicly known to i, and for $c \in U$ the map $\sigma : U \to \{0, \ldots, 5\}$ gives that card's holder, which by Proposition 1 is constant in time. $M_c \subseteq \{0, \ldots, 5\}$ is the surviving support of $\sigma(c)$, and q_p is the capacity of seat p, the number of cards of U that seat must hold. Z is the number of assignments σ consistent with the constraints below, that is the partition function of the posterior. The three quantities the policy consumes are the per-card ownership marginal $\mu_{c,p} = \Pr[\sigma(c) = p]$; the probability $\tau(H, T)$ that team T owns every card of half-suit H; and the probability $\alpha(A)$ that a named allocation A — an assignment of all six cards of one half-suit to named seats — is correct. Unhatted symbols are values from the exact reference engine, hatted ones ($\hat{\mu}_{c,p}$, $\hat{\tau}(H, T)$, $\hat{\alpha}(A)$) the same values from the Fast path of §5.6. Probabilities are conditional on the observer's information $\mathcal{I}$ throughout; the conditioning bar is suppressed.

5.3 The constraint system

The observer's information is exactly the following five constraints on σ.

(C1) Own hand. $\sigma(c) \neq i$ for every $c \in U$, and every card i holds is resolved.

(C2) Public transfers and correct declarations. Any card whose holder has been revealed leaves U with its holder known exactly. A successful ask reveals the holder *before* the transfer, and that is the value relevant to every earlier constraint.

(C3) Exclusions. An ask for c proves the asker did not hold c; a miss proves the target did not hold c. Both are permanent, again by Proposition 1. The surviving support M_c records them.

(C4) Capacities. Hand counts are public, so for every player p

$$\big|\{c \in U : \sigma(c) = p\}\big| \;=\; q_p \;:=\; h_p - k_p, \tag{1}$$

where h_p is p's public card count and k_p the number of live cards already known to be p's. Note that $\sum_p q_p = |U|$ automatically, and that a declaration — correct or not — is absorbed by (1) without special handling, because the count drop it causes is public.

(C5) Ask legality. An ask for c in half-suit H by player a proves that a held some *other* card of H at that moment. Because unresolved cards never move, this is the time-independent disjunction

$$\bigvee_{d \in D} [\sigma(d) = a], \qquad D = \{d \in H \setminus \{c\} : d \text{ unresolved}, \ a \in M_d\}, \tag{2}$$

vacuous if some card of H is already known to be a's. The asking seat is written a to keep A reserved for the named allocation of §5.2. Every such certificate is confined to a single half-suit.

The prior is uniform over deals consistent with the rules, so the posterior is uniform over the assignments σ satisfying (C1)–(C5), and $\Pr[\sigma] = 1/Z$ for each of them. This is exact with respect to that prior, and it is policy-agnostic.

5.4 Counting: a capacity-vector dynamic program

Set (C5) aside for a moment. Counting assignments subject to (C3) and (C4) is a degree-constrained bipartite counting problem — the permanent of a 0/1 matrix with at most six distinct column types — solved exactly by a forward–backward recursion over the vector $n = (n_0, \ldots, n_5)$ of *remaining* capacities from (1). Ordering U as $c_1, \ldots, c_{|U|}$, $F_j(n)$ counts placements of the first j cards leaving n unused and $B_j(n)$ placements of the rest out of n:

$$F_j(n) = \sum_{p \in M_{c_j} : n_p < q_p} F_{j-1}(n + e_p), \qquad F_0(q) = 1, \tag{3}$$

$$B_j(n) = \sum_{p \in M_{c_{j+1}} : n_p \geq 1} B_{j+1}(n - e_p), \qquad B_{|U|}(\mathbf{0}) = 1, \tag{4}$$

$$Z = F_{|U|}(\mathbf{0}) \ = \ B_0(q), \tag{5}$$

Here e_p is the p-th unit vector. Recursions (3)–(5) count rather than sample, so they and the per-card marginal (16) of §B.1, which contracts the two tables at card j, are *exact* for (C1)–(C4); the layer index is fixed by the state, so a full forward–backward pass costs at most 6×10^5 multiply–adds regardless of the position (§B.1, §B.2, (17)). The backward table also yields a direct sampler, rejection-free for (C1)–(C4) but combined with an exact accept/reject test against (2) when a (C5) certificate is live, so the combined procedure is exact but not rejection-free; it is the ground truth for the E2 cross-checks of §10.1 (§B.3).

5.5 Ask legality by half-suit block enumeration

Constraint (C5) is disjunctive and breaks the product form of (3)–(4), and inclusion–exclusion over the k certificates a game accumulates — tens of them — would cost 2^k passes. The escape is structural: every certificate lives inside one half-suit, so U partitions into half-suit blocks admitting one of two exact treatments. A block with no live certificate is expanded card by card as in (3)–(4); one with a live certificate is enumerated exhaustively — at most $\prod_{c \in H} |M_c| \leq 5^6$ assignments — each tested against (2) directly, and the survivors aggregated into a count-vector table

$$g_H(t) \ = \ \big|\{\text{assignments } \beta \text{ of } H : \text{count}(\beta) = t, \ \beta \models \text{certificates of } H\}\big|, \qquad \textstyle\sum_p t_p = m_H, \tag{6}$$

where m_H is the number of unresolved cards of H and $\text{count}(\beta)_p$ is the number of them β gives to seat p. Either way a block consumes a known number of cards, so the layer identity survives and the

dynamic program runs over blocks rather than cards (§B.4 gives the mechanics of both). Writing S_t for the path mass of count vector t at its block's entry layer,

$$S_t \;=\; \sum_{|n| \,=\, \text{entry layer}} F(n)\, B(n-t), \tag{7}$$

the three quantities of §5.2 are read straight off the tables:

$$\mu_{c,p} = \frac{1}{Z} \sum_{t} g_H^{(c \to p)}(t)\, S_t \qquad \text{(per-card ownership)} \tag{8}$$

$$\alpha(A) = \frac{S_{t(A)}}{Z} \qquad \text{(the declaration query)} \tag{9}$$

$$\tau(H,T) = \frac{1}{Z} \sum_{t\,:\,\mathrm{supp}(t) \subseteq T} g_H(t)\, S_t \qquad \text{(does one team hold the half-suit?)} \tag{10}$$

with $g_H^{(c \to p)}$ the table restricted to survivors that give card c to seat p and $t(A)$ the count vector of allocation A. All three are *exact* under the uniform prior of §5.3; no independence assumption and no sampling enters.

Equation (9) is the one that matters for play. Declaring requires naming an exact allocation, so the quantity that governs the decision is the joint probability that all six assignments are simultaneously correct — not a product of per-card marginals, which is badly biased because the six cards of a half-suit are strongly dependent through the capacity constraint (1). FishBot v0.3 scored declaration candidates with a geometric mean of per-card marginals; v0.4-Fast and v0.4-Block both target (9), exactly in the latter case and approximately in the former (§5.6).

Corollary 2 (Equal probability within a count vector)**.** *Under the uniform posterior of §5.3, any two allocations of the same half-suit that satisfy (C1)–(C5) and share a count vector have equal probability.*

Proof. By (9) the probability of a surviving allocation A depends on A only through $t(A)$, since S_t is a function of the count vector and the dynamic-programming tables outside the block. □

The maximum-probability allocation is therefore determined by the count vector alone, so the choice among survivors within a count vector is arbitrary and `bestTeamAllocation` may return any of them. An allocation that violates a certificate of (2), however, has probability zero even when its count vector is shared by survivors, so survival must be checked explicitly rather than inferred from the count vector. The corollary and the surviving-allocation caveat are both validated against exhaustive enumeration on small reachable states in E15 (§10.1).

5.6 A fast approximate posterior

The reference engine is affordable per query, but six observers invoke it after every public event, putting it in the inner loop of every large experiment (§10.7). We therefore also implement an approximation, the *Fast path*: it keeps the exact bookkeeping of (C1)–(C3) and the exact capacities (C4), fits them by Sinkhorn scaling of the support matrix, and interleaves a conditioning step (18) for (C5) that reweights a certificate's cards D by treating their entries $p_{d,a}$ as independent Bernoulli indicators before capacity fitting is re-run (derivation and iteration schedule in §B.7). The cost is $O(\text{iters} \cdot |U| \cdot 6)$ rather than the $O(\prod_p (q_p + 1))$ of (17).

Equation (18) is an *approximation*, and so are the $\hat{\mu}$, $\hat{\tau}$ and $\hat{\alpha}$ it produces: its independence assumption is false, because the capacity constraints that make (9) necessary also couple the cards inside a certificate. E2 measures the gap over 40 games and 293,524 paired comparisons — mean absolute difference 0.01705 between $\hat{\mu}$ and μ, maximum 0.4982, accurate on average with a heavy tail

whose positions E2 does not record (§10.1). The Fast path is the compiled default (`BeliefMode::Fast`) and produced every performance number reported in this paper; that configuration is v0.4-Fast, and v0.4-Block denotes the same fitted policy with the exact reference engine substituted for it, which appears only in the validation of §10.1 and the substitution ablation of §10.4.

5.7 The residual: policy-aware inference

The posterior of (C1)–(C5) is exact with respect to the rules but is not the full Bayesian posterior over deals, which would additionally weight each deal x by the likelihood $L(\mathcal{I} \mid x) = \prod_t \pi_{p_t}(a_t \mid \mathcal{I}_{<t}, x)$ that the observed actions would have been chosen given x. That likelihood is playstyle-dependent: a player who has taken many turns and never asked in half-suit H is, under any plausible policy, less likely to hold cards of H, and the rules alone say nothing about this.

We implement a two-parameter family of such corrections as prior weights inside the same machinery,

$$w(c, p) \;=\; \exp\!\big(\theta\, a_{p,H(c)} - \phi\,(a_p - a_{p,H(c)})\big), \tag{11}$$

where $a_{p,H}$ counts p's public asks in half-suit H, a_p their total public asks, and $H(c)$ is the half-suit of card c. The weights replace the uniform initialisation of the Sinkhorn fit, so $\theta = \phi = 0$ recovers the policy-agnostic posterior of §5.3, and $\phi = 0$ recovers, in form, the heuristic FishBot v0.3 used in place of the hard certificate (2). Equation (11) is *fitted*: θ and ϕ are two coordinates of the 34-coordinate vector optimised jointly with the rest of the policy (§8).

In the frozen-policy ablation E5 (§10.4), disabling the prior — setting $\theta = \phi = 0$ while leaving every other coordinate at its fitted value — lowered the win rate by +3.88 points, 95% paired percentile bootstrap interval over deals +2.08–+5.73. The fitted policy therefore depends on the prior. Because the remaining coordinates were fitted with the prior active, this measures the sensitivity of *this* fitted vector to removing it and is not evidence about a policy retrained without it.

6 Locked half-suits, declaration timing, and termination

Declaration is the only action in the dialect of §2 that converts information into score. This section states the one structural property of declaration timing that follows from the rules alone, separates it from the properties it does *not* imply, gives the fitted rule v0.4-Fast uses, and reports what is and is not known about termination.

6.1 A half-suit held by one team cannot be taken back

Theorem 3 (Locked half-suits)**.** *Suppose every card of half-suit H is held by members of team T. Then no member of the opposing team can ever legally ask for a card of H, so no card of H can change hands, and H remains wholly held by T for as long as it stays undeclared. Moreover any declaration of H by the opposing team is necessarily incorrect and awards H to T.*

Proof. A legal ask for a card of H requires the asker to hold another card of H, so with all six cards held by members of T every opposing player is void in H and has no such ask, and a player holding no cards at all has no legal ask of any kind. Cards move only through successful asks, and the only asks in H available to anyone are asks by members of T directed at opponents, which are guaranteed misses; ownership of H is therefore frozen. For the second part, a declaration must assign every card of H to a teammate of the declarer, so an opposing declarer would assign cards held by T to the other team, which is false, and H is awarded to T. □

Theorem 3 does not assert that the probability of a correct claim rises monotonically, and it does not: as a ratio of counts of consistent assignments, both numerator and denominator shrink as constraints

accumulate, and a deduction elsewhere in the deal can eliminate more completions of the true allocation than of a rival. What is monotone is the support: the allocations of H extendable to a consistent deal are non-increasing.

Corollary 4 (Waiting carries no ownership risk)**.** *For a locked half-suit the "steal" term in the declaration trade-off is identically zero: H can leave the undeclared state only through a declaration by T, or one by the opposing team, which by Theorem 3 awards H to T in any case. Declaring immediately therefore cannot protect ownership; the remaining reasons to declare are the point itself and the positional consequences of removing six cards from play.*

Corollary 4 concerns ownership only, which matters for §6.3.

Observation 5 (Ownership freezes; allocation information need not)**.** Let H be a half-suit all of whose cards sit with team T. Two statements hold simultaneously and should not be merged. First, ownership of H is frozen by Theorem 3. Second, asks inside H remain *legal* for members of T; such an ask transfers nothing, every target being void, but it is a public event and still carries the certificate (2), which constrains which teammate holds which card. Allocation information about H can therefore continue to change while ownership cannot, deliberately at the price of a turn and incidentally as capacities tighten elsewhere through (1). Ownership monotonicity alone therefore does not establish that a position with only locked half-suits is informationally frozen, and does not prove universal non-termination.

Develin [3] records the same channel from the human side, and v0.4-Fast does not exploit it deliberately (§12). Two forces therefore act on declaration timing in opposite directions — a retained locked half-suit keeps six cards absorbing capacity mass in U, while a hand of locked cards licenses only asks certain to fail — and which dominates is a quantitative question about a position and an opponent population rather than one settled by Theorem 3. §H develops both sides.

6.2 A value-based declaration rule

At each declaration opportunity v0.4-Fast chooses between converting a live half-suit into a scored point and leaving it live, comparing two estimates of position value rather than testing a confidence threshold. Let $V(\cdot)$ be the fitted linear evaluation of §7.4, in units of final half-suit differential scaled to $[-1, 1]$; s the current position, Δ the declaring team's half-suit differential, H the candidate half-suit, A the named allocation the policy would declare, and $\hat{q} = \hat{\alpha}(A)$ the Fast estimate (9) that A is correct. The rule declares when

$$\hat{q}\, V\big(s \ominus H,\, \Delta{+}1\big) + (1 - \hat{q})\, V\big(s \ominus H,\, \Delta{-}1\big) \;>\; V(s) + \varepsilon, \tag{12}$$

where $s \ominus H$ denotes the position with H removed and ε is a fitted margin, one of the 34 fitted coordinates of §8.

Both sides of (12) are values of a fitted linear model evaluated on features produced by an approximate transition, so it compares a fitted value estimate under an approximate feature transition; it is not a solution to a Bellman equation, and we make no optimality claim for it. The approximation is in $s \ominus H$, where the implementation updates aggregate team-level quantities only: it removes the half-suit's contributions to expected control, sharpened control, the locked differential and the contested mass, removes its unresolved cards from U, and reduces the declaring team's card count by six. That card-count reduction is applied on *both* the correct and the incorrect branch, although on an incorrect declaration some of those six cards were held by opponents and the true post-declaration counts differ, and player-specific features are not re-derived on either branch. The resulting feature vector is an approximate summary of the successor position rather than the successor position itself.

What distinguishes (12) from a threshold on $\hat{q}$ is which terms survive. The immediate expected score change from declaring is $2\hat{q} - 1$; but a value function fitted on positions in which the half-suit is still live already credits the team's expected control of it, so that term is largely cancelled by the control and locked-differential contributions removed in $s \ominus H$, and what remains to decide the comparison are the positional terms Corollary 4 identifies — the information the six unresolved slots absorb against the turn efficiency of a lighter hand. In the limiting case $\hat{q} = 1$ with H locked, the score term vanishes entirely and (12) is a comparison between exactly those two effects. This is how the rule is constructed; the ablation below is what the construction is worth, and it resolves nothing.

The frozen-policy ablation does not isolate a benefit from this formulation. Replacing (12) with a fixed confidence threshold changed the win rate by +0.12 points, 95% paired CI -1.23–+1.47 (§10.4), and moving the declaration confidence threshold of the shipped configuration between 0.80 and 0.99 also changes nothing the experiment can resolve. One reason the achievable effect is small is the resolution of the model class V is drawn from: a linear function of 16 summary features caps what any rule built on differences of V can extract. The only recorded fit in that class is E14, at $R^2 = 0.2909$ in sample on 405,348 self-play decision points; its coefficients are not the ones compiled into the shipping policy (§7.4, §C), so that figure describes the class and not the deployed vector.

Measured over the primary head-to-head bank of §10.2, v0.4-Fast holds a half-suit that has become locked for 5.16 public events before declaring it, against 14.82 for FishBot v0.3. Three qualifications apply and none can be removed: the moment a half-suit becomes locked is computed from ground truth after the fact and is never available to either policy; both figures are conditioned on the declaration being correct; and the two policies declare correctly at very different rates (98.55% and 82.59% on these same games), so the averages are taken over differently selected subsets. This is measured behaviour, not a theorem: the qualifications leave the magnitude of the difference unusable, and only the sign survives all three (§H). The direction is at least consistent with the two estimators: FishBot v0.3 scores a candidate allocation by a geometric mean of per-card marginals, which needs many more public events to clear a threshold than the joint estimate (9) does, so the earlier declaration is what a sharper allocation probability would be expected to produce. That is an account, not a measurement; no experiment here isolates it.

6.3 Termination

Nothing in the rules of the dialect studied compels a declaration. Consider a position in which every live half-suit is wholly owned by one team or the other, but at least one team cannot resolve which of its own members holds which card: by Theorem 3 card movement ceases, every player being void in every live half-suit owned by the opposing team, so every legal ask is a guaranteed miss. Whether the position is absorbing depends on whether further allocation information can arrive, which by Observation 5 is not settled by Theorem 3 alone, those misses being legal, public and certificate-carrying. A policy whose declaration rule is sensitive to the certificates may eventually declare from such a position; one whose rule is not may not.

Observation 6 (Observed non-termination in mirror self-play)**.** An earlier fitted configuration of the policy, played against a copy of itself with no forcing rule, failed to terminate within the action cap in 21.0% of games at the 400-ask cap of §2. Raising the cap to 1000 asks left the figure unchanged, which distinguishes a cycle from slow convergence (E11, 150 deals in two seat orientations, seed 31). This is a measured property of that fitted configuration on that corpus, not a theorem about the rules dialect.

The remedy adopted is in the policy. A test that forces a declaration as soon as no productive ask remains also fires in ordinary early positions, where a player holding only cards their own team

already owns has no productive ask but ample reason to wait; wiring it in reduced the mirror cap-hit rate to 16.7% but cost 28.3 win-rate points against FishBot v0.3 (E11). The shipped alternative is a graduated escalation on the public event count: below a forcing horizon (12) decides, past it the policy declares any half-suit whose estimated allocation probability exceeds one half, and past a second horizon its best candidate whatever the estimate, an undeclared half-suit scoring nothing. Under this rule every game terminated through play and the action-limit rate is 0.000% in E1, E3, E7, E10 and E13; the exception is the deliberately unforced mirror configuration of Observation 6, measured precisely because it does reach the cap. Both horizons are hand-set constants chosen during development, not the output of any analysis in this paper, and no sensitivity study over them is reported.

A general termination result would require more than we have: either a constructive reachable position, with a strategy profile, in which the full information state as well as the physical position recurs, so that play is provably periodic — Observation 5 is the obstacle, since the certificate stream from legal misses must also be shown to add nothing — or a proof over the joint dynamics of the full information state and the action-selection rule of the policies involved. We have neither, and Observation 6 is evidence only that non-termination is reachable in practice for some fitted configurations. The rules literature already records a forced-claims proposal for tournament play [2], with which these measurements are consistent.

7 v0.4-Fast

7.1 Overview

The FishBot v0.4 policy family is deterministic and fully inspectable, and has four parts: an exact constraint tracker, a posterior over deals (§5), a scored choice over legal asks, and a declaration module. Each member observes only its own hand and the public record, and there is no neural network, no tree search over sampled deals, and no private channel between teammates: everything a teammate learns, an opponent learns at the same instant.

The deployed configuration, evaluated throughout §10, is v0.4-Fast: its belief is the approximate path of §5.6, so it consumes the per-card marginals $\hat{\mu}$ and acts on the allocation probabilities $\hat{\alpha}$, while the exact reference engine of §5 runs only in validation and in the v0.4-Block substitution ablation, never in the deployed inner loop. Each equation below is labelled fitted, approximate, or exact.

7.2 Choosing an ask

On its turn the policy scores every legal ask — a pair $a = (c, t)$ naming a card c in a half-suit of which it holds at least one card, and an opposing target t — and plays the maximiser of

$$U(a) \;=\; \lambda_{\text{lin}} \sum_{k=0}^{19} w_k \, \varphi_k(a) \;+\; \lambda_{\text{val}} \Big[p\, V(s_a^{+}) + (1-p)\, V(s_a^{-}) \Big]. \tag{13}$$

Here $p = \hat{\mu}_{c,t}$ is the Fast posterior probability that the target holds the named card; $\varphi(a) = (\varphi_0, \ldots, \varphi_{19})$ is the feature vector defined in Table 7; $w \in \mathbb{R}^{20}$ are the fitted ask weights; λ_{lin} and λ_{val} are the fitted linear and value mixing weights; V is the value function of §7.4; and s_a^{+}, s_a^{-} are the positions reached after a hit and after a miss. The bracketed term is a one-ply expectimax over V: a hit moves the card into our hand and retains the turn, while a miss excludes the target — renormalising that card's ownership over the remaining candidates, so a miss is informative as well as costly — and hands the turn to the target.

Equation (13) is a *fitted* score, not a derived quantity: the weights w, λ_{lin} and λ_{val} come from the search of §8, and the expectation is taken against $\hat{\mu}$ rather than the exact μ. Ties break deterministically (§C), and the chosen ask's p is retained as the forecast scored in the calibration study of §10.3.

The 20 coordinates of φ are defined in Table 7, placed in §C beside the weights fitted to them so that a definition and its coefficient can be read together. Five are representative. *Hit probability* is p itself, entered linearly and squared so that the fit can curve the trade-off between certainty and value; *lock completion* is the probability that this ask alone completes team ownership of the named card's half-suit H; *reply threat* is $(1 - p)$ times the best ask the target could make against us if the miss hands them the turn; and *information leak*, with a companion coordinate scaling it by how much of H we already hold, indicates that our team has never publicly revealed an interest in H. *Runway* estimates the length of the run this ask begins, since a hit retains the turn, nesting the best remaining per-card posteriors as $p\,(1 + q_1(1 + q_2(1 + q_3)))$ with $q_1 \geq q_2 \geq q_3$ the next-best probabilities; it is an approximation rather than an expectation over a modelled continuation, because it ignores the opponents' replies and any change in the posterior along the run. All are computed from the Fast marginals $\hat{\mu}$.

7.3 Two-ply lookahead with approximate branch beliefs

Equation (13) evaluates both branches of an ask against the *current* posterior, and the continuation and runway features stand in for what happens after it. For the leading candidates the policy instead re-derives a belief on each branch: it ranks all legal asks by (13), keeps the top K, and for each survivor builds the post-hit and post-miss knowledge states (§C), propagates capacities, and recomputes marginals on each.

The recomputation on both branches uses the same Sinkhorn/disjointness approximation as the main path (`engine/src/v04.hpp`, `chooseAsk`, which calls `Belief::sinkhornDisj` on each branch state). Neither branch belief is exact, and this operation is therefore not exact two-ply Bayesian inference; it is a lookahead whose branch beliefs carry the same approximation error as $\hat{\mu}$ itself, measured against the reference engine in §10.1. Writing $\Pr'$ for the recomputed branch marginals, the two summaries taken are

$$\text{follow}(a) = \max_{a'} \Pr'[a' \text{ hits}] \quad \text{over our legal asks in the post-hit position,}$$

$$\text{threat}(a) = \max_{H} \Big(\Pr'[t \text{ holds a card of } H] \cdot \max_{c' \in H} \Pr'[c' \text{ is ours}] \Big) \quad \text{in the post-miss position,}$$

and the candidate is rescored as

$$U_2(a) \;=\; U(a) \;+\; \lambda_{\text{chain}}\, p\, \text{follow}(a) \;-\; \lambda_{\text{threat}}\, (1 - p)\, \text{threat}(a), \tag{14}$$

with the ask maximising U_2 played. The mixing weights λ_{chain}, λ_{threat} and the width K are all fitted coordinates of the parameter vector (§8), so (14) is fitted and approximate on both counts. It is not a determinized search: it stays inside the belief, for the reasons set out in §3.

In the frozen-policy ablations of §10.4, removing the refinement changed the win rate by +1.23 points with a 95% paired interval of -0.53–+3.05 (E5). The interval contains zero, so this experiment does not resolve the refinement's contribution from zero.

7.4 The value function

V maps a vector of summary features of the current position to an estimate of the final half-suit differential, scaled to $[-1, 1]$ and signed from the evaluating team's point of view. It is linear in 16

features — the score and card differentials, three control terms built from e_H, the expected fraction of half-suit H our team holds, counts of what is left to play for, and side to move with its interactions — defined and named in full in Table 10 of §C.

V is fitted by ridge regression on self-play decision points labelled with the eventual half-suit differential, with features collected from *all six seats* at every decision rather than from the mover alone; under mover-only collection the side-to-move feature is constant at +1 and its coefficient — precisely what the miss branch of (13) and the stopping rule (12) depend on — would be unidentified (§C).

The fit reported as E14 has 405,348 rows and an in-sample R^2 of 0.2909 (RMSE 0.3047) on the same self-play corpus that generated the rows; no held-out evaluation of the value model was run. The 16 coefficients compiled into the shipping policy are *not* the coefficients in that artifact, because the freeze step writes only the 34 policy parameters. The two vectors are printed side by side, with the consequences of the gap, in §C; the metrics of E14 therefore describe the artifact's fit and not the coefficients that produced the results in §10.

7.5 Declaring

At every public event each agent is offered the chance to declare, so the full posterior evaluation would run six times per event over every live half-suit. Two cheap pre-gates screen candidates first: `Knowledge::cheapTeamProb` scores a half-suit from capacities and supports alone, with no dynamic program, against `gateTeamProb` (0.008), and if no live half-suit clears it the policy declines to declare without building a posterior at all; a second gate inside `evaluateSet` compares a product of per-card team probabilities against `marginalGate` (0.008) and discards the half-suit before the allocation query is run.

Neither gate is proved to preserve every candidate the ungated evaluation would accept. They are pruning heuristics, and §10.1 reports a corpus-wide false-negative audit (E16) that measures the loss directly: rerunning the full evaluation and the same stopping rule with both gates disabled found 1,017 false negatives among the rejected pairs, and the chosen action differed at 0.0101% of declaration opportunities. The gates are therefore not lossless, and the audit bounds the size of the effect on the primary evaluation corpus rather than eliminating it.

A half-suit that survives the gates is scored, and which estimator does the scoring depends on the belief mode. v0.4-Fast computes $\hat{\alpha}$, the probability that the named allocation is correct, by sequential conditioning over the six cards (§B.8): the chaining is exact in structure, and the approximation is entirely in the marginal solver. v0.4-Block instead reads the exact queries (9) and (10) straight off the block tables, giving τ and α. Under both modes the scored candidate is then passed to the value-based stopping rule (12).

Four quantities can override the stopping rule rather than consult it: the policy is *pressed* when the unresolved pool falls below `patiencePool` cards, when the opposing team holds fewer than `oppCardFloor` cards between them, when its own best available ask has posterior below `askFloor`, or when the public event count passes the forcing horizon — the first three fitted coordinates of the parameter vector, the horizon a fixed configuration constant. When pressed, a plain probability threshold (the fitted `declThreshold`) replaces (12). The ask floor is a softened form of the dead-ask trigger that §6.3 reports as costly in its hard, unconditional form: as a fitted floor combined with a threshold rather than a forced declaration, the optimiser retained it at the value listed in Table 8. The rate at which the floor fires was not measured.

Above all of this sits the graduated escalation of §6.3, which exists because two policies that both defer declaration were measured failing to terminate within the action cap in mirror self-play (Observation 6). Past its second horizon the pre-gates are relaxed to match, so that a half-suit whose allocation is not sharply estimated can still be named.

Two further decisions belong to the policy rather than to the driver: which live teammate receives

the turn when a declaration leaves a seat cardless, and the willingness bit the policy supplies at each level of the forced-endgame ladder of §A.7. Both are described in §C.

8 Fitting

8.1 A population-robust objective

The quantity we optimise is not the mean win rate against a fixed pool but the worst win rate across a panel of opponents, softened so that a noisy search can make progress on it. For a parameter vector w and an opponent panel $\mathcal{O}$, writing $\mathrm{wr}_o(w)$ for the win rate of the configuration w against opponent o on the current seed bank, the fitting objective is

$$\mathrm{score}(w) \;=\; -\frac{1}{\beta}\,\log \sum_{o\in\mathcal{O}} \exp\!\left(-\beta\,\mathrm{wr}_o(w)\right). \tag{15}$$

This tends to $\min_o \mathrm{wr}_o(w)$ as β grows and is smooth in w for finite β. The selection reported here used $\beta = 8$, the default of `engine/select_final.py`. Two other values appear in the repository and are not the reported ones: the tuner command line defaults to $\beta = 10$, and the local response probe of §10.6 used $\beta = 1$, because that search targets a single frozen opponent and the soft minimum degenerates to a mean.

The panel $\mathcal{O}$ used for fitting holds six policies: FishBot v0.3, the turn-starvation lockout, the posterior detective, FishBot v0.2, the adaptive diversifier and the focused hunter. The misdirection artist and the random legal control were excluded from every fitting and selection run so that they remain held-out opponent identities. Two of the eight opponents in the evaluation panel of §10 are therefore held-out identities and six are not, so the head-to-head results measure generalisation to new deals rather than to new strategic populations.

8.2 Optimiser

We use the cross-entropy method with a diagonal Gaussian over the parameter vector. Each generation samples 24 candidates around the incumbent mean, retains the 6 best as the elite set, and smooths the mean and the per-coordinate standard deviation towards the elite statistics. The objective is a noisy win rate estimated from a finite bank of deals, a regime in which optimisers that trust small differences between candidates behave poorly. Every candidate within a generation is evaluated on *identical* seed banks, so within-generation comparisons are paired and the deal draw cancels, and the banks are redrawn between generations, so the search cannot lock onto a particular set of deals.

The parameter vector is fitted jointly and has 34 coordinates: the 20 ask weights $w_0, \ldots, w_{19}$ of Table 7, followed by 14 decision knobs — five thresholds and pool sizes governing declaration and patience, the minimum team probability, the two mixing weights λ_{val} and λ_{lin} of (13), the stopping margin ε of (12), the two policy-prior coefficients θ and ϕ of (11), and the lookahead width K with the chain and threat weights of §7.3. Bounds are imposed per coordinate so that probabilities stay in $[0, 1]$, counts stay non-negative integers, and the lookahead width stays within its implementation limit; every knob is named with its bound in Table 9 of §C.

The value function of §7.4 is not part of this vector. It was fitted separately by ridge regression on self-play decision points and then held fixed while the policy weights were fitted around it, so the search never adjusted the value coefficients and the policy weights are conditional on them.

8.3 Schedule and selection

Fitting ran in five rounds. Round one fitted the 20 ask weights alone against the Fast posterior on a four-opponent panel, from base seed 20260821; round two started from its mean and fitted the joint

vector on the six-opponent panel, from base seed 770077, and is superseded because its coordinate layout predates the two ask features added afterwards; round three re-ran the joint fit on the aligned 34-coordinate vector from base seed 313131; and round four repeated it from base seed 888111 after a correction to the information-leak features. Round five produced the shipping configuration. Its base seed is not recorded in any committed script or artifact; this is a reproducibility gap, and the round can be inspected but not reproduced exactly from the trace `research/v04/runs/tune-round5.jsonl`.

The shipping configuration was chosen on a validation bank the search itself never saw, because the per-generation best reported by the optimiser is a maximum over a population evaluated on shared seeds and is therefore biased upwards. The last six generation means of the round-five trace were re-evaluated on seed 1357911 (500 deals, two rotations, panel {FishBot v0.3, lockout, detective, FishBot v0.2}), and generation 8 won that selection, with a validation soft-min of 0.6096 and per-opponent win rates of 0.756, 0.802, 0.765, 0.819, as recorded in `research/v04/runs/selected.json`. Every seed bank used from §9 onwards is disjoint from the fitting and selection banks, and the configuration was frozen before any of those runs.

Table 8 in §C lists the frozen values. It is a reproducibility artifact and should not be read as a description of strategy: the ask features are correlated with one another and are normalised to different effective ranges, so the sign and magnitude of a single coefficient do not identify the contribution of the corresponding feature. Two of the decision knobs — the locked-half-suit threshold and the patience switch — are inert while the value-based rule of (12) is active, so their fitted values carry no behavioural meaning in the shipping configuration.

9 Evaluation protocol

Fish has a single chance node — the deal — followed by a public transcript, and that structure permits an unusually strong variance-reduction design. This section fixes the experimental unit, the interval construction, the fitting/evaluation separation, and the metrics of §10.

9.1 Six-rotation duplicate blocks

Seats alternate between the two teams, so the cyclic group of six seat rotations of a fixed deal forms a balanced block. The three odd rotations exchange which team holds which hand-triple; the three even rotations permute hands within a team. Playing all six rotations of every deal therefore has each policy hold every hand-triple of that deal exactly once, and the intrinsic advantage of the deal cancels from the comparison rather than merely being averaged down. A matchup reported over D deals is $6D$ games organised into D blocks of six.

This is a stronger control than the two-orientation swap used in the v0.3 study [1], which balances the team label only and leaves the assignment of individual hands within a team uncontrolled. The frozen-policy ablations of §10.4 use two rotations rather than six, for budget reasons; that is stated in the caption of Table 2 and is a weaker block than the primary bank uses.

9.2 Clustered interval construction

The six rotations of one deal are one correlated cluster: they share the same card distribution and differ only in seat assignment. Resampling *games* would treat those six observations as independent and understate the variance. Every interval reported in this paper therefore resamples *deals*: a cluster bootstrap of the per-deal win count for a single arm, and a paired bootstrap of the per-deal difference for a comparison between two configurations played on the same deals, both implemented in `engine/src/arena.hpp` and both taking the 2.5% and 97.5% quantiles of 20,000 resamples.

They are therefore *percentile* bootstrap intervals, with the deal as the resampling cluster, and they inherit the known coverage error of that construction: [77] measures raw percentile intervals failing at 5.7–11.2% against a nominal 5% rate. We keep the percentile construction for two reasons. First, clustering by deal is the correction that matters most in this design, because the within-deal correlation across the six rotations is large and ignoring it would understate the variance by a substantial factor, whereas the percentile-versus-BCa distinction is a second-order adjustment at the sample sizes used here. Second, we have not implemented the bias-corrected and accelerated (BCa) or bootstrap-t corrections, so no claim of nominal coverage is made: the intervals should be read as approximate, and differences whose intervals sit close to zero should not be treated as resolved. Wilson intervals on the raw game counts are computed alongside the bootstrap for continuity with the v0.3 paper; they ignore clustering and are reported only as a reference point.

9.3 Seed separation and what was held out

Fitting used recorded base seeds 20260821 (round 1), 770077 (round 2), 313131 (round 3) and 888111 (round 4). The base seed of round 5, from whose trace the shipping configuration was drawn, is not recorded in any committed script or artifact; this is a reproducibility gap and is documented as one in §G. Selection over the last six generation means of the round-5 trace used a further validation bank, seed 1357911.

Every number reported in §10 comes from a bank disjoint from all of those: 90210 for the head-to-head, the declaration pre-gate audit and the arbitration sensitivity run; 515151 for the round-robin matrix; 606060 for the frozen-policy ablations; 717171 for calibration; 828282, 838383 and 848484 for the rule dialects; 20260820 for the v0.3 port validation; 959595 for the belief substitution under an unfitted policy; 464646 for action-cap incidence; 31415 for the value-function fit; 20260822 for the brute-force oracle; and 515253 (response fitting) with 6543210 (response evaluation) for the local response probe. The configuration was frozen before any evaluation bank was run.

That separation is narrower than "held out" usually implies: deals were held out — no deal used in any reported experiment appears in any fitting or selection bank — but most opponent identities were not, and §10.2 states the consequence for the evaluation panel and scopes every result of §10 accordingly.

9.4 Metrics

Win rate is the primary metric, but it is not sufficient on its own: a policy can win while converting a smaller fraction of its asks, and §10.2 reports a matchup in which exactly that happens. For every matchup we therefore also report *mean half-suits* won out of nine, giving the margin and not only the sign of the result; *ask accuracy*, the fraction of asks that transfer a card; and *declaration accuracy and rate*, declarations per game and the fraction scored correctly. The engine tallies voluntary and forced endgame declarations separately; the win-rate tables report the voluntary counters only, the calibration study pools the two, and forced declarations run at well under a tenth per game on the primary bank, so that contamination is small but is not zero. *Out-of-turn declarations* are reported per game; they are impossible under the v0.3 dialect, which permits declarations only on turn.

Forecast calibration is the Brier score, log loss, expected calibration error and a decile reliability table for the policy's own Pr[ask succeeds] and Pr[allocation correct] forecasts, descriptive properties of the estimators computed as point estimates without clustered uncertainty. The *worst-case profile* is the minimum win rate over the opponent panel, not only the mean: a policy that averages well and collapses against one playstyle is not well described by its mean. The *local response probe* is a policy fitted specifically against a frozen target, inside the same policy class and with the same optimiser and budget; the score such a response achieves is an achieved *lower* bound on the exploitability of

the frozen target within the searched policy class, since a larger budget, a better optimiser or a policy class outside the one searched could all achieve more. It is a restricted-policy-class diagnostic and not an equilibrium computation.

10 Results

10.1 Engine and belief validation

Information-safety audit (E1). The verification sweep plays the ordered cross-product of nine policies with the knowledge audit of §4 active and recorded 0 violations in 23,594,580 soundness checks: no derived knowledge state, capacity or certificate was ever contradicted by the actual deal. Card conservation, resolution of all nine half-suits and bit-identical replay hold in every game, and the action-limit rate is 0.000%. The same run under the v0.3 dialect records 0 violations in 10,910,844 checks and, again, no action-limit game. E13 measures action-cap incidence over 300 deals in six seat rotations, seed 464646, against every panel member and the v0.4-Fast mirror, and records 0.000% on every row.

Belief cross-checks (E2). Over 40 games and 293,524 comparisons the exact reference engine agrees with the card-level capacity dynamic program to 3.414×10^{-15} with no certificate live — machine precision, as it must, the two computing one quantity two ways — and with exact rejection sampling to 4.066×10^{-2} with certificates live, Monte-Carlo error at these sample sizes. The Fast approximation has mean absolute marginal error 0.01705 and maximum 0.4982: accurate on average, with a heavy tail that E2 does not localise to any class of position.

Brute-force allocation oracle (E15). E2 compares two structured algorithms against each other and does not establish that either enumerates the correct collection of assignments. E15 tests that directly against an enumerator using no dynamic programming, no factorisation and no sampling; the procedure, its per-state cut-off of 200,000 consistent deals and the sampler comparison are in §B.

Every deterministic comparison (Table 6, §B.5) differs by exactly zero, which is exact rather than approximate agreement because both sides are integer counts in double precision; of 15,235 checks on the chosen allocation 0 were inconsistent and none failed to be the argmax of α, and sampler frequencies fall within 0.0532 of the exact marginals over 46,345,121 draws. Coverage is partial by construction: 75,203 states, more than were enumerated, were skipped as too large, and skipping is correlated with what makes a state hard, a large space of unresolved cards. E15 validates the reference engine on small reachable states, not the whole state space.

Declaration pre-gate audit (E16). Two cheap pre-gates screen declaration candidates before the full posterior evaluation runs (§7.5), and neither is proved to be an upper bound on the quantity it screens. E16 re-runs the full evaluation and the same stopping rule with both gates disabled, on every live half-suit at every declaration opportunity of the primary bank (§D).

Of 58,530,669 (opportunity, live half-suit) pairs over 10,102,149 opportunities, 24,114,786 (41.20%) were gated out, and 1,017 of those — 0.00422% — would have been declared by the full rule; the gated path makes 332,645 declarations against 333,661 ungated, choosing differently at 1,016 opportunities, 0.0101% of the total and 0.30% of the ungated rule's declarations (Table 13, §D.6). The pre-gates are therefore not lossless: they are a pruning heuristic with a measured false-negative rate of roughly one in ten thousand declaration opportunities, specific to this policy, this corpus and the compiled thresholds. The audit bounds the size of that effect rather than eliminating it, and does not license describing the gates as safe or as a proved upper bound.

Port validation (E8). Every comparison in this paper is made against a port of FishBot v0.3 into the new engine. Under the v0.3 dialect that port reproduces six of the seven published figures within their intervals (Table 17, §E.3). The row for the misdirection artist misses by half a point, near the top of a scale on which both figures exceed 98%.

10.2 Primary result: v0.4-Fast against FishBot v0.3

On the primary evaluation bank of 700 held-out deals played in six seat rotations for 4,200 games, v0.4-Fast defeated FishBot v0.3 in 3,153 games — a 75.07% win rate with a deal-clustered 95% interval of 73.71–76.40% — taking 5.457 of the nine half-suits on average.

Table 15 converts the round-robin — reproduced cell by cell as Table 16 in §E.1 — into Bradley–Terry ratings: v0.4-Fast is rated +216 on a scale where FishBot v0.3 is zero, against +8 for the posterior detective and +5 for the turn-starvation lockout, so those two sit within a few Elo of FishBot v0.3. Such ratings are not invariant to redundancy in the population and cannot represent cyclic dominance [75], so the table summarises this panel rather than giving a transferable strength scale.

10.3 Calibration

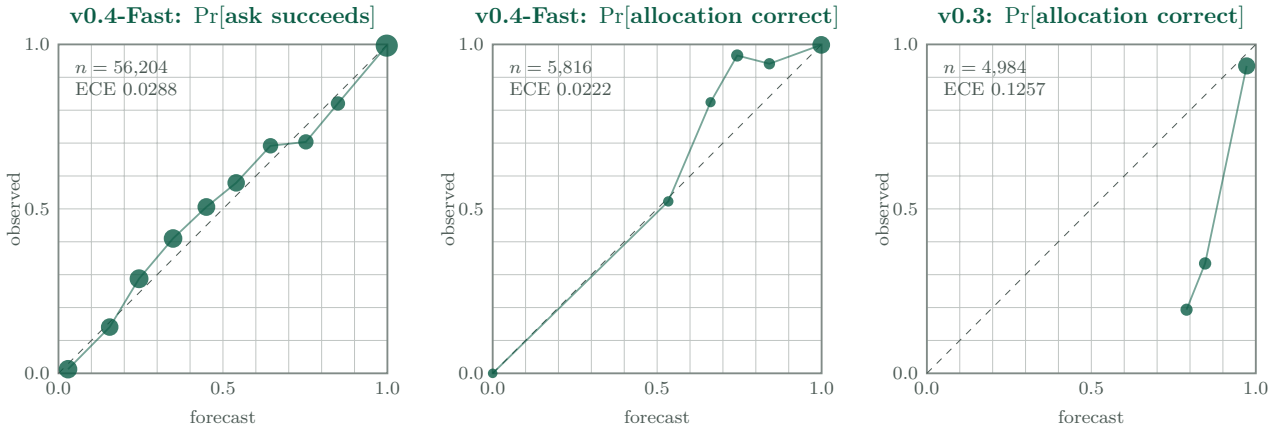

Figure 2: Reliability diagrams (E6) for the two forecasts v0.4-Fast exposes and for FishBot v0.3's declaration forecast. Population: the calibration bank of 600 deals, seed 717171, v0.4-Fast against FishBot v0.3, weights frozen; the unit is one forecast. Marker size increases with bin count on a compressed scale, so it orders the bins rather than measuring them. Points above the diagonal are underconfident, below it overconfident. No clustered intervals are computed.

Table 1: Aggregate forecast reliability (E6). Population: the calibration bank of 600 deals, seed 717171, v0.4-Fast against FishBot v0.3, both frozen; the unit is one forecast and n counts forecasts of that type. FishBot v0.3 exposes no ask forecast, so only the declaration rows are comparable. All entries are point estimates: no clustered uncertainty for Brier score, log loss or ECE. Voluntary and forced declarations are pooled here, unlike in Table 14.

Policy	Forecast	n	Brier	Log loss	ECE	Mean pred / obs
v0.4-Fast	Pr[ask succeeds]	56,204	0.1297	0.3790	0.0288	0.5338 / 0.5504
v0.4-Fast	Pr[allocation correct]	5,816	0.0155	0.0592	0.0222	0.9575 / 0.9789
FishBot v0.3	Pr[allocation correct]	4,984	0.1339	0.3990	0.1257	0.9459 / 0.8202

Table 1 gives the aggregate scores, Figure 2 the reliability diagrams and §F the decile breakdown. Every calibration score in this paper — here, in the appendix, and wherever one is quoted elsewhere

— is a point estimate with no clustered uncertainty, so small differences between the estimators are not resolved by these numbers.

v0.4-Fast's declaration aggregate is dominated by one bin: 85.7% of the 5,816 forecasts fall in the top decile, close to 1 and almost exactly right, which is most of what an expected calibration error of 0.0222 against realised accuracy 97.89% measures. The decision-relevant bins are the middle ones, and in $[0.6, 0.9)$ the forecast is *under*confident — realised accuracy exceeds predicted probability by 10.0 to 22.3 points — on counts of 74, 415 and 221, individually noisy and carrying little aggregate weight. Whatever the stopping rule of §6.2 does there, the aggregate score does not characterise it. FishBot v0.3 is the contrasting case on the same bank, expected calibration error 0.1257 running in the overconfident direction, mean predicted 94.59% against realised 82.02%. That is a descriptive comparison of two estimators on the same games, not a causal explanation of either policy's behaviour, and it does not establish why the declaration-accuracy difference of §10.2 exists.

10.4 Which mechanisms matter: frozen-policy ablations

Each ablated variant plays the same 500 deals in two seat rotations against the same four-policy panel with the fitted weights left *unchanged*, so each row measures how the fitted policy behaves when one component is removed from it, not the standalone causal value of that component in a policy refitted around its absence: a cheap removal may still be doing work a refitted policy would find another way to do, and an expensive removal may be expensive only because the surrounding weights were fitted to rely on it. No matched-budget refit was run (§12). That qualification applies to every row below and to §D and is not repeated per row; all ablation intervals are paired percentile bootstrap over deals, 20,000 resamples.

Belief. The three rows that degrade the belief itself are the largest in Table 2 by an order of magnitude over anything else, and none of them isolates a single constraint. The fitted prior of (11) is applied only along the Fast path (`engine/src/belief.hpp`), so every one of these rows removes the prior together with whatever else it removes. Keeping the Sinkhorn solver and dropping the certificates and the prior costs +52.78 points (CI +50.98–+54.55), leaving 24.73%; making the same removal with the exact capacity dynamic program in place of Sinkhorn costs +48.90 points (CI +47.00–+50.78), leaving 28.60%; dropping capacity conditioning as well costs +56.03 points (CI +54.27–+57.75), leaving 21.48% and the largest decrease in the table. The prior alone is worth +3.88 points, so the certificates account for most but not all of each difference, and no row in this experiment separates the two. None of the three uses the exact reference engine — all substitute a degraded approximate belief — and only the v0.4-Block row substitutes the engine of §5.

Prior and ask features. Removing the fitted policy-aware prior costs +3.88 points (CI +2.08–+5.73), leaving 73.62%; the fitted policy depends on it. Of the ask-side rows, which probe single coordinates of the 20-weight ask score and the lookahead terms, only the lock-completion weight has an interval excluding zero: +3.20 points (CI +1.55–+4.85), leaving 74.30%. Every other ask-side interval contains zero — hit probability +1.55 (CI -0.25–+3.35), two-ply +1.23 (CI -0.53–+3.05), information leak +0.12 (CI -1.38–+1.65) — and three are negative point estimates: one-ply value -0.33 (CI -1.88–+1.23), reply threat -0.35 (CI -1.98–+1.27), runway -0.35 (CI -1.65–+0.97). Correlated features produce exactly this pattern, one coordinate's removal being absorbed by the others, and a paired design controls the deals but not the collinearity; we keep all three, because dropping a component on a non-significant ablation is selection on noise.

Declaration. Every declaration-side row is unresolved from zero: the value-based stopping rule against a fixed threshold at +0.12 points (CI -1.23–+1.47), the threshold settings 0.80 and 0.99 at

Table 2: Frozen-policy ablations (E5). Population: 500 deals in two seat rotations against the four-policy panel {FishBot v0.3, turn-starvation lockout, posterior detective, FishBot v0.2}, 1,000 games per opponent, seed 606060, full rules dialect. Fitted weights are frozen at the shipping values in every row; only the named component changes. "Full − ablated" is the reference win rate 77.50% minus the row's, so positive means removal hurt. Intervals: paired percentile bootstrap over deals. The three belief rows at 21–29% substitute weaker approximate beliefs, not the exact reference engine; only the v0.4-Block row uses that. The two policy-prior rows carry identical figures by construction rather than by coincidence, because the Fast belief is already the default (§D).

Group	Change (policy weights frozen)	Win rate	Full − ablated	95% paired CI
	v0.4-Fast (reference configuration)	77.50%	—	—
Belief	Drop capacities (C4) and the certificates (C5) and the prior; uniform over the surviving support	21.48%	+56.03	+54.27–+57.75
	Drop the certificates (C5) and the prior; Sinkhorn over C1–C4	24.73%	+52.78	+50.98–+54.55
	Drop the certificates (C5) and the prior; exact capacity dynamic program over C1–C4	28.60%	+48.90	+47.00–+50.78
	v0.4-Block belief substituted into the Fast-fitted policy (exact C1–C5, no prior)	71.30%	+6.20	+4.35–+8.05
	Fast belief with the policy prior off (matches the reference engine's prior-free belief)	73.62%	+3.88	+2.08–+5.73
	Remove the fitted policy-aware prior	73.62%	+3.88	+2.08–+5.73
Ask	Zero the lock-completion weight	74.30%	+3.20	+1.55–+4.85
	Zero the hit-probability weight	75.95%	+1.55	-0.25–+3.35
	Remove the two-ply refinement	76.28%	+1.23	-0.53–+3.05
	Zero both information-leak weights	77.38%	+0.12	-1.38–+1.65
	Remove the one-ply value lookahead	77.83%	-0.33	-1.88–+1.23
	Zero the reply-threat weight	77.85%	-0.35	-1.98–+1.27
	Zero the runway weight	77.85%	-0.35	-1.65–+0.97
Declaration	Declaration threshold 0.99	77.30%	+0.20	-0.20–+0.60
	Name the allocation by conditioned greedy MAP	77.33%	+0.18	-0.12–+0.47
	Fixed threshold in place of the value-based stopping rule	77.38%	+0.12	-1.23–+1.47
	Declaration threshold 0.80	77.48%	+0.03	-0.18–+0.22
	Cash locked half-suits immediately (inert while the stopping rule is active)	77.50%	+0.00	+0.00–+0.00

+0.03 (CI -0.18–+0.22) and +0.20 (CI -0.20–+0.60). The "cash locked half-suits immediately" row is zero by construction rather than by measurement: the value-based rule decides first, so the cashing rule never fires. With the belief rows and the declaration-accuracy difference of §10.2, these support only the claim that the *joint* allocation probability matters, which is what the certificates and the capacity conditioning sharpen. They do not establish that the stopping formulation of §6.2 outperforms a well-calibrated fixed threshold; this experiment lacks the power to resolve that.

Belief substitution. Substituting the v0.4-Block belief — the exact reference engine — into the Fast-fitted policy costs +6.20 points (CI +4.35–+8.05), leaving 71.30%. The matched comparator — the Fast belief with the policy prior switched off, so that it is prior-free as the reference engine is — costs +3.88 points on its own. These are two separate frozen-policy measurements and do not decompose the +6.20-point difference additively; the remainder is in the same direction and no interval is reported for it. Repeating the substitution under a deliberately unfitted posterior-greedy policy (E12, 400 deals in six seat rotations, 2,400 games per arm), which removes the confound of weights fitted around Fast beliefs, gives 49.29% (CI 47.29–51.25%) against 48.17% (CI 46.29–50.04%),

deal-clustered percentile bootstrap: same direction, overlapping intervals, so E12 does not resolve the question either. The fitted policy is therefore not invariant to the belief representation. This does not establish that an exact-belief policy would be weaker after matched retraining, which would require separate matched-budget fits that we did not run; as an analogy only, [25] report a comparable observation in Skat, where more accurate inference did not translate into better play under a policy tuned around the cheaper estimate.

10.5 Robustness: rules dialects and arbitration

The margin over FishBot v0.3 depends on the dialect (E7, 400 deals in six seat rotations, 2,400 games per row, deal-clustered percentile bootstrap, tables in §E): it falls to 64.46% (CI 62.58–66.38%) under the v0.3 dialect, which permits declarations only on turn, to 72.54% (CI 70.79–74.21%) under the 48-card, eight-half-suit form, and to 67.71% (CI 65.92–69.50%) with out-of-turn declarations disabled. Comparator rows do not move uniformly downwards — against the lockout, disabling out-of-turn declarations *raises* the win rate to 80.17% — and since E7 and E3 differ in bank as well as dialect, these rows bound the size of the dialect dependence rather than isolating the rule's contribution.

Arbitration of simultaneous voluntary declarations is a modelling choice rather than a rule of the game (§2). Replacing the default lowest-seat order by highest seat or by a scan from the turn-holder moves every win rate by at most 0.7 points, with overlapping deal-clustered percentile bootstrap intervals throughout (E17, 700 deals in six seat rotations, tables in §E); against FishBot v0.3 the alternatives give 74.93% and 74.38% for a default of 75.07%, so both put the worst panel result marginally below the 75.07% of §10.2, which is why that figure is scoped to the default order. What the choice does move is the out-of-turn declaration rate, from 3.4–3.6 per game under either seat order to 2.06–2.12 under the turn-holder scan, declarations per game unchanged. Both checks are within-dialect and within-panel.

10.6 Local response probe

Table 3: Local response probe (E10). Each row fits a fresh policy in the same policy class, with the same optimiser and budget, whose only objective is to beat the frozen policy named; fitting used seed 515253, evaluation a separate bank of 600 deals in six seat rotations, 3,600 games per row, seed 6543210, target weights frozen. The win rate is what the response achieved, so lower is better for the frozen policy. Intervals: deal-clustered percentile bootstrap, 20,000 resamples. Action-limit rate 0.000% in all three probes.

Frozen target	Response win rate	95% CI	Mean half-suits conceded
v0.4-Fast	51.19%	49.67–52.72	4.550
FishBot v0.3	77.31%	75.97–78.67	5.547
Posterior detective	76.47%	75.11–77.81	5.471

A response fitted specifically against frozen v0.4-Fast achieved 51.19% (CI 49.67–52.72%, Table 3); the interval includes 50%, so this search did not resolve an advantage in either direction. The identical procedure achieved 77.31% (CI 75.97–78.67%) against FishBot v0.3 and 76.47% against the posterior detective, so it is sensitive enough to find large exploits when they exist; the action-limit rate is 0.000% in all three probes, the v0.4-Fast mirror included. As §9.4 sets out, an achieved response score is a *lower* bound on the exploitability of the frozen target within the searched class [71]: a restricted-policy-class diagnostic, not evidence of an equilibrium.

Table 4: Whole-game throughput (E9). Population: self-play games under the full rules dialect with policy weights frozen; the unit is one complete game and the last column gives how many were timed. The v0.4-Fast and v0.4-Block rows differ only in the belief path. No intervals; these are wall-clock rates for one timing run each.

Configuration	Games/s	Games timed
v0.4-Fast (deployed)	162.8	400
v0.4-Block (exact reference belief)	11.9	120
FishBot v0.3	6,954	800

10.7 Throughput

v0.4-Fast runs at 162.8 games/s against 11.9 for v0.4-Block, a factor of about 14×; FishBot v0.3, whose posterior carries neither capacity conditioning nor ask-legality certificates, runs at 6,954 games/s. That Fast-to-Block factor is why the exact reference engine validates probabilities and serves as an ablation arm rather than running in the deployed inner loop: at 11.9 games/s the primary bank of 4,200 games per row would dominate the experimental budget. Every evaluation run reported in §10 completes in minutes at these rates; the fitting rounds of §8 and the exhaustive enumeration of E15 do not, and their costs are not reported here.

11 Discussion

11.1 Whether one policy can be best across playstyles

The answer has two parts, one for each half of the agent: the inference component, derived from the rules, and the decision component, fitted against a panel of opponents.

The inference half is playstyle-independent. Constraints (C1)–(C5) of §5 are logical consequences of the dialect specified in §2, so they are true whatever policy the observed player is running, and no opponent behaviour can make them false. Nothing in the observer-conditioned deal posterior is fitted to an opponent, and the same is true of Theorem 3, which is a statement about which asks the rules permit rather than about how anyone chooses among them. Consistent with this, the frozen-policy ablations that remove parts of the constraint set produce the largest win-rate decreases recorded in Table 2: dropping the ask-legality certificates gives a full-minus-ablated difference of +52.78 points, with a 95% paired percentile bootstrap interval over deals of +50.98–+54.55. That row removes the fitted prior along with the certificates, and it is a frozen-policy sensitivity rather than the standalone causal value of either (§10.4).

The decision half cannot be a best response to every opponent at once. Choosing among asks trades off quantities — the cost of handing a particular opponent the turn, the information an ask reveals, when to declare a half-suit — whose values depend on what the other five seats do next. A best response to a policy is a function of the policy it responds to, so no single deterministic policy is simultaneously a best response to every opponent, and no experiment in this paper could establish that one was. The defensible target is therefore minimal exploitability rather than dominance over an arbitrary opponent. For a game with two three-player teams, no communication channel, and a shared team payoff, the appropriate solution concept is a team-maxmin equilibrium with correlation [36, 38], which we do not compute and which is out of computational reach at this game size. The fitting objective (15) is a deliberate finite-panel approximation of it: a soft minimum over a hand-built six-opponent panel rather than a minimum over the whole policy space.

What the evidence supports. Within the eight-policy panel of Table 14, one policy is ahead of every member simultaneously: v0.4-Fast's lowest win rate against any panel member is 75.07%, recorded against FishBot v0.3 on the primary bank under the dialect of §2 and its default arbitration

order, two alternatives to which put it slightly lower (§10.5). That statement is bounded by the panel, and it measures generalisation to new deals rather than to new strategic populations (§10.2). The local response probe is inconclusive in the other direction: a response fitted specifically to frozen v0.4-Fast reached 51.19% with a 95% deal-clustered percentile bootstrap interval of 49.67–52.72 that includes 50%, and an achieved exploiter score is in any case a lower bound within the searched class (§10.6). The two results are not in tension: the fitted policy is uniformly ahead of the opponent population we were able to construct, and its exploitability by a purpose-built response remains unresolved.

11.2 Interpreting the belief-substitution result

Substituting the exact reference belief into a policy fitted around Fast beliefs cost +6.20 points, with a 95% paired percentile bootstrap interval over deals of +4.35–+8.05 (E5, §10.4). The fitted policy is therefore not invariant to the belief representation; that is not evidence that an exact-belief policy would be weaker after matched retraining, and no matched fit was run. Two mechanisms are consistent with the observation, and this study cannot separate them.

The first is threshold placement. The 14 decision knobs of §7 — among them the declaration threshold, the minimum team probability, the two cheap gate thresholds and the stopping margin — were fitted against the distribution of scores produced by the Fast estimator. E2 records a mean absolute difference of 0.01705 and a maximum of 0.4982 between Fast and block per-card marginals, so the reference engine presents a differently distributed input to knobs placed at quantiles of the Fast distribution. Under that account the loss is a calibration mismatch between fitted constants and a substituted estimator, and refitting would remove it.

The second is rank tolerance. The ask rule scores the legal ask candidates enumerated each turn and takes an argmax, which is insensitive to perturbations that do not reorder the leaders, so a more accurate estimator can change the chosen ask only where the ranking is close. Whether the largest Fast–Block deviations coincide with close rankings is a conjecture rather than a measurement: E2 records the size of the deviations but no position-conditioned statistic. E12 does not resolve the question either — under a deliberately unfitted posterior-greedy asking rule the two belief paths reached 49.29% and 48.17%, with 95% deal-clustered percentile bootstrap intervals of 47.29–51.25 and 46.29–50.04, a gap in the same direction as the fitted comparison but with substantially overlapping intervals.

Settling the question needs the matched-budget pair of fits described in §12: the cross-entropy procedure of §8 run twice from the same seeds and generation count, once with each inference path in the inner loop. At the reference engine's 11.9 games per second (E9) that is expensive but not infeasible; it was not attempted here.

11.3 Implications for human play

Three of the mechanisms this analysis relies on are cheap enough to apply at a table without a posterior over deals. An ask is a confession: under the dialect of §2 it establishes that the asker lacks the named card and holds some other card of its half-suit, and a miss establishes that the target lacks the named card, which are constraints (C1)–(C3) and (C5) of §5 applied by hand. Public hand sizes then close those exclusions into certainties — the capacity constraint (C4) applied by hand — and the single frozen-policy ablation that removes capacity conditioning produced the largest win-rate decrease in Table 2. A half-suit held entirely by one team cannot change hands, so waiting carries no risk to ownership (Theorem 3, Corollary 4), but no experiment here isolates the profit of waiting: the "cash locked half-suits immediately" row is inert by construction, and the locked-hold times of §6 are observational. §H states these three mechanisms in full and adds the ask-ordering, declaration and endgame material, tagging each item by whether it is a consequence of the rules, a measurement, or an untested suggestion.

12 Threats to validity

Limitations are grouped by the kind of validity each threatens; caveats stated in full at the point of measurement are cross-referenced rather than restated.

Internal validity. Every ablation is a frozen-policy ablation with the 34 fitted coordinates left as selected, so no row measures a component's standalone value (§10.4). The 20 ask features are correlated and the paired design controls the deal, not the collinearity, so a component can look essential because the surrounding weights lean on it, or redundant because a correlated term covers it. Substituting the reference belief compounds this, changing the inference, every feature computed from it and the constants they are compared against (§11). No retrained ablation was run for any row.

Construct validity. The posterior is exact with respect to the rules and the uniform prior over consistent deals, not to opponents' policies; (11) is a parametric stand-in for that correction, not the correction (§5.7). The declaration rule (12) compares fitted linear value estimates under an approximate team-level feature transition rather than true continuation values, and is not a Bellman-optimal stopping value (§7.5). The two declaration pre-gates are pruning heuristics with a measured, non-zero false-negative rate (§10.1). And win rate, the primary metric, does not capture the half-suit differential, declaration reliability or calibration reported beside it (§9.4).

External validity. Deals were held out from fitting and selection; opponent identities largely were not (§10.2, §9.3). Every opponent was written by us or ported from v0.3, and with no expert human logs for this dialect and no independently authored Fish engine to play, nothing tests the agent against a strategy source outside this project.

Statistical validity. Win-rate intervals are percentile bootstrap intervals over 20,000 resamples clustered on the deal (§9.2); such intervals under-cover in evaluations of this kind [77], and neither BCa nor bootstrap-t was implemented. The calibration metrics carry no clustered uncertainty at all (§10.3). Many rows of Table 2 have intervals including zero: the study was not powered below about a point, and an unresolved row is an absence of evidence, not evidence of absence.

Opponent-population dependence. Leading every member of this panel is not being unexploitable, and nothing here bounds exploitability from above: the response probe searches one parametric class with one optimiser at a fixed budget, so its score is a lower bound within that class (§10.6), and a differently shaped adversary or a wider search could do better.

Rules-dialect dependence. All results are for the dialect of §2, given in full in §A; §E varies three rules and the three arbitration orders, not every consequential choice, and arbitration by lowest seat is itself a modelling choice, not a rule. The asks inside a locked half-suit that only the owning team may make are a channel the fitted configuration has no term for (§6): unexploited here, not shown to be worthless.

Computational and model-class limits. The policy class is small — a 20-feature linear ask score, 14 decision knobs and a 16-feature linear value function — which caps what the one-ply lookahead and the declaration rule built on it can extract. No goodness-of-fit figure is reported for the deployed value coefficients: the in-sample $R^2 = 0.2909$ of E14 belongs to the artifact's ridge fit, whose coefficients are not the compiled ones, and neither vector was evaluated out of sample (§C). Termination is imposed by hand-set escalation horizons rather than derived, and a differently tuned pair of policies could cycle inside them (§6.3). The brute-force oracle validates only small reachable states, 75,203 having been skipped as too large (§10.1). And the base seed of fitting round 5, which produced the shipping configuration, is in no committed script or artifact, so that round cannot be reproduced from scratch (§8).

Experiments that would resolve the open questions, each targeting one limitation above, none of which was run:

- Matched-budget fits against the Fast path and against the reference engine on a common held-out

bank, in place of the non-invariance result.
- Retrained ablations for the C5 certificates, the policy prior and the belief mode.
- A declaration-timing experiment sized for effects below a point.
- Deal-clustered intervals for the calibration metrics.
- An opponent family excluded from all fitting and selection, ideally an independently authored engine or expert human logs.
- A held-out corpus and a goodness-of-fit figure for the deployed value coefficients.
- For termination, a constructive reachable recurrent state whose information state also repeats.

13 Conclusion

This paper constructed four things: an information-safe simulator for an explicitly specified dialect of six-player Canadian Fish, each consequential rule a switch and each agent's view audited against the public record (§4); an exact observer-conditioned deal posterior, closed-form because card movement in Fish is public so the hidden state is exactly the initial deal, giving ownership marginals, team-ownership and named-allocation probabilities and validated against a brute-force enumeration oracle on small reachable states (§5, §10.1); the cheaper approximation of it that the deployed v0.4-Fast policy runs, beneath fitted ask, lookahead and declaration machinery (§7); and a duplicate-block protocol — every primary-bank deal in six seat rotations, two on the ablation bank, intervals bootstrapped with the deal as the resampling cluster (§9).

Four findings are empirical. v0.4-Fast defeated FishBot v0.3 in 3,153 of 4,200 games — 700 held-out deals in six seat rotations — at 75.07%, deal-clustered 95% interval 73.71–76.40%: its lowest rate against any member of this eight-policy panel, on this bank, under this dialect and the default arbitration order, six of the eight also being fitting-panel members (§10.2). The margin is declaration reliability rather than ask conversion: on the same games v0.4-Fast declared correctly 98.55% of the time against 82.59% for FishBot v0.3, at expected calibration error 0.0222 against 0.1257 (§10.3); no experiment decomposes that margin causally. Among the frozen-policy ablations, the rows removing parts of the rules-derived constraint set, the policy prior, the lock-completion weight and the v0.4-Block belief substitution are resolved from zero; the declaration timing rule, the two-ply refinement and the information-leak weights are not, and support no conclusion in either direction (Table 2). And the two cheap declaration pre-gates are a pruning heuristic, not a proved bound: over the primary evaluation corpus they rejected 1,017 candidate pairs the ungated rule would have declared — a measured, non-zero false-negative rate (E16, §10.1).

Three readings are suggested by the analysis without being established by it. The declaration-accuracy difference appears carried by the joint allocation probability rather than by the timing rule above it: a fixed threshold in place of the fitted stopping rule moved the win rate +0.12 points, 95% paired interval -1.23–+1.47 (E5) — an interval that resolves nothing, beside a large and consistent reliability difference. Substituting the v0.4-Block belief cost +6.20 points, 95% paired interval +4.35–+8.05, so a policy fitted around one belief representation does not transfer to the other unchanged; that does not show an exact-belief policy would be weaker after matched retraining, and no matched fit was run. And ownership of a half-suit can freeze while allocation information about it keeps arriving, asks inside a locked half-suit remaining legal for the owning team and still carrying certificates (§6): waiting is safe with respect to ownership without being profitable, and this study does not isolate its value.

More remains unresolved than settled. Exploitability is not bounded: the response probe returns a lower bound within one parametric class, its interval including even money (§10.6). No team equilibrium was computed. Termination is imposed by hand-set horizons rather than derived, and the observed cycling is a property of specific fitted configurations in self-play, not of the game. Nothing

was tested against expert human play or an independently authored engine. The open question the belief-substitution and response-probe results jointly raise is whether a separately fitted exact-belief agent, a policy class broader than a linear ask score with a linear value function, or an equilibrium-oriented method can improve on the current v0.4-Fast policy.

References

[1] FishLab Research Project, "FishBot v0.3: A Count-Conditioned, Empirically Tuned Policy for Six-Player Canadian Fish," project technical report, 2026. Project repository: https://github.com/dylann29/fish-optimization

[2] J. McLeod, "Literature," *Pagat*, 2006–2025, last updated 1 July 2026. https://www.pagat.com/quartet/literature.html

[3] M. Develin, "Canadian Fish," chapter 9 of *Card Games*, author's online manual. https://web.archive.org/web/20170720075756/http://bantha.org/~develin/cardgames.html#ch9

[4] Deposit Genius, "Literature Game Strategy (Canadian Fish)," online article. https://depositgenius.com/literature-strategy-canadian-fish/

[5] Canadian Fish Project, "Strategy notes," 2026. https://canadian-fish.vercel.app/strategy

[6] Wikipedia, "Literature (card game)," online encyclopedia entry. https://en.wikipedia.org/wiki/Literature_(card_game)

[7] N. Somani, "literature: card game implementation and learning bot," GitHub repository, MIT licence, Python, 2018–. https://github.com/neelsomani/literature

[8] N. Somani, "literature-server," GitHub repository, with a live deployment at https://literature.neelsomani.com/ https://github.com/neelsomani/literature-server

[9] "Literature: Fish Card Game" (`com.cards.game.literature`), Google Play store listing, closed source. https://play.google.com/store/apps/details?id=com.cards.game.literature

[10] D. Zha, K.-H. Lai, S. Huang, Y. Cao, K. Reddy, J. Vargas, R. Wei, J. Guo, and X. Hu, "RLCard: A Toolkit for Reinforcement Learning in Card Games," arXiv:1910.04376, 2019.

[11] D. Levy, "The Million Pound Bridge Program," *Heuristic Programming in Artificial Intelligence: The First Computer Olympiad*, Ellis Horwood, 1989.

[12] M. L. Ginsberg, "GIB: Imperfect Information in a Computationally Challenging Game," *Journal of Artificial Intelligence Research* 14:303–358, 2001.

[13] I. Frank and D. Basin, "Search in Games with Incomplete Information: A Case Study Using Bridge Card Play," *Artificial Intelligence* 100(1–2):87–123, 1998.

[14] I. Frank, D. Basin, and H. Matsubara, "Finding Optimal Strategies for Imperfect Information Games," *AAAI-98*, pp. 500–507, 1998.

[15] J. Long, N. R. Sturtevant, M. Buro, and T. Furtak, "Understanding the Success of Perfect Information Monte Carlo Sampling in Game Tree Search," *AAAI-10*, pp. 134–140, 2010.

[16] R. Pavlicek, "Dealing with Constraints," *Bridge with Pavlicek*, online article. https://www.rpbridge.net/8h11.htm

[17] P. I. Cowling, E. J. Powley, and D. Whitehouse, "Information Set Monte Carlo Tree Search," *IEEE Transactions on Computational Intelligence and AI in Games* 4(2):120–143, 2012.

[18] P. I. Cowling, D. Whitehouse, and E. J. Powley, "Emergent Bluffing and Inference with Monte Carlo Tree Search," *IEEE Conference on Computational Intelligence and Games 2015*, pp. 114–121, 2015.

[19] D. Whitehouse, P. I. Cowling, E. J. Powley, and J. Rollason, "Integrating MCTS with Knowledge-Based Methods to Create Engaging Play in a Commercial Mobile Game," *AIIDE 2013*, 2013.

[20] J. Goodman, "Re-determinizing Information Set Monte Carlo Tree Search in Hanabi," arXiv:1902.06075, 2019.

[21] V. Lisý, M. Lanctot, and M. Bowling, "Online Monte Carlo Counterfactual Regret Minimization for Search in Imperfect Information Games," *AAMAS 2015*, pp. 27–36, 2015.

[22] I. Zuckerman, A. Felner, and S. Kraus, "Mixing Search Strategies for Multi-Player Games," *IJCAI-09*, pp. 646–651, 2009.

[23] M. Buro, J. R. Long, T. Furtak, and N. R. Sturtevant, "Improving State Evaluation, Inference, and Search in Trick-Based Card Games," *IJCAI-09*, pp. 1407–1413, 2009.

[24] C. Solinas, D. Rebstock, and M. Buro, "Improving Search with Supervised Learning in Trick-Based Card Games," *AAAI-19*, 2019; arXiv:1903.09604.

[25] D. Rebstock, C. Solinas, M. Buro, and N. R. Sturtevant, "Policy Based Inference in Trick-Taking Card Games," *IEEE Conference on Games 2019*, 2019; arXiv:1905.10911.

[26] C. Solinas, D. Rebstock, N. R. Sturtevant, and M. Buro, "History Filtering in Imperfect Information Games: Algorithms and Complexity," *NeurIPS 2023*, 2023; arXiv:2311.14651.

[27] M. Zinkevich, M. Johanson, M. Bowling, and C. Piccione, "Regret Minimization in Games with Incomplete Information," *NIPS 2007*, 2007.

[28] M. Lanctot, K. Waugh, M. Zinkevich, and M. Bowling, "Monte Carlo Sampling for Regret Minimization in Extensive Games," *NIPS 2009*, 2009.

[29] O. Tammelin, "Solving Large Imperfect Information Games Using CFR+," arXiv:1407.5042, 2014.

[30] N. Brown and T. Sandholm, "Solving Imperfect-Information Games via Discounted Regret Minimization," *AAAI 2019*, 2019; arXiv:1809.04040.

[31] N. Brown, A. Lerer, S. Gross, and T. Sandholm, "Deep Counterfactual Regret Minimization," *ICML 2019*, 2019; arXiv:1811.00164.

[32] E. Steinberger, A. Lerer, and N. Brown, "DREAM: Deep Regret Minimization with Advantage Baselines and Model-Free Learning," arXiv:2006.10410, 2020.

[33] N. Brown, A. Bakhtin, A. Lerer, and Q. Gong, "Combining Deep Reinforcement Learning and Search for Imperfect-Information Games," *NeurIPS 2020*, 2020; arXiv:2007.13544.

[34] M. Schmid, M. Moravčík, N. Burch, R. Kadlec, J. Davidson, K. Waugh, N. Bard, F. Timbers, M. Lanctot, Z. Holland, E. Davoodi, A. Christianson, and M. Bowling, "Student of Games: A Unified Learning Algorithm for Both Perfect and Imperfect Information Games," *Science Advances* 9(46), 2023; arXiv:2112.03178.

[35] A. Nayyar, A. Mahajan, and D. Teneketzis, "Decentralized Stochastic Control with Partial History Sharing: A Common Information Approach," *IEEE Transactions on Automatic Control*, 2013.

[36] A. Celli and N. Gatti, "Computational Results for Extensive-Form Adversarial Team Games," *AAAI 2018*, 2018; arXiv:1711.06930.

[37] G. Farina, A. Celli, N. Gatti, and T. Sandholm, "Connecting Optimal Ex-Ante Collusion in Teams to Extensive-Form Correlation: Faster Algorithms and Positive Complexity Results," *ICML 2021*, PMLR 139:3164–3173, 2021.

[38] B. H. Zhang, G. Farina, A. Celli, and T. Sandholm, "Team Belief DAG: Generalizing the Sequence Form to Team Games for Fast Computation of Correlated Team Max-Min Equilibria via Regret Minimization," *ICML 2023*, 2023; arXiv:2202.00789.

[39] L. Carminati, F. Cacciamani, M. Ciccone, and N. Gatti, "A Marriage between Adversarial Team Games and 2-Player Games: Enabling Abstractions, No-Regret Learning and Subgame Solving," *ICML 2022*, 2022; arXiv:2206.09161.

[40] S. McAleer, G. Farina, G. Zhou, M. Wang, Y. Yang, and T. Sandholm, "Team-PSRO for Learning Approximate TMECor in Large Team Games via Cooperative Reinforcement Learning," *NeurIPS 2023*, 2023.

[41] Z. Xu, Y. Liang, C. Yu, Y. Wang, and Y. Wu, "Fictitious Cross-Play: Learning Global Nash Equilibrium in Mixed Cooperative-Competitive Games," *AAMAS 2023*, 2023; arXiv:2310.03354.

[42] N. Bard, J. N. Foerster, S. Chandar, N. Burch, M. Lanctot, H. F. Song, E. Parisotto, V. Dumoulin, S. Moitra, E. Hughes, I. Dunning, S. Mourad, H. Larochelle, M. G. Bellemare, and M. Bowling, "The Hanabi Challenge: A New Frontier for AI Research," *Artificial Intelligence* 280, 2020; arXiv:1902.00506.

[43] J. N. Foerster, F. Song, E. Hughes, N. Burch, I. Dunning, S. Whiteson, M. Botvinick, and M. Bowling, "Bayesian Action Decoder for Deep Multi-Agent Reinforcement Learning," *ICML 2019*, 2019; arXiv:1811.01458.

[44] H. Hu and J. N. Foerster, "Simplified Action Decoder for Deep Multi-Agent Reinforcement Learning," *ICLR 2020*, 2020; arXiv:1912.02288.

[45] H. Hu, A. Lerer, A. Peysakhovich, and J. N. Foerster, "'Other-Play' for Zero-Shot Coordination," *ICML 2020*, 2020; arXiv:2003.02979.

[46] H. Hu, A. Lerer, B. Cui, L. Pineda, N. Brown, and J. N. Foerster, "Off-Belief Learning," *ICML 2021*, 2021; arXiv:2103.04000.

[47] A. Lerer, H. Hu, J. N. Foerster, and N. Brown, "Improving Policies via Search in Cooperative Partially Observable Games," *AAAI 2020*, 2020; arXiv:1912.02318.

[48] C. Cox, J. De Silva, P. DeOrsey, F. H. J. Kenter, T. Retter, and J. Tobin, "How to Make the Perfect Fireworks Display: Two Strategies for Hanabi," *Mathematics Magazine* 88(5):323–336, 2015.

[49] J. Farrell and R. Gibbons, “Cheap Talk with Two Audiences,” *American Economic Review* 79(5):1214–1223, 1989.

[50] A. D. Wyner, “The Wire-Tap Channel,” *Bell System Technical Journal* 54(8):1355–1387, 1975.

[51] I. Csiszár and J. Körner, “Broadcast Channels with Confidential Messages,” *IEEE Transactions on Information Theory* 24(3):339–348, 1978.

[52] R. Lowe, J. N. Foerster, Y.-L. Boureau, J. Pineau, and Y. Dauphin, “On the Pitfalls of Measuring Emergent Communication,” *AAMAS 2019*, 2019; arXiv:1903.05168.

[53] J. Rong, T. Qin, and B. An, “Competitive Bridge Bidding with Deep Neural Networks,” *AAMAS 2019*, 2019; arXiv:1903.00900.

[54] Y. Tian, Q. Gong, and T. Jiang, “Joint Policy Search for Multi-Agent Collaboration with Imperfect Information,” *NeurIPS 2020*, 2020; arXiv:2008.06495.

[55] E. Lockhart, N. Burch, N. Bard, S. Borgeaud, T. Eccles, L. Smaira, and R. Smith, “Human-Agent Cooperation in Bridge Bidding,” arXiv:2011.14124, 2020.

[56] H. J. Ryser, *Combinatorial Mathematics*, Carus Mathematical Monographs 14, Mathematical Association of America, 1963.

[57] D. G. Glynn, “The Permanent of a Square Matrix,” *European Journal of Combinatorics* 31(7):1887–1891, 2010.

[58] M. Jerrum, A. Sinclair, and E. Vigoda, “A Polynomial-Time Approximation Algorithm for the Permanent of a Matrix with Non-Negative Entries,” *Journal of the ACM* 51(4):671–697, 2004.

[59] A. Newman and M. Vardi, “FPRAS Approximation of the Matrix Permanent in Practice,” arXiv:2012.03367, 2020.

[60] N. Linial, A. Samorodnitsky, and A. Wigderson, “A Deterministic Strongly Polynomial Algorithm for Matrix Scaling and Approximate Permanents,” *Combinatorica* 20(4):545–568, 2000.

[61] Y. Chen, P. Diaconis, S. P. Holmes, and J. S. Liu, “Sequential Monte Carlo Methods for Statistical Analysis of Tables,” *Journal of the American Statistical Association* 100(469):109–120, 2005.

[62] I. Bezáková, A. Sinclair, D. Štefankovič, and E. Vigoda, “Negative Examples for Sequential Importance Sampling of Binary Contingency Tables,” *Algorithmica* 64:606–620; preliminary version, *ESA 2006*.

[63] M. Cryan and M. Dyer, “A Polynomial-Time Algorithm to Approximately Count Contingency Tables When the Number of Rows Is Constant,” *Journal of Computer and System Sciences* 67(2):291–310, 2003; and M. Cryan, M. Dyer, L. A. Goldberg, M. Jerrum, and R. Martin, *SIAM Journal on Computing* 36(1):247–278, 2006.

[64] N. Bard, J. Hawkin, J. Rubin, and M. Zinkevich, “The Annual Computer Poker Competition,” *AI Magazine* 34(2):112–118, 2013; and the competition rules describing duplicate matches and common seeds.

[65] M. Zinkevich, M. Bowling, N. Bard, M. Kan, and D. Billings, “Optimal Unbiased Estimators for Evaluating Agent Performance,” *AAAI-06*, pp. 573–578, 2006.

[66] M. Bowling, M. Johanson, N. Burch, and D. Szafron, "Strategy Evaluation in Extensive Games with Importance Sampling," *ICML 2008*, 2008.

[67] M. White and M. Bowling, "Learning a Value Analysis Tool for Agent Evaluation," *IJCAI-09*, pp. 1976–1981, 2009.

[68] N. Burch, M. Schmid, M. Moravčík, D. Morrill, and M. Bowling, "AIVAT: A New Variance Reduction Technique for Agent Evaluation in Imperfect Information Games," *AAAI-18*, 2018; arXiv:1612.06915.

[69] J. Kim and T. Sandholm, "Heuristic Pathologies and Further Variance Reduction via Uncertainty Propagation in the AIVAT Family of Techniques," arXiv:2605.14261, 2026.

[70] B. Li, Y. Chen, and L. Huang, "AV-AIVAT: 74× Cheaper Agent Evaluation with Certified Anytime-Valid Stopping in Imperfect-Information Games," arXiv:2608.06362, 2026.

[71] V. Lisý and M. Bowling, "Equilibrium Approximation Quality of Current No-Limit Poker Bots," arXiv:1612.07547, 2016.

[72] K. Waugh, D. Schnizlein, M. Bowling, and D. Szafron, "Abstraction Pathologies in Extensive Games," *AAMAS 2009*, pp. 781–788, 2009.

[73] R. Coulom, "Whole-History Rating: A Bayesian Rating System for Players of Time-Varying Strength," *Computers and Games 2008*, 2008.

[74] R. Herbrich, T. Minka, and T. Graepel, "TrueSkill™: A Bayesian Skill Rating System," *Advances in Neural Information Processing Systems* 19, 2006.

[75] D. Balduzzi, K. Tuyls, J. Pérolat, and T. Graepel, "Re-evaluating Evaluation," *NeurIPS 2018*, 2018; arXiv:1806.02643.

[76] S. Omidshafiei, C. Papadimitriou, G. Piliouras, K. Tuyls, M. Rowland, J.-B. Lespiau, W. M. Czarnecki, M. Lanctot, J. Pérolat, and R. Munos, "α-Rank: Multi-Agent Evaluation by Evolution," *Scientific Reports* 9:9937, 2019.

[77] S. M. Jordan, Y. Chandak, D. Cohen, M. Zhang, and P. S. Thomas, "Evaluating the Performance of Reinforcement Learning Algorithms," *ICML 2020*, PMLR 119:4962–4973, 2020.

[78] M. Van den Bergh, "Comments on Normalized Elo," Fishtest technical note; and Stockfish contributors, "Statistical Methods and Algorithms in Fishtest."
https://cantate.be/Fishtest/normalized_elo_practical.pdf

A The rules dialect in full

This appendix states the dialect of §2 at the length required to re-implement it. Table 5 is the index: it lists every consequential choice against the engine switch that changes it, and the subsections that follow give each choice in full. The two files that carry the definitions are `engine/src/fish.hpp` (deck, indexing, the `Rules` record, the legality predicate) and `engine/src/game.hpp` (the driver: declaration rounds, turn handling, the forced endgame, the action cap). Where the text below names a field it is a member of `Rules`; where it names a routine it is a member of the driver.

Table 5: The rules dialect used for every experiment in this paper unless stated otherwise, and the engine switch that changes each choice. The switches are command-line flags of the `fish` driver; the underlying fields are members of `Rules` in `engine/src/fish.hpp`. Rows marked "modelling choice" are not fixed by any published statement of the rules.

Choice	Value used	Switch
Deck composition	54 cards in nine half-suits (low and high band of each suit, plus the four eights and two jokers); nine cards per player	`--sets=8` gives the 48-card, eight-half-suit form
Ask legality	Live half-suit; asker holds another card of it and not the named card; target is an opponent and still holds cards	—
Turn transfer	A successful ask retains the turn; a miss passes it to the target	—
Out-of-turn declarations	Permitted at any moment, including during an opponent's turn	`outOfTurnDeclare` (`--no-out-of-turn`)
Cardless players may declare	Permitted	`cardlessMayDeclare` (`--no-cardless-declare`)
Incorrect declaration	Awards the half-suit to the opposing team	—
Declaration aftermath	The six cards leave play and the turn returns to the player who held it	—
Cardless turn-holder	Chooses which live teammate receives the turn	—
Forced endgame (whole team cardless)	The live team declares every remaining half-suit, sharing only willingness, over an eight-level ladder $\{0.995, 0.98, 0.95, 0.90, 0.80, 0.65, 0.50, \text{best guess}\}$	`--legacy` gives the two-level ladder $\{0.38, \text{best guess}\}$
Simultaneous declarations	Lowest seat index declares first (a modelling choice; alternatives measured in §E)	`declArbitration` (`--arb=low|high|turn`)
Action cap	400 asks (360 under the v0.3 dialect), any residue adjudicated neutrally by majority physical holding, ties to the holder of the lowest card	`--maxasks`
Conventions	No prearranged signalling conventions; team communication is limited to the willingness bit described above	—

A.1 Players, teams, and the deck

Six players occupy seats $0, \ldots, 5$. Teams alternate: seats $\{0, 2, 4\}$ form team 0 and seats $\{1, 3, 5\}$ form team 1, so every player's two neighbours are opponents. The deck is 54 cards partitioned into $N_H = 9$ half-suits of six cards. Half-suits are indexed $h \in \{0, \ldots, 8\}$ and positions within a half-suit $i \in \{0, \ldots, 5\}$; a card is the integer

$$c \;=\; 6h + i, \qquad h = \lfloor c/6 \rfloor, \qquad i = c \bmod 6.$$

The ordering is fixed and matches the v0.3 engine card for card, so that ported policies and cross-engine checks line up:

h	Half-suit	Cards at positions $i = 0, \ldots, 5$
0	Low Spades	2S, 3S, 4S, 5S, 6S, 7S
1	High Spades	9S, 10S, JS, QS, KS, AS
2	Low Hearts	2H, 3H, 4H, 5H, 6H, 7H
3	High Hearts	9H, 10H, JH, QH, KH, AH
4	Low Diamonds	2D, 3D, 4D, 5D, 6D, 7D
5	High Diamonds	9D, 10D, JD, QD, KD, AD
6	Low Clubs	2C, 3C, 4C, 5C, 6C, 7C
7	High Clubs	9C, 10C, JC, QC, KC, AC
8	Eights and Jokers	8S, 8H, 8D, 8C, red joker, black joker

Even h are low bands, odd h are high bands, and $\lfloor h/2 \rfloor$ is the suit in the order spades, hearts, diamonds, clubs; $h = 8$ is the mixed half-suit. Hands are 64-bit masks with bit c set when the seat holds card c, so a half-suit is the mask `0x3F` $\ll 6h$. The code identifier for a half-suit is `set`; the term is used nowhere else in this paper.

With `deckSets` $= 8$ (the `--sets=8` switch) the ninth half-suit is removed, giving the 48-card, eight-half-suit form with eight cards per player; all other rules are unchanged.

A.2 The deal and the opening turn

The deck is shuffled by a Fisher–Yates pass driven by a splitmix64 generator seeded from the deal seed, and dealt in contiguous blocks of $54/6 = 9$ cards (eight cards under `deckSets` $= 8$) to seats $0, \ldots, 5$ in order. The initial deal is retained separately as ground truth and is never exposed to a policy. A dealer seat is then drawn uniformly and the opening turn is given to the seat clockwise of the dealer. Because the shuffle is a pure function of the seed, a deal is reproducible from its seed alone, which is what makes the duplicate-block protocol of §9 possible.

A.3 Asking

Let $\mathcal{H}$ be the set of live half-suits, team(p) the team of seat p, and n_p the public card count of seat p. Seat a asking seat t for card c is legal exactly when all five of the following hold:

1. $h(c) \in \mathcal{H}$: the half-suit has not yet been claimed;
2. team(t) $\neq$ team(a): the target is an opponent;
3. $n_t > 0$: the target still holds cards;
4. a does not hold c;
5. a holds at least one card of $h(c)$ — necessarily a card other than c, by the previous condition.

Two properties of this predicate matter for the rest of the paper. First, the only private input is the asker's own hand: conditions 1–3 are functions of the public state alone, and conditions 4–5 are functions of the public state and the asker's hand. An observer watching a legal ask therefore learns a certificate about the asker's hand without needing to see it, which is the C5 constraint of §5. Second, the predicate is evaluated identically on the ground-truth hand inside the driver and on the observer's reconstruction, so the two cannot disagree.

The target must surrender the card if they hold it, and no other transfer is possible. On a hit the asker keeps the turn; on a miss the turn passes to the target. The ask, the identity of asker and target, the named card, the outcome, and the resulting card counts are all appended to a public event history.

If the seat to move has no legal ask at all — which happens only when every card it holds lies in a half-suit whose other five cards it also holds — it must declare instead, and the driver requires a declaration of one such half-suit from that seat.

A.4 Declarations

A declaration names a live half-suit h and an allocation $A : \{0, \ldots, 5\} \to \{0, \ldots, 5\}$ assigning each position i of h to the seat claimed to hold card $6h + i$. Three conditions are checked before the declaration is applied: the half-suit must still be live; every named seat must belong to the declarer's own team; and, when `outOfTurnDeclare` is disabled or `cardlessMayDeclare` is disabled, the declarer must respectively hold the turn or hold at least one card. Under the dialect studied both switches are enabled, so any seat may declare at any moment whether or not it holds cards and whether or not it holds cards of the half-suit concerned.

Resolution is all-or-nothing. The declaration is correct when, for every i, seat $A(i)$ actually holds card $6h+i$; a single misassignment — including one that names the right team but the wrong teammate — makes the whole declaration incorrect. A correct declaration awards h to the declarer's team; an incorrect one awards h to the opposing team. In both cases the six cards are removed from every hand, h leaves $\mathcal{H}$, the public card counts are updated, and the turn returns to the seat that held it before the declaration. The declarer's own stated confidence is recorded in the event history as a diagnostic and is never used to resolve anything.

The driver polls for declarations before every ask, in a loop that repeats while any seat still offers a declaration, so several half-suits can be claimed between two consecutive asks.

A.5 Arbitration between simultaneous declarations

Because any seat may declare at any moment, the poll can find more than one willing declarer, and the rules as published do not say who acts first. `declArbitration` selects the scan order over seats and the first willing seat in that order declares:

- `0` (default, `--arb=low`): ascending seat index $0, 1, \ldots, 5$;
- `1` (`--arb=high`): descending seat index $5, 4, \ldots, 0$;
- `2` (`--arb=turn`): ascending from the current turn-holder, $g, g + 1, \ldots$ modulo six.

Selecting the most confident proposer is deliberately not offered. Confidence is private to a seat, and comparing confidences across seats would move information between opponents as well as teammates; it is the same leak that the forced endgame was corrected to remove (§A.8). E17 reports the primary head-to-head result under all three orders (§E).

A.6 Cardless players and the turn

A seat with no cards leaves the asking cycle: it cannot ask, and condition 3 of §A.3 makes it an illegal target. It remains part of the game in two ways — it can be named in a teammate's declaration as the holder of a card, which is impossible while it is genuinely empty and therefore constrains the allocations a teammate may name, and, with `cardlessMayDeclare` enabled, it may itself declare.

The turn can reach a cardless seat only immediately after a declaration removed its last cards. That seat then chooses which of its live teammates receives the turn; the choice is a policy decision, and an invalid choice is replaced by the lowest-indexed live teammate. If the seat has no live teammate, its whole team is cardless and the forced endgame begins.

A.7 The forced endgame

When one team holds no cards at all, no legal ask remains for either side, and the live team must declare every remaining half-suit. Teammates may negotiate openly but may exchange nothing except willingness to declare, so the driver implements the selection as a threshold sweep rather than a comparison of confidences. The willingness ladder is

$$\Theta \;=\; \{\, 0.995,\; 0.98,\; 0.95,\; 0.90,\; 0.80,\; 0.65,\; 0.50,\; \text{best guess} \,\},$$

eight levels held in `forcedTh`, the last entry (stored as -1) meaning that somebody must declare and should submit their best guess. For each threshold $\theta \in \Theta$ in descending order, the driver scans the live team's seats over the remaining half-suits and asks each seat whether it is willing to declare that half-suit at confidence at least θ; the first seat that says yes declares, the ladder restarts at its top, and the sweep repeats until every half-suit is resolved. Only the willingness bit crosses between seats. A consequence of restarting the ladder is that early, safer declarations resolve cards and thereby sharpen the allocations available for the later ones.

Declarations in the forced endgame resolve exactly as in §A.4, including the possibility of misdeclaring and handing a half-suit to the cardless team. They are recorded separately from voluntary declarations in the event history. If every seat refuses at every level — reachable only with a policy that declines its own best guess — the remaining half-suits are awarded to the team physically holding the majority of their cards.

Under `--legacy` the ladder is the two-level {0.38, best guess} of the v0.3 dialect.

A.8 Two further differences from the v0.3 dialect

§2.2 lists the two v0.3 differences that bear on the reported results — turn-only declarations and the 360-ask cap. The other two concern the two teammate-to-teammate channels of §A.11, and are recorded here.

The successor of a cardless turn-holder was fixed. In the v0.3 engine [1] the turn passed automatically to the live teammate holding the most cards. In the dialect studied the cardless seat chooses (§A.6). The v0.3 rule is a function of the public card counts alone and so conveys nothing between teammates; the choice made here is a policy decision and is therefore the second of the two channels listed in §A.11. What it can carry is bounded by its range: one selection among at most two live teammates, made only immediately after a declaration emptied the chooser's hand.

The forced endgame selected the declarer by comparing private confidences. v0.3 gave the declaration to whichever live seat stated the highest confidence in a remaining half-suit. Ordering seats by confidence conveys more than willingness, because the ordering is itself a statistic of the hands being compared. The threshold sweep of §A.7 replaces that comparison with a descending sequence of thresholds shared in advance, so that only a willingness bit crosses between seats. The same argument is why confidence ranking is not offered as an arbitration order in §A.5.

A.9 The action cap and adjudication

The driver counts asks and stops the game once `maxAsks` is reached: 400 under the dialect studied, 360 under `--legacy`, and settable with `--maxasks`. Awarding the unresolved residue to either side would bias the outcome towards whichever policy stalls more readily, so with `adjudicateOnLimit` enabled each remaining half-suit is awarded to the team physically holding the majority of its six cards, ties going to the team holding the lowest-indexed card of that half-suit. The game result records whether the cap was reached, and that incidence is reported alongside every experiment in this paper. It is 0.000% in E1, E3, E7, E10 and E13; the exception is the deliberately unforced mirror configuration of E11 (§6.3), which is measured precisely because it does reach the cap.

A.10 Termination of a game and scoring

The game ends when no live half-suit remains: every half-suit has been claimed by a declaration, resolved in the forced endgame, or adjudicated at the cap. Each of the nine half-suits is awarded

to exactly one team, so the final scores sum to nine and the team taking at least five wins. Under `deckSets` $= 8$ the scores sum to eight, a level four-all result is possible, and the driver awards such a game to team 0; that case cannot arise with nine half-suits.

A.11 Communication and conventions

No prearranged signalling conventions are available to any policy in this paper. The only channel between teammates beyond the public event history is the willingness bit of §A.7 and the choice of successor made by a cardless turn-holder. Every other quantity a policy computes — ownership marginals, allocation probabilities, declaration confidences — is private to the seat that computed it, and the information-safety audit of §4 checks that no policy is ever handed a quantity derived from another seat's hand.

B Inference: derivations and implementation notes

This appendix supplies the derivations and implementation detail behind §5. Notation is that of §5.2. The source files are `engine/src/belief.hpp` (constraint bookkeeping and the Fast path), `engine/src/blockdp.hpp` (the exact reference engine), and `engine/src/oracle.hpp` (the brute-force enumerator used in E15).

B.1 The layer identity

Order the unresolved cards $c_1, \dots, c_{|U|}$ and let $q = (q_0, \dots, q_5)$ be the capacity vector of (1). A partial assignment of the first j cards leaves a vector n of unused capacities. Because every one of the first j cards consumes exactly one unit of capacity from exactly one seat,

$$\sum_p n_p \;=\; \sum_p q_p - j \;=\; |U| - j,$$

so j is recoverable from n and the layer index need not be stored. This is the identity that collapses the natural $|U|$-layer table into two single arrays. Concretely, the forward and backward tables are laid out as one array each of size $\prod_p (q_p + 1)$ — at most 10^5 entries by (17), so the $|U|$-layer table that (3)–(4) would otherwise require never has to be materialised — addressed by the mixed-radix offset $\mathrm{off}(n) = \sum_p n_p\, s_p$ with strides $s_p = \prod_{r<p}(q_r + 1)$, and a companion index sorts the reachable states by $\sum_p n_p$ so that a sweep visits one layer at a time. Count vectors are packed one byte per seat into a single `uint64_t`, which lets the dominance test $n \geq t$ that guards a block transition be evaluated as one branch-free word operation.

The recursions are those of (3)–(4), evaluated with the layer constraint $\sum_p n_p = |U| - j$ in force at every step and initialised at $F_0(q) = 1$ and $B_{|U|}(\mathbf{0}) = 1$; e_p is the p-th unit vector and q the full capacity vector of (1), so a transition consumes one unit of one seat's capacity and the branching factor at card c_j is $|M_{c_j}| \leq 5$. The per-card marginals of §5.4 are the contraction of the two tables at card j,

$$\mu_{c_j,p} \;=\; \frac{\mathbf{1}[\,p \in M_{c_j}\,]}{Z} \sum_{\substack{n\,:\,|n| = |U|-j+1 \\ n_p \geq 1}} F_{j-1}(n)\; B_j(n - e_p), \tag{16}$$

which is *exact*, since every consistent assignment is counted once by splitting it at card j into a prefix counted by F_{j-1} and a suffix counted by B_j. Both tables are evaluated in double precision on integer-valued data. Each entry is a count of assignments, so the arithmetic is exact while the counts stay below 2^{53}, and the state-space bound of §B.2 keeps the number of terms per entry at six. E15 does not report the magnitudes of the partition functions it compared; what it reports is that the block

engine and the brute-force enumerator agreed exactly, with a maximum relative difference of zero in Z and a maximum absolute difference of zero across all 656,826 marginal, 26,930 team-ownership and 3,189,103 allocation comparisons (§10.1), that is, at integer rather than rounding scale.

B.2 Why the state space is bounded by 10^5

The observer's own cards are resolved by (C1), so $q_i = 0$ and the table is a product over the five other seats. Also $|U| \le 45$: at the deal the 54 cards lie in nine live half-suits, the observer holds nine of them and those are resolved by (C1), so $|U| = 45$ initially; thereafter a card leaves U when its holder is revealed or its half-suit is declared and, by Proposition 1, never re-enters. Subject to $\sum_{p\neq i} q_p = |U|$, the product $\prod_{p\neq i}(q_p + 1)$ is maximised when the capacities are equal, by the arithmetic–geometric mean inequality, giving

$$\prod_{p\neq i}(q_p + 1) \;\le\; \Big(\frac{\sum_{p\neq i}(q_p + 1)}{5}\Big)^5 \;=\; \Big(\frac{|U|}{5} + 1\Big)^5 \;\le\; 10^5. \tag{17}$$

Each state has at most six outgoing transitions, so a full forward–backward pass is bounded by 6×10^5 multiply–adds independently of the position, and the two arrays together occupy under two megabytes. In practice most positions are far smaller, because resolved cards leave U and completed half-suits leave $\mathcal{H}$. The implementation additionally refuses to build a table above a fixed state cap and reports the failure to the caller rather than silently degrading; the build-failure count was zero in both E2 and E15.

B.3 The direct sampler

The backward table doubles as a sampler over assignments. Having placed $c_1, \dots, c_{j-1}$ and arrived at the remaining-capacity vector n, card c_j is given to seat p with probability proportional to $B_j(n - e_p)$, restricted to $p \in M_{c_j}$ with $n_p \ge 1$. A seat receives positive weight only when the suffix can still be completed from $n - e_p$, so every prefix the sampler constructs is extendable and no draw is ever discarded: the procedure is rejection-free for (C1)–(C4), and the resulting distribution is uniform over the assignments satisfying those four constraints, because the weights are path counts. When a live (C5) certificate is present the sampler is unchanged and the completed assignment is tested against the literal disjunction (2), drawing again on failure, so the combined procedure is exact but not rejection-free. The sampler is used only off the deployed decision path, as the Monte-Carlo ground truth in the E2 cross-checks of §10.1; the residual difference there is at Monte-Carlo scale (4.066×10^{-2}) rather than at the exact zero E15 obtains by enumeration.

B.4 Card groups and block groups

Every certificate (2) constrains only cards of one half-suit, so no certificate couples two half-suits. That is what makes the exhaustive branch affordable: enumerating one half-suit costs at most $\prod_{c\in H} |M_c|$ candidate assignments and is paid once per block, whereas handling the same k live certificates by inclusion–exclusion over the whole of U would cost 2^k forward–backward passes. The reference engine therefore partitions U by half-suit and classifies each group:

Card group.
No live certificate in that half-suit. The group's cards are expanded one at a time by the recursions of §B.1, with branching factor $|M_c| \le 5$. This is the ordinary capacity dynamic program restricted to the group.

Block group.
At least one live certificate. The group's assignments are enumerated exhaustively by depth-first

search over the supports M_c — at most $5^6 = 15{,}625$ candidates for a six-card half-suit — each is tested against every certificate of that half-suit, and the survivors are aggregated by count vector into the table $g_H(t)$ of (6). Alongside $g_H(t)$ the engine accumulates, for each count vector, the per-card breakdown $g_H^{(c\to p)}(t)$ that (8) needs, so a single enumeration serves all three queries.

A group of either kind consumes a known number of cards, so the layer identity of §B.1 survives the substitution and the outer dynamic program runs over groups: $F_b(n) = \sum_t g_b(t) F_{b-1}(n+t)$, with $g_b(t)$ the indicator of a single card placement in the card-group case. The path mass (7) is then a contraction of the forward and backward arrays at the group's entry layer, and the three queries (8)–(10) are weighted sums over the group's own table. Nothing about this treatment is approximate: the block enumeration is exhaustive, the certificate test is the literal disjunction (2), and the outer recursion is a counting argument.

The exhaustive branch is what makes the equal-probability statement of Corollary 2 usable and also what makes its caveat necessary. The count-vector table records only survivors, so two allocations with the same count vector are equiprobable *given that both survive*; an allocation that fails a certificate has probability zero even though its count vector may be shared with survivors. The engine's `bestTeamAllocation` therefore returns a stored survivor rather than reconstructing an allocation from the count vector. E15 checks this directly: over 15,235 calls, none returned an allocation of probability zero and none returned a non-maximal one (§10.1).

B.5 The brute-force allocation oracle (E15)

Table 6: Brute-force oracle (E15) against the exact reference engine. Population: reachable states from 150 replayed v0.4-Fast versus FishBot v0.3 games, seed 20260822, policy weights frozen; the unit is one state. 15,544 states were enumerated, 14,942 with a live ask-legality certificate, over 32,727,257 consistent deals; 75,203 were skipped as too large. No intervals: every entry is a deterministic maximum over the checks in the last column, except the sampler row, a Monte-Carlo frequency comparison.

Quantity	Comparison	Value	Checks
partition function Z	Z max relative difference	0	—
per-card marginals $\mu_{c,p}$	max absolute difference	0	656,826
team ownership $\tau(H,T)$	max absolute difference	0	26,930
named allocation $\alpha(A)$	max absolute difference	0	3,189,103
equal-probability corollary	max within-class spread	0	—
sampler frequencies	max absolute difference	0.0532	46,345,121

The block engine of §5.5 and the card-level dynamic program of §5.4 share a derivation, so agreement between them is weak evidence that either enumerates the right collection of assignments. E15 supplies independent evidence by recomputing the same posterior with no dynamic programming, no factorisation and no sampling. The enumerator is `engine/src/oracle.hpp` and the driver is the `oracle` command of `engine/src/main.cpp`.

The enumerator. For one observer's constraint state the unresolved cards are ordered and searched by iterative depth-first search over their assignments to seats. At depth j the candidate seats for card c_j are exactly the surviving support M_{c_j}, which carries (C1)–(C3), and a candidate is pruned at once if it would take the number of cards already given to that seat above its capacity q_p, which carries (C4). Every leaf is therefore a complete capacity-feasible assignment, and (C5) is tested there: the literal disjunction (2) is evaluated against the completed assignment, certificate by certificate, with no relaxation and no early aggregation. Surviving leaves are counted into Z, tallied into per-card ownership counts, and tallied into a per-half-suit table keyed by the packed assignment of that half-

suit's unresolved cards. Nothing is factorised: the count behind every named allocation is an explicit count of leaves.

Coverage. The search is exponential in $|U|$, so each state carries a cut-off. A state is attempted only when the capacity dynamic program reports at most 200,000 consistent deals, and the search is abandoned if the product of the support sizes or the node count exceeds a fixed multiple of that budget. A state that hits either bound is counted as skipped and reported — 75,203 of them against 15,544 enumerated — rather than dropped silently, so the coverage of the test is visible rather than implied. Skipping is correlated with the property that makes a state hard, a large unresolved pool, so E15 is evidence about small reachable states and not about the whole state space.

What is compared. The block engine is built on the same constraint state, and seven quantities are compared on every enumerated state: the partition function Z; the per-card marginals (8) against the leaf tallies; the team-ownership probabilities (10), for both teams and every half-suit still feasible for the team in question, against the summed counts of the allocations lying wholly inside it; the probability (9) of *every* named allocation the enumeration produced rather than a sample of them; the equal-probability corollary (Corollary 2), as the spread between the largest and the smallest enumerated probability within each count-vector class; the reference declarer's own named allocation, by locating `bestTeamAllocation`'s output in the enumerated table and asking both whether it has positive probability and whether it attains the maximum over that team's allocations; and the frequencies of the direct sampler of §B.3 drawn on the same state, against the enumerated marginals. The command and its per-state draw budget are in §G.2.

Why the deterministic differences are exactly zero. Six of the seven comparisons are deterministic. Five of those are the zero entries of Table 6, and the sixth, the reference declarer's allocation, is reported in §10.1 as 15,235 checks with 0 inconsistent. Zero here is not rounding luck. Both sides compute integer counts over the same finite set of assignments — the enumerator by incrementing a counter at each surviving leaf, the block engine by the counting recursions (3)–(5) and the block tables (6) — and both hold those counts in double precision, where the per-state cut-off of 200,000 consistent deals keeps every one of them far below 2^{53} and therefore exactly represented. The two sides form the same ratio count/Z from identical operands and obtain identical doubles, so the difference is zero rather than small and the comparison has no tolerance absorbing an error. The seventh comparison, the sampler, is a Monte-Carlo frequency and differs at Monte-Carlo scale: 0.0532 over 46,345,121 accepted draws.

B.6 Declarations require no special handling

A declaration — correct or incorrect — removes six cards from play and changes the public hand counts of the players who held them. Both effects are public. In the constraint system this means the six cards leave U with their holders known, the affected half-suit leaves $\mathcal{H}$, and each h_p in (1) drops by the number of those cards that seat held. The capacities $q_p = h_p - k_p$ therefore re-derive correctly with no bespoke rule: the same identity that absorbs a successful ask absorbs a declaration. The one thing the observer learns beyond the removal is the declarer's belief, which the rules posterior does not use; §5.7 is where information of that kind would enter.

The same argument explains why an incorrect declaration needs no special case either. It reveals the true holders of all six cards, which is strictly more information than a correct one gives the opposing team, and that information arrives through exactly the same channel: resolved holders and updated public counts.

B.7 The Sinkhorn conditioning step

The Fast path maintains a matrix $p_{c,p}$ over unresolved cards and seats, initialised to the prior weights (11) on the support M_c and zero off it. One Sinkhorn iteration normalises each row to sum to one, then scales each column p by $q_p / \sum_c p_{c,p}$, which enforces (1) in expectation. The deployed setting is four outer sweeps of eight such iterations, with a conditioning step for the certificates applied between consecutive sweeps and a final row normalisation.

The conditioning step derives as follows. Fix a certificate with asking seat a (the notation of (C5); A remains the named allocation of §5.2) and card set D, and let E be the event $\bigvee_{d \in D}[\sigma(d) = a]$. Treating the entries $\{p_{d,a}\}_{d \in D}$ as independent Bernoulli indicators,

$$\Pr[\neg E] \;=\; \prod_{e \in D} (1 - p_{e,a}), \qquad \Pr[E] \;=\; 1 - \prod_{e \in D} (1 - p_{e,a}).$$

For a particular $d \in D$, the event $\sigma(d) = a$ implies E, so

$$\Pr[\sigma(d) = a \mid E] \;=\; \frac{p_{d,a}}{\Pr[E]}.$$

For a seat $p \neq a$, the event $\sigma(d) = p$ is compatible with E only if some other card of D goes to a, whose independent probability is $1 - \prod_{e \in D \setminus \{d\}} (1 - p_{e,a})$. Together the two cases give the conditioning step of §5.6,

$$\Pr[\sigma(d) = a \mid E] = \frac{p_{d,a}}{1 - \prod_{e \in D} (1 - p_{e,a})}, \qquad \Pr[\sigma(d) = p \mid E] = p_{d,p} \, \frac{1 - \prod_{e \in D \setminus \{d\}} (1 - p_{e,a})}{1 - \prod_{e \in D} (1 - p_{e,a})}. \tag{18}$$

Rows are renormalised afterwards, and the next Sinkhorn sweep restores the capacity constraints that the reweighting disturbed. The implementation guards two degenerate cases: when $\Pr[E]$ underflows, the mass is forced onto the certificate rather than dividing by a near-zero denominator; when $\Pr[\neg E]$ underflows, the certificate is already implied and the step is skipped.

The independence assumption is false — the capacity constraints couple the cards of D — so (18) is an approximation, and interleaving it with capacity fitting does not converge to the exact marginals of (8). E2 measures the residual gap directly against the reference engine: mean absolute difference 0.01705, maximum 0.4982 (§10.1).

B.8 The deployed declaration estimator $\hat{\alpha}$

The declaration query (9) asks for the joint probability of a named allocation. The Fast path has only marginals, so it evaluates the joint by sequential conditioning. Given a candidate allocation A that assigns cards $c_1, \ldots, c_6$ of one half-suit to seats $p_1, \ldots, p_6$, the estimator is

$$\hat{\alpha}(A) \;=\; \prod_{j=1}^{6} \hat{\mu}^{(j)}_{c_j, p_j},$$

where $\hat{\mu}^{(j)}$ is the Fast marginal recomputed on the constraint state in which $c_1, \ldots, c_{j-1}$ have already been resolved to $p_1, \ldots, p_{j-1}$. Each step fixes one card, decrements that seat's capacity in (1), propagates the resulting support reductions, and refits. A card already resolved contributes a factor of one if it matches A and zero otherwise; a card whose support excludes its assigned seat returns zero immediately; and the product short-circuits once it falls below 10^{-9}, since candidates that weak are rejected regardless.

The chain rule this implements is exact — the joint is the product of successive conditionals in any fixed order — so the estimator is exact in structure and approximate only through the marginal solver

of §B.7. Refitting between cards carries the capacity coupling of (1), which a product of unconditioned marginals ignores; no experiment in this paper measures the two estimators against each other, so the comparison is structural rather than empirical.

Naming the allocation is a separate step, and the deployed default handles it more crudely. For each card of the candidate half-suit, the seat is chosen as the teammate with the largest *unconditioned* Fast marginal $\hat{\mu}_{c,p}$, with a card the observer holds assigned to the observer and a card already resolved to a teammate assigned to that teammate. Only then is $\hat{\alpha}$ evaluated on the resulting allocation. Choosing greedily under unconditioned marginals is not the same as maximising the joint, precisely because of the coupling just described; the alternative of choosing each seat under the *conditioned* marginal at that step is implemented and is one of the frozen-policy ablations of E5, where it changed the win rate by +0.18 points with a 95% paired percentile bootstrap interval over deals of -0.12–+0.47 (§D). Under v0.4-Block the naming step is replaced by `bestTeamAllocation`, which returns a maximum-probability survivor directly from the block table (§B.4).

B.9 Where each path is used

To summarise the division of labour. v0.4-Fast runs the Fast path of §5.6 for every query on the decision path: ask scoring, the two-ply branch re-evaluation, the declaration pre-gates, and $\hat{\alpha}$. The reference engine of §5.5 is invoked in three places only — the cross-checks of E2, the brute-force oracle comparison of E15, and the v0.4-Block substitution ablation — none of which is part of the deployed policy. The throughput cost of the difference is reported in §10.7.

C Configuration and fitted parameters

This appendix records the frozen configuration in enough detail to reconstruct it exactly. It is a reproducibility artifact. As stated in §8, no strategic interpretation should be placed on individual coefficients: the ask features are mutually correlated and normalised to different effective ranges, and two of the decision knobs are inert in the shipping configuration. Table 7 and Table 8 are a pair: the first defines the 20 ask features, the second gives the weight fitted to each of them, under the same feature names. The appendix then records the per-coordinate bounds, the implementation detail of ask scoring and of the two-ply lookahead, the value-function features, coefficients and provenance gap, the two endgame decisions that belong to the policy rather than to the driver (§C.6), and the specification grammar that names any configuration on the command line.

C.1 The ask features

C.2 Ask scoring and lookahead implementation

Two details of §7.2 and §7.3 are recorded here rather than in the main text.

Ties in (13), and in the rescoring (14) that follows it, are broken deterministically: first by the index of the named card, then by the index of the target seat, so no randomisation enters the choice — this is part of what the determinism claimed in §7 rests on.

The branch states of the two-ply refinement are constructed explicitly rather than sampled. On the post-hit branch the named card is resolved to the asker's hand and the two affected public hand counts change; on the post-miss branch an exclusion is added, removing the target from that card's surviving support M_c. Capacities (1) are re-derived on each branch and the Fast marginals of §5.6 refitted there, which is why the branch beliefs carry the same approximation error as $\hat{\mu}$ itself.

Table 7: The 20 ask features φ_k entering the fitted linear term of (13), as chosen in §7.2. All are normalised to roughly $[0, 1]$ so that the fitted weights are comparable in scale; the weight fitted to feature k appears in Table 8 under the same name. H denotes the named card's half-suit and $p = \hat{\mu}_{c,t}$ the Fast posterior that the target holds the named card; probabilities written $\Pr[\cdot]$ are Fast per-card marginals, so every feature is computed from the approximate inference path rather than from the reference engine. See §8 for why individual coefficients should not be read as strategy.

k	Name	Definition
0	hit probability	$p = \hat{\mu}_{c,t}$, the Fast posterior that the target holds the named card
1	squared hit	p^2; lets the fit curve the trade-off between certainty and value
2	certain hit	$\mathbf{1}[p > 0.9995]$
3	own progress	cards of H in hand, over six
4	team control	expected fraction of H held by our team
5	lock completion	$\prod_{d \in H \setminus \{c\}} \Pr[d \text{ is ours}]$: the probability this ask alone completes team ownership of the half-suit
6	continuation	best posterior on any other missing card of H
7	completion bonus	1 at four or more cards of H in hand, 0.35 at three
8	reply threat	$(1-p)\cdot$ the best ask the target could make against us if handed the turn
9	information leak	1 if our team has never publicly revealed an interest in H
10	target hand size	target's public card count over nine
11	empties the target	p if the target holds exactly one card, else 0
12	repeats our half-suit	1 if this repeats our own previous half-suit
13	known team cards	publicly located cards of H on our team, over six
14	location entropy	$-p \log_2 p - (1-p) \log_2 (1-p)$
15	team owns H	$\prod_{d \in H} \Pr[d \text{ is ours}]$ before the ask; an independence proxy, not the exact query (10)
16	exposure on a miss	$(1-p)\cdot$ our cards visible in half-suits the target has asked in
17	trailing pressure	p when behind by two or more half-suits
18	runway	expected length of the run this ask begins, from the best remaining per-card posteriors
19	leak magnitude	feature 9 scaled by how much of H we hold

C.3 The frozen parameter vector

The same vector can be supplied to any build of the engine as a single `allparams=` string. Concatenate the three lines below with no intervening whitespace; coordinates are separated by `|` and appear in the order *ask weights* 0–19*, then the 14 knobs in the order of Table 8*:

```
11.506|3.2948|3.1978|1.6881|2.1333|4.0705|1.4679|1.4281|-3.0978|-0.8536|-2.0219|1.166|
1.2697|0.9142|-2.6534|-0.8045|1.904|0.0473|1.4108|-0.999|0.8177|0.7969|0.3325|6.249|
2.8676|6.0432|0.7667|0.7925|-0.0342|0.2638|0.1328|5.6239|3.358|2.6127
```

C.4 Bounds imposed per coordinate

The optimiser samples from an unconstrained diagonal Gaussian; the bounds below are applied when a sampled vector is turned into a configuration, both by the engine's parameter parser (`engine/src/factory.hpp`) and by the freeze step (`engine/freeze_config.py`), so a coordinate outside its bound is clamped rather than rejected. The two integer coordinates are rounded to the nearest integer after clamping.

Table 8: The frozen v0.4-Fast configuration: the 20 fitted ask weights of Table 7 (left pair of columns) and the 14 fitted decision knobs (right pair), together the 34 coordinates of the vector searched in §8. Values are those selected at generation 8 of round five on validation seed 1357911 (500 deals, two rotations, panel {FishBot v0.3, lockout, detective, FishBot v0.2}) and recorded in `research/v04/runs/selected.json`; the same values are compiled into `V04Config` and are used unchanged in every experiment reported in §10. The two integer coordinates are shown before rounding: the patience pool is applied as 6 and the lookahead width K as 6. This table is a reproducibility record, not a description of strategy.

Ask feature	Weight	Decision knob	Value
hit probability	+11.506	declare threshold	0.818
squared hit	+3.295	locked threshold	0.797
certain hit	+3.198	ask floor	0.333
own progress	+1.688	patience pool	6.249
team control	+2.133	opp. card floor	2.868
lock completion	+4.071	value weight	6.043
continuation	+1.468	linear weight	0.767
completion bonus	+1.428	min. team prob.	0.792
reply threat	-3.098	stopping margin	-0.034
information leak	-0.854	prior θ	0.264
target hand size	-2.022	prior ϕ	0.133
empties the target	+1.166	two-ply K	5.624
repeats our half-suit	+1.270	chain weight	3.358
known team cards	+0.914	threat weight	2.613
location entropy	-2.653		
team owns H	-0.804		
exposure on a miss	+1.904		
trailing pressure	+0.047		
runway	+1.411		
leak magnitude	-0.999		

C.5 Value-function coefficients, and the provenance gap

The 16 features of V are named in Table 10 and defined as follows: a bias and the current score differential; three control terms built from e_H, the expected fraction of half-suit H our team holds, namely $\sum_H(2e_H-1)$, a sharpened version of it and the contested mass $\sum_H e_H(1-e_H)$; the signed count of half-suits already owned each way; counts of what is left to play for, from the card differential and the size of $|U|$ to the near-complete half-suits leaning each way; and side to move with its interactions with control and with $|U|$. Features are collected from all six seats at every decision point rather than from the mover alone, because under mover-only collection the side-to-move feature is constant at +1 and its coefficient — which the miss branch of (13) and the stopping rule (12) both depend on — would be unidentified.

The 16 coefficients of the value function V (§7.4) are not part of the 34-coordinate vector and are not written by the freeze step, which edits only the policy parameters. Two distinct vectors therefore exist in the repository, and they differ numerically: Table 10 prints both. The left column is what `V04Config::vw` contains and is what produced every result in §10. The right column is the ridge fit reported as E14 (405,348 rows, in-sample $R^2 = 0.2909$, RMSE 0.3047), stored in `research/v04/results/E14-valuefit.txt`.

The two vectors come from different runs, and no step in the pipeline copies the second into the first. The consequence is that the E14 metrics describe the quality of the artifact's fit and not the quality of the coefficients that were actually deployed, and that there is no reported goodness-of-fit figure — in-sample or held-out — for the deployed vector. We record this rather than repair it, because repairing it would change the configuration that produced every number in this paper.

Either vector can be supplied at the command line as a `vweights=` string, in the coordinate order

Table 9: Bounds applied to each coordinate of the 34-coordinate parameter vector when it is instantiated as a configuration. Coordinates 0–19 are the ask weights of Table 7 and are not clamped; coordinates 20–33 are the decision knobs. Bounds are identical in the engine parser and in the freeze step, so the vector scored during fitting and the vector compiled into the shipping binary are subject to the same constraints.

Coordinate	Quantity	Bound
0–19	ask weights $w_0, \ldots, w_{19}$	unbounded
20	declaration threshold	[0.5, 0.9999]
21	locked-half-suit threshold	[0.5, 0.99999]
22	ask floor	[0, 0.9]
23	patience pool	[0, 45], rounded to an integer
24	opponent-card floor	[0, 20]
25	value weight λ_{val}	$[0, \infty)$
26	linear weight λ_{lin}	$[0, \infty)$
27	minimum team probability	[0.05, 0.99]
28	stopping margin ε	unbounded
29	prior θ	[0, 2]
30	prior ϕ	[0, 1]
31	lookahead width K	[0, 24], rounded to an integer
32	chain weight λ_{chain}	$[0, \infty)$
33	threat weight λ_{threat}	$[0, \infty)$

of Table 10. The compiled vector is

```
0.001242|0.888965|0.421266|-0.145791|0.225573|0.022896|0.422207|0.007678|
0.005904|-0.000997|-0.006601|-0.007472|-0.022484|-0.025189|0.080416|-0.021409
```

and the E14 artifact vector is

```
0.007023|0.909779|0.115403|-0.249275|0.011511|0.026911|0.114251|-0.079727|
0.035886|-0.003278|-0.011850|0.062496|0.005226|-0.034832|0.027557|-0.019608
```

again concatenating the two lines with no intervening whitespace.

C.6 Successor choice and the forced-endgame ladder

Two decisions in the endgame described in §A.7 are made by the agent rather than by the rules, and both belong to the v0.4-Fast policy rather than to the driver.

The first is the successor choice. When a declaration leaves a player cardless and the turn reaches them, the policy hands it to the live teammate with the strongest publicly estimable ask: for each live teammate u it scores

$$\max_{H}\Big(\Pr[u \text{ holds a card of } H] \cdot \max_{c \in H} \Pr[c \text{ is with an opponent}]\Big)$$

over the live half-suits, using Fast per-card marginals, and gives the turn to the maximiser. The score is computed from the public record and the policy's own hand only, so no teammate's private holding enters it; an invalid choice is replaced by the driver with the lowest-indexed live teammate.

The second is the willingness bit. When a whole team is cardless, the driver sweeps the descending ladder of §A.7 and the policy answers, for each remaining half-suit at each level θ, whether it would declare that half-suit at confidence at least θ. The confidence used is $\hat{\alpha}$ from the same estimator that feeds (12), and only the bit crosses between seats. Under the FishBot v0.3 dialect the ladder has two levels only. Because the ladder restarts after each declaration, the earlier and safer declarations announce exact allocations and thereby sharpen the capacity constraints available to the ones that follow. How much of a confidence ordering among seats this recovers is not measured, and the ordering that would reveal it is excluded as an information leak.

Table 10: The 16 linear coefficients of the value function V. "Compiled" is the vector in `V04Config::vw` used by v0.4-Fast and v0.4-Block in every experiment reported here; "E14 artifact" is the separate ridge fit of `research/v04/results/E14-valuefit.txt`, obtained on 250 self-play deals at seed 31415. The two vectors differ because the freeze step writes only the 34 policy parameters and never the value coefficients.

j	Feature	Compiled	E14 artifact
0	bias	+0.001242	+0.007023
1	score differential	+0.888965	+0.909779
2	expected control	+0.421266	+0.115403
3	sharpened control	−0.145791	−0.249275
4	locked differential	+0.225573	+0.011511
5	side to move	+0.022896	+0.026911
6	card differential	+0.422207	+0.114251
7	unresolved pool	+0.007678	−0.079727
8	live half-suits	+0.005904	+0.035886
9	side to move × control	−0.000997	−0.003278
10	own hand size	−0.006601	−0.011850
11	smallest friendly hand	−0.007472	+0.062496
12	our near-complete half-suits	−0.022484	+0.005226
13	their near-complete half-suits	−0.025189	−0.034832
14	contested mass	+0.080416	+0.027557
15	side to move × unresolved	−0.021409	−0.019608

C.7 The policy-specification grammar

Every agent in the engine is named by a policy-specification string, parsed by `makeAgent` in `engine/src/factory.hpp`. The grammar is

`name` or `name:key=value,key=value,...`

where `name` selects the policy family and each `key=value` pair overrides one field of that family's configuration. Unrecognised keys are ignored; keys are order-independent; there is no whitespace. The example `v04:belief=block,decl=0.95,topk=6` constructs the v0.4-Fast policy family with the exact reference belief substituted (`belief=block`, i.e. v0.4-Block), the declaration threshold raised to 0.95, and the two-ply lookahead width set to $K = 6$; every other field keeps its compiled value.

The keys used in this paper are as follows. `belief` selects the inference path and accepts `fast` (the default, v0.4-Fast), `block` (the exact reference engine, v0.4-Block), `exact` (the card-level capacity dynamic program, without C5), `exactdisj`, `sinkhorn` (plain Sinkhorn, without C5), `indep` (no capacity conditioning) and `hybrid`. The ablation rows of §10.4 that use `exact`, `sinkhorn` and `indep` are therefore not exact-inference rows, and are labelled accordingly there. `decl`, `lockthr`, `minteam`, `askfloor`, `pool`, `oppfloor`, `vweight`, `lweight`, `vmargin`, `ptheta`, `pphi`, `topk`, `chain` and `threat` set the corresponding coordinates of Table 9. `gate` and `mgate` set the two cheap declaration pre-gates of §7.5; `gateaudit=1` enables the instrumentation used by E16. `value`, `vdecl`, `patient`, `gmap` and `declare` are boolean switches controlling, respectively, the one-ply value lookahead in (13), the value-based stopping rule of (12), the patience switch, greedy maximum-a-posteriori allocation naming, and declaration itself. `souter`, `sinner` and `particles` control the inference solver. Individual coefficients may be set as `w0`–`w19` and `v0`–`v15`, or in bulk as `weights=`, `vweights=` and `allparams=`.

The shipping specification used in every reported experiment is `v04:mgate=0.008`, which is equal to the compiled defaults; it is written explicitly in the run scripts so that the gate value appears in the recorded command line.

D Extended frozen-policy ablations

This appendix records the full design of experiment E5, the exact variant specification strings passed to the ablation driver, and the per-opponent breakdown that the summary in §10.4 compresses into a single win rate. The grouped summary itself is Table 2 and is not repeated here. It also gives the E12 belief-substitution comparison under a deliberately unfitted policy in full, and the method of the E16 declaration pre-gate audit whose results are reported in §10.1.

D.1 Design

The reference configuration is v0.4-Fast as shipped, the policy specification `v04:mgate=0.008`, which equals the compiled defaults. Every variant differs from it only in the switch named in its row; the 34-coordinate fitted parameter vector is held at its shipping values throughout, and no variant is refitted.

The experiment plays 500 deals against each member of a four-policy panel — FishBot v0.3, the turn-starvation lockout, the posterior detective and FishBot v0.2 — with each deal played in two seat rotations, giving 1,000 games per opponent per variant. The reported win rate is the mean over the four opponents; the reference configuration scores 77.50%. The seed bank is 606060, which is disjoint from every fitting and selection bank (§8). The command is

```
./fish ablate --ref=v04:mgate=0.008 --games=500 --rotations=2 --seed=606060 \
              --panel=v03,lockout,detective,v02 --variants="..."
```

and the artifact is `research/v04/results/E5-ablations.json`.

Because the reference and every variant play the identical deals in the identical rotations, the design is matched at the level of the deal. The interval in the "full − ablated" column is a paired percentile bootstrap in which the *deal*, not the game, is the resampling unit: deals are resampled with replacement, the paired difference is recomputed on each resample, and the endpoints are the 2.5% and 97.5% percentiles of 20,000 draws. Resampling deals rather than games is the relevant correction here, because the six (or, in E5, two) games generated from one deal are not independent observations.

> **Every row in E5 is a frozen-policy effect.** A row measures what happens to *this* fitted parameter vector when one component is removed or replaced. It does not measure the standalone value of that component, and it does not predict what a policy refitted without the component would score. No matched-budget refitting experiment was run.

D.2 Variant specification strings

Table 11 gives the specification string for each row exactly as it is passed to `fish ablate`, so that any row can be reconstructed with a single `fish match` invocation. The grammar is documented in §G. The rows of Table 11 and of Table 12 below carry the same "Change" labels, in the same order, as Table 2, so a row can be followed across all three.

Table 11: E5 variant specifications. Each string is the complete policy specification for one ablation row; the reference is `v04:mgate=0.008`. 500 deals × 2 seat rotations = 1,000 games against each of FishBot v0.3, the turn-starvation lockout, the posterior detective and FishBot v0.2; seed 606060; fitted weights frozen throughout.

Change	Specification string
v0.4-Fast (reference configuration)	`v04:mgate=0.008`

Change	Specification string
Drop capacity conditioning (C4) as well as the certificates	`v04:mgate=0.008,belief=indep`
Drop the ask-legality certificates (C5); plain Sinkhorn over C1–C4	`v04:mgate=0.008,belief=sinkhorn`
Drop the ask-legality certificates (C5); exact capacity DP over C1–C4	`v04:mgate=0.008,belief=exact`
v0.4-Block belief substituted into the Fast-fitted policy	`v04:mgate=0.008,belief=block`
Fast belief with the policy prior off (matches Block's prior-free belief)	`v04:mgate=0.008,belief=fast,ptheta=0,pphi=0`
Remove the fitted policy-aware prior	`v04:mgate=0.008,ptheta=0,pphi=0`
Zero the lock-completion weight	`v04:mgate=0.008,w5=0`
Zero the hit-probability weight	`v04:mgate=0.008,w0=0`
Remove the two-ply refinement	`v04:mgate=0.008,topk=0`
Zero both information-leak weights	`v04:mgate=0.008,w9=0,w19=0`
Remove the one-ply value lookahead	`v04:mgate=0.008,value=0`
Zero the reply-threat weight	`v04:mgate=0.008,w8=0`
Zero the runway weight	`v04:mgate=0.008,w18=0`
Declaration threshold 0.99	`v04:mgate=0.008,decl=0.99`
Name the allocation by conditioned greedy MAP	`v04:mgate=0.008,gmap=1`
Fixed threshold in place of the value-based stopping rule	`v04:mgate=0.008,vdecl=0`
Declaration threshold 0.80	`v04:mgate=0.008,decl=0.80`
Cash locked half-suits immediately	`v04:mgate=0.008,patient=0`

The `belief=fast,ptheta=0,pphi=0` row exists so that the belief-substitution row has a prior-matched comparator: the reference engine carries no fitted policy prior, so switching to it changes two things at once, and this row isolates the prior half of the change. Because `belief=fast` is already the default, it is numerically identical to the row that sets `ptheta=0,pphi=0` alone. The `patient=0` row is inert by construction: when the value-based stopping rule is active it decides first, so the immediate-cash switch is never consulted, and the paired difference is exactly zero with a zero-width interval. Both are reported for completeness rather than as evidence.

D.3 Belief-mode variants are not all exact

Three rows sit between 21.48% and 28.60% and it would be easy to read them as ablations of exact inference. They are not. `belief=exact` is the card-level capacity dynamic program *without* the ask-legality certificates (C5); `belief=sinkhorn` is plain Sinkhorn scaling with neither certificates nor the disjointness handling; `belief=indep` additionally drops capacity conditioning (C4), leaving independent per-card ownership estimates. Their paired differences from the reference are +48.90 (+47.00–+50.78), +52.78 (+50.98–+54.55) and +56.03 (+54.27–+57.75) points respectively, each a paired percentile bootstrap over deals. The exact reference engine of §5 is `belief=block` alone, and it appears in the row at 71.30%. The name `exact` in the specification grammar is a legacy identifier for the capacity dynamic program and is unfortunate; it is retained because it appears in the committed artifact.

D.4 Per-opponent breakdown

Table 12 gives the four per-opponent win rates behind each averaged row, read directly from `research/v04/results/E5-ablations.json`. The artifact records per-opponent win rates but not per-opponent intervals, so no uncertainty is attached to the individual columns; only the averaged difference carries the paired interval reported in Table 2. With 1,000 games behind each cell the resolution of a cell is 0.1 percentage points. The averaged win rate of each row is not repeated here, because it is the "win rate" column of Table 2; the paired difference is repeated so that each row's overall effect sits beside its per-opponent split, and the averaged win rate is 77.50% minus that difference.

Table 12: E5 per-opponent breakdown. Win rate of the frozen v0.4-Fast configuration or of the named variant against each panel member; 500 deals × 2 seat rotations = 1,000 games per cell, seed 606060, fitted weights frozen in every row. Rows carry the same labels and order as Table 2. "Full − ablated" is the averaged paired difference of that table, so the row's overall win rate is 77.50% minus it. Per-opponent cells carry no interval; the artifact reports intervals only for the averaged paired difference. Source: `research/v04/results/E5-ablations.json`.

Change	Full − ablated	FishBot v0.3	Lockout	Detective	FishBot v0.2
v0.4-Fast (reference configuration)	—	75.0%	78.3%	72.8%	83.9%
Drop capacity conditioning (C4) as well as the certificates	+56.03	21.1%	20.9%	19.0%	24.9%
Drop the ask-legality certificates (C5); plain Sinkhorn over C1–C4	+52.78	20.0%	23.2%	24.1%	31.6%
Drop the ask-legality certificates (C5); exact capacity DP over C1–C4	+48.90	27.2%	26.5%	28.1%	32.6%
v0.4-Block belief substituted into the Fast-fitted policy	+6.20	69.2%	70.4%	69.6%	76.0%
Fast belief with the policy prior off (matches Block's prior-free belief)	+3.88	69.8%	75.7%	69.6%	79.4%
Remove the fitted policy-aware prior	+3.88	69.8%	75.7%	69.6%	79.4%
Zero the lock-completion weight	+3.20	71.3%	74.1%	69.6%	82.2%
Zero the hit-probability weight	+1.55	76.9%	78.4%	69.2%	79.3%
Remove the two-ply refinement	+1.23	73.3%	77.2%	73.2%	81.4%
Zero both information-leak weights	+0.12	74.9%	76.7%	74.7%	83.2%
Remove the one-ply value lookahead	-0.33	77.3%	79.5%	72.4%	82.1%
Zero the reply-threat weight	-0.35	74.7%	78.9%	74.8%	83.0%
Zero the runway weight	-0.35	75.1%	76.7%	76.0%	83.6%
Declaration threshold 0.99	+0.20	74.4%	78.5%	72.8%	83.5%
Name the allocation by conditioned greedy MAP	+0.18	74.6%	78.2%	73.1%	83.4%
Fixed threshold in place of the value-based stopping rule	+0.12	76.1%	78.0%	73.5%	81.9%
Declaration threshold 0.80	+0.03	75.0%	78.0%	73.1%	83.8%
Cash locked half-suits immediately	+0.00	75.0%	78.3%	72.8%	83.9%

The three degraded-belief rows are degraded against all four opponents together, by between forty and sixty percentage points in every cell of the table above; no single panel member drives the averaged effect. Several of the small-effect rows are not uniform in sign across the panel. Zeroing the hit-probability weight raises the win rate against FishBot v0.3 (75.0% to 76.9%) and against the lockout (78.3% to 78.4%) while lowering it against the detective (72.8% to 69.2%) and FishBot v0.2 (83.9% to 79.3%); zeroing the runway weight raises it against the detective (72.8% to 76.0%) while lowering it against the lockout (78.3% to 76.7%). Where the averaged interval already includes zero, these sign reversals are consistent with sampling variation, and the per-opponent cells are not powered to distinguish that from a real opponent-specific effect.

D.5 Belief substitution under a deliberately unfitted policy (E12)

The belief-substitution row of Table 2 compares two beliefs under weights that were fitted around one of them. E12 repeats the substitution under a policy chosen so that neither belief has been fitted to: pure posterior-greedy asking, with the single ask weight $w_0 = 12$ and every other ask weight zero, no value term, no two-ply refinement, and no policy prior. The two specifications are

```
v04:weights=12|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0,\
    value=0,topk=0,ptheta=0,pphi=0,belief=fast

v04:weights=12|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0|0,\
    value=0,topk=0,ptheta=0,pphi=0,belief=block
```

(written on one line each in `engine/experiments.sh`; the backslash above is a line break for the page only). Each plays 400 deals in six seat rotations, 2,400 games, against FishBot v0.3 on seed bank 959595, under the full dialect.

The Fast belief scores 49.29% and the reference belief 48.17%. The 95% intervals are 47.29–51.25% and 46.29–50.04%, each a single-arm deal-clustered percentile bootstrap. The two arms share the seed bank, so the deals are matched, but the artifact reports those single-arm intervals rather than an interval on the difference; the paired quantity was not computed for this experiment.

The Fast arm's point estimate exceeds the Block arm's by just over one percentage point, in the same direction as the fitted comparison of Table 2. The intervals overlap substantially, so E12 does not resolve the question, and it does not establish that the direction of the fitted result is independent of the ask weights having been fitted around the Fast path. The experiment that would resolve it — refitting the entire policy against the reference belief under a matched optimisation budget and comparing the two fitted agents — was not run, and is listed as a limitation in §12.

D.6 Method of the declaration pre-gate audit (E16)

Table 13: Declaration pre-gate false-negative audit (E16). Population: the primary bank of 700 held-out deals in six seat rotations, seed 90210, all eight panel opponents, v0.4-Fast with policy weights frozen; the unit is one declaration opportunity, or for the first three rows one (opportunity, live half-suit) pair. Shares are exact fractions of the stated denominator; no intervals are constructed.

Quantity	Count	Share
Declaration opportunities examined	10,102,149	—
(opportunity, live half-suit) pairs	58,530,669	—
Pairs rejected by a cheap gate	24,114,786	41.20% of pairs
False negatives (rejected, would have been declared)	1,017	0.00422% of rejections
Declarations made, gated path	332,645	—
Declarations made, ungated path	333,661	—
Opportunities where the chosen action differs	1,016	0.0101% of opportunities

The results of E16 are in §10.1 and Table 13; this subsection records how the audit is constructed.

v0.4-Fast screens declaration candidates with two cheap scores before the expensive posterior query of §7.5 runs, and the two screens are separate because they cut at different stages and read different information. The first is `Knowledge::cheapTeamProb` in `engine/src/belief.hpp`, a capacity-only product: for each unresolved card of the half-suit it takes the share of the surviving support's total capacity that belongs to the declarer's team, and multiplies. It uses no dynamic program and no Sinkhorn iteration, so it can be evaluated before the posterior is refreshed at all. `proposeDeclaration`

in `engine/src/v04.hpp` applies it twice: once across all live half-suits, to decide whether to refresh the posterior for this opportunity at all, and once per half-suit, against the threshold `gateTeamProb`. The second screen lives inside `evaluateSet` and is a product of per-card team probabilities read off the refreshed posterior marginals, tested against `marginalGate`. It therefore prunes a half-suit that survived the capacity-only screen but whose posterior team mass is small, before the joint allocation query is run. Both thresholds are 0.008 in the shipping configuration, and both are bypassed when the unresolved pool has fallen to eight cards or fewer, or when the forced-endgame pressure level is non-zero.

With `gateaudit=1` the policy carries out a second, ungated evaluation alongside the one it acts on. At every declaration opportunity, and for *every* live half-suit rather than only the ones that survive the screens, `evaluateSet` is re-run with both gates disabled and the result is passed to the same stopping rule (12) at the same pressure level. The audited path changes no action: the declaration the policy actually makes is still the gated one, so the games played are the games of the primary bank.

A (opportunity, half-suit) pair counts as *rejected* when the gated path did not reach a full verdict on it — because no half-suit passed the first screen at that opportunity, or because this half-suit failed the per-half-suit screen, or because the gated `evaluateSet` returned no verdict while the ungated one did. A rejected pair counts as a *false negative* when the ungated evaluation plus the same stopping rule would have declared it. Separately, an opportunity is counted as one where the chosen action differs when the gated and ungated paths disagree about whether to declare at all, or agree to declare but select different half-suits. The two counts are not the same quantity: several pairs can be falsely rejected at one opportunity, and a falsely rejected pair changes nothing if the gated path declared a different half-suit at that opportunity anyway. Both counts, and the number of declarations each path made, are in Table 13.

The audit is run by

```
PANEL=v03,lockout,detective,v02,diversifier,hunter,bluffer,random
./fish gateaudit --games=700 --rotations=6 --seed=90210 --panel=$PANEL
```

which replays the primary bank of 700 deals in six seat rotations against all eight panel opponents with the fitted weights frozen, and accumulates the counters across every seat and every game. Every figure it reports is an exact count over that corpus; no interval is constructed, and the measurement is specific to this policy, this bank and the two thresholds compiled into the shipping configuration.

E Population, rules-dialect and arbitration results

E.1 The round-robin population matrix

Table 15 in §E.2 summarises the round-robin as Bradley–Terry ratings. Table 16 gives the win rates the ratings were fitted to, so that a reader can check any individual pairing rather than taking the rating on trust — which matters, because Bradley–Terry ratings are not redundancy-invariant and cannot represent cycles [75].

75.07% is the *lowest* win rate v0.4-Fast recorded against any member of this eight-policy panel, on this bank, under this rules dialect and under the default arbitration order of §2, two alternatives to which put it slightly lower (§10.5). It is not a claim about arbitrary opponents. Deals were held out from fitting and selection, opponent identities mostly were not — six of the eight panel members were also in the fitting panel, only the misdirection artist and the random legal control were withheld (§9.3) — so these experiments measure generalisation to new deals and only weakly to new strategic populations. That scoping governs every result in this section and is not repeated at each one. For scale, FishBot v0.3's worst published result against its three credible challengers was 56.50% [1].

Table 14: Primary head-to-head (E3). Population: 700 held-out deals in all six seat rotations, 4,200 games per row, seed 90210, full rules dialect, v0.4-Fast frozen at the shipping configuration before the bank was run, against the eight-policy panel. Intervals: deal-clustered percentile bootstrap on the per-deal win count, 20,000 resamples. "Mean half-suits" is half-suits won of nine; the two declaration columns count *voluntary* declarations only, forced endgame declarations being tallied separately in the run artifact and not shown here (§9.4). Action-limit rate 0.000% on every row.

Opponent	Win rate	95% CI	Mean half-suits	Ask acc.	Decl. acc.	Decl./game
FishBot v0.3	75.07%	73.71–76.40	5.457	55.81%	98.55%	4.80
Turn-starvation lockout	78.12%	76.88–79.36	5.542	48.01%	98.14%	5.16
Posterior detective	75.74%	74.45–77.02	5.440	52.79%	98.36%	5.12
FishBot v0.2	84.24%	83.12–85.33	5.857	54.42%	98.85%	5.17
Adaptive diversifier	92.14%	91.31–92.98	6.442	64.05%	97.42%	6.14
Focused hunter	97.64%	97.17–98.10	7.000	55.69%	97.51%	5.09
Misdirection artist	99.95%	99.88–100.00	8.161	55.12%	98.16%	3.80
Random legal control	100.00%	100.00–100.00	8.579	54.26%	96.72%	6.99

The margin is concentrated in declaration reliability rather than in ask conversion. Against the turn-starvation lockout v0.4-Fast converts the lowest fraction of asks in the panel (48.01%) and still wins by a wider margin (78.12%) than against the comparably strong FishBot v0.3 (75.07%, ask accuracy 55.81%) and posterior detective (75.74%), while declaring correctly 98.55% of the time on games where FishBot v0.3 manages 82.59%. Every incorrect declaration awards the half-suit to the opposing team in this dialect, so a policy that converts fewer asks can still win by giving away fewer half-suits. This describes where the observed margin sits; it is not a causal decomposition, and §10.4 reports what the frozen-policy ablations do and do not establish about the components involved. v0.4-Fast also uses the out-of-turn rule heavily, 3.43 declarations out of turn per game against FishBot v0.3; §10.5 reports how the margin moves without it.

E.2 Population ratings

Table 15: Bradley–Terry ratings (E4) in Elo with FishBot v0.3 at zero. Population: the round-robin of Table 16 — 200 deals in six seat rotations, 1,200 games per ordered pair, seed 515151, nine policies, all weights frozen. The mean win-rate column averages over the row's opponents. Ratings are point estimates fitted to the full matrix; no intervals are constructed for them.

Policy	Elo (v0.3 = 0)	Mean win rate
v0.4-Fast	+216	87.73%
Posterior detective	+8	68.84%
Turn-starvation lockout	+5	68.48%
FishBot v0.3	+0	68.02%
FishBot v0.2	-32	64.74%
Adaptive diversifier	-218	46.46%
Focused hunter	-441	29.60%
Random legal control	-759	13.26%
Misdirection artist	-1029	2.86%

E.3 Validating the ported FishBot v0.3 baseline

Every comparison in this paper is against a re-implementation of FishBot v0.3 in the v0.4 engine, so that re-implementation has to be shown faithful before its scores mean anything. Table 17 replays it

Table 16: Round-robin win rates (E4), row policy against column policy, per cent. Population: 200 deals in all six seat rotations for every ordered pair, seed 515151, full rules dialect, all weights frozen. Each cell pools both orderings, so a cell is 2,400 games, 1,200 per ordered pair. Cells are point estimates; no intervals per cell.

Row vs column	v0.4-Fast	FishBot v0.3	FishBot v0.2	Lockout	Detective	Diversifier	Hunter	Misdirection	Random
v0.4-Fast	—	75.7	83.5	78.7	74.0	92.7	97.3	99.9	100.0
FishBot v0.3	24.3	—	51.5	49.5	49.8	78.8	91.0	99.3	99.9
FishBot v0.2	16.5	48.5	—	41.6	44.8	76.0	91.0	99.6	99.9
Turn-starvation lockout	21.3	50.5	58.4	—	49.9	77.0	91.2	99.5	100.0
Posterior detective	26.0	50.2	55.2	50.1	—	76.9	92.8	99.5	100.0
Adaptive diversifier	7.3	21.2	24.0	23.0	23.1	—	74.4	99.5	99.3
Focused hunter	2.7	9.0	9.0	8.8	7.2	25.6	—	93.1	81.5
Misdirection artist	0.1	0.7	0.4	0.5	0.5	0.5	6.9	—	13.4
Random legal control	0.0	0.1	0.1	0.0	0.0	0.7	18.5	86.6	—

under the v0.3 rules dialect on the v0.3 seed bank and compares against the published figures.

Table 17: FishBot v0.3 as published [1] versus the C++ port. Population: 1,000 deals in two orientations, 2,000 games per row, seed 20260820, v0.3 dialect, the seven opponents of the v0.3 study, ported weights frozen. Intervals: deal-clustered percentile bootstrap on the port's win rate, 20,000 resamples.

Opponent	Published v0.3	C++ port	95% CI
Turn-starvation lockout	57.85%	57.60%	55.50–59.75
Posterior detective	57.20%	59.10%	56.95–61.25
FishBot v0.2	56.50%	56.60%	54.50–58.70
Adaptive diversifier	86.15%	84.75%	83.20–86.30
Focused hunter	95.15%	94.55%	93.55–95.50
Misdirection artist	99.20%	98.70%	98.20–99.15
Random legal control	100.00%	100.00%	100.00–100.00

The engine implements every consequential rule choice of §2 as a switch, so the same frozen v0.4-Fast configuration can be replayed under other dialects and under other resolutions of simultaneous voluntary declarations without recompiling or refitting. This appendix reports both sweeps. Neither is a claim about which dialect is correct; both are sensitivity analyses on the modelling choices the main results are conditioned on.

E.4 Rules dialects

Table 18 shows that the size of the margin depends on the dialect, and that the dependence is not in the same direction for every opponent.

Against FishBot v0.3, the margin is most sensitive to the declaration timing rules. Under the v0.3 dialect, in which a player may declare only on their own turn, v0.4-Fast wins 64.46% (95% deal-clustered percentile-bootstrap interval 62.58–66.38). Under the full dialect with out-of-turn declarations disabled but everything else intact it wins 67.71% (65.92–69.50). Both are below the 75.07% recorded under the full dialect in E3. The 48-card, eight-half-suit form gives 72.54% (70.79–74.21).

Against the turn-starvation lockout the ordering is different. Disabling out-of-turn declarations raises the margin to 80.17%, above the 78.12% of E3, while the v0.3 dialect gives 77.25% and the 48-card form gives 74.79%, the lowest of the three. Against the posterior detective the v0.3 dialect gives the highest of the three figures, 76.21%, against 75.74% under the full dialect in E3; disabling out-of-turn declarations gives 74.75% and the 48-card form 72.21%. The interval for each of these six cells is in Table 18 and is a deal-clustered percentile bootstrap on the same construction.

Two quantities do not move across the sweep. No cell in E7 recorded an action-limit game: on these three banks, under the graduated forcing rule of §6, every game terminated through play. That

Table 18: E7: the frozen v0.4-Fast configuration (`v04:mgate=0.008`, the 34-coordinate fitted vector unchanged, no refitting) replayed under three rules dialects against three opponents. Each cell is 400 deals played in six seat rotations, 2,400 games; seeds 828282 (v0.3 dialect), 838383 (48-card, eight-half-suit form) and 848484 (out-of-turn declarations disabled), all disjoint from the fitting and selection banks. "Win rate" is v0.4-Fast's share of games won; intervals are deal-clustered percentile bootstrap, the 2.5% and 97.5% percentiles of 20,000 resamples of deals. Every row recorded an action-limit rate of 0.000%. The full-dialect comparators quoted in the text come from E3 (Table 14), which is a different bank (700 deals, seed 90210), so the comparison is across banks as well as across dialects.

Rule dialect	Opponent	Win rate	95% CI
v0.3 dialect (turn-only declarations)	FishBot v0.3	64.46%	62.58–66.38
48-card, eight half-suits	FishBot v0.3	72.54%	70.79–74.21
No out-of-turn declarations	FishBot v0.3	67.71%	65.92–69.50
v0.3 dialect (turn-only declarations)	Turn-starvation lockout	77.25%	75.58–78.88
48-card, eight half-suits	Turn-starvation lockout	74.79%	73.17–76.42
No out-of-turn declarations	Turn-starvation lockout	80.17%	78.46–81.83
v0.3 dialect (turn-only declarations)	Posterior detective	76.21%	74.42–78.00
48-card, eight half-suits	Posterior detective	72.21%	70.50–73.92
No out-of-turn declarations	Posterior detective	74.75%	73.00–76.46

is a measurement on 2,400 games per cell, not a termination guarantee, and it does not extend to dialects or configurations outside the three tested here (§6.3). And v0.4-Fast retains a substantial margin in every cell: the smallest figure in the table is 64.46%. The dialect changes the size of the margin, not its sign, over the three dialects and three opponents tested. We do not attempt to explain the opponent-specific orderings; disentangling them would require an experiment that varies one rule at a time against a policy whose response to that rule is independently characterised, which E7 does not do.

E.5 Simultaneous-declaration arbitration

When more than one player would offer a declaration at the same declaration opportunity, the driver must choose one. The shipped choice is the lowest seat index. This is a modelling decision rather than a rule of the game, and the main results are conditioned on it, so E17 replays the primary bank under all three orders the engine supports: lowest seat index (`--arb=low`, the default), highest seat index (`--arb=high`) and a scan beginning from the current turn-holder (`--arb=turn`).

Across the nine cells of Table 19 the win rate moves by at most 0.7 percentage points, and every deal-clustered percentile-bootstrap interval overlaps the corresponding intervals under the other two orders. Against FishBot v0.3 the three orders give 75.07%, 74.93% and 74.38%; against the lockout, 78.12%, 78.05% and 78.19%; against the detective, 75.74%, 75.74% and 75.76%. Declarations per game are unchanged to the reported precision under all three orders (Table 19).

The one quantity that does move is the rate of out-of-turn declarations. Under either seat-index order it is between 3.4 and 3.6 per game; under the scan from the turn-holder it falls to between 2.06 and 2.12. That is the expected consequence of the change — the turn-holder is examined first and, when it also offers a declaration, resolves the opportunity in turn rather than out of turn — and it is a change in how declarations are labelled at least as much as a change in which declarations happen, since the total declaration rate is unchanged.

The reported margins are therefore not an artifact of the arbitration order. Two limits on that statement should be kept in view. Only three orders were tested, all of them deterministic functions of seat index or turn position; a randomised tie-break, or an order that depends on the game state in some other way, was not run. And one candidate order was deliberately excluded: ranking simultaneous

Table 19: E17: sensitivity of the primary head-to-head results to the order in which simultaneous voluntary declarations are arbitrated. The frozen v0.4-Fast configuration (`v04:mgate=0.008`, no refitting) replayed against three opponents under three arbitration orders; 700 deals played in six seat rotations, 4,200 games per cell, seed 90210, full dialect. "Win rate" is v0.4-Fast's share of games won; intervals are deal-clustered percentile bootstrap, the 2.5% and 97.5% percentiles of 20,000 resamples of deals. The last column is v0.4-Fast's out-of-turn declarations per game. The lowest-seat rows are the shipped configuration and coincide with Table 14.

Arbitration order	Opponent	Win rate	95% CI	Out-of-turn decl./game
Lowest seat index (default)	FishBot v0.3	75.07%	73.71–76.40	3.43
	Turn-starvation lockout	78.12%	76.88–79.36	3.59
	Posterior detective	75.74%	74.45–77.02	3.57
Highest seat index	FishBot v0.3	74.93%	73.60–76.26	3.46
	Turn-starvation lockout	78.05%	76.81–79.26	3.58
	Posterior detective	75.74%	74.43–77.02	3.59
Scan from the current turn-holder	FishBot v0.3	74.38%	73.02–75.74	2.06
	Turn-starvation lockout	78.19%	76.95–79.43	2.12
	Posterior detective	75.76%	74.48–77.05	2.11

declarers by their stated confidence in their own allocation. That rule would compare private quantities across seats, so it would let the arbitration mechanism act as a channel through which one player's belief influences the outcome of another player's decision, and the resulting simulator would no longer satisfy the information-safety property audited in E1.

F Calibration detail

The aggregate scores are in Table 1 and the reliability diagrams in Figure 2. This appendix gives the bin-by-bin counts behind them and states what they do and do not support.

Table 20: E6: bin-by-bin reliability of the two forecasts v0.4-Fast makes, Pr[ask succeeds] and Pr[allocation correct]. Population is 600 deals of v0.4-Fast against FishBot v0.3 under the full dialect, seed 717171, fitted weights frozen; the experimental unit is the individual forecast, not the deal or the game, and forecasts are pooled across all six seats. Declaration forecasts pool voluntary and forced declarations, which are not separated in the artifact. "n" is the number of forecasts falling in the bin, "forecast" is their mean predicted probability and "observed" the realised frequency. A dash means no forecast fell in that bin. Every entry is a point estimate: no clustered interval is attached to any count, mean or frequency in this table. Source: `research/v04/results/E6-calibration-v04.txt`.

	Pr[ask succeeds]			Pr[allocation correct]		
Forecast bin	n	forecast	observed	n	forecast	observed
$[0.0, 0.1)$	6,341	0.0291	0.0126	34	0.0000	0.0000
$[0.1, 0.2)$	4,711	0.1557	0.1407	—	—	—
$[0.2, 0.3)$	7,023	0.2449	0.2876	—	—	—
$[0.3, 0.4)$	6,547	0.3483	0.4100	—	—	—
$[0.4, 0.5)$	5,096	0.4496	0.5057	—	—	—
$[0.5, 0.6)$	4,681	0.5402	0.5791	88	0.5339	0.5227
$[0.6, 0.7)$	2,216	0.6446	0.6918	74	0.6624	0.8243
$[0.7, 0.8)$	2,089	0.7522	0.7037	415	0.7434	0.9663
$[0.8, 0.9)$	1,359	0.8496	0.8205	221	0.8413	0.9412
$[0.9, 1.0)$	16,141	0.9980	0.9963	4,984	0.9989	0.9986

F.1 Declaration forecasts

The declaration column of Table 20 contains 5,816 forecasts distributed over six occupied bins.

Thirty-four forecasts fall in $[0.0, 0.1)$ with a mean predicted probability of 0.0000 and a realised frequency of 0.0000. The estimator assigned probability zero to all thirty-four, and none succeeded. The artifact does not label a forecast as voluntary or forced, so the table does not record which decision path produced these thirty-four.

Eighty-eight forecasts fall in $[0.5, 0.6)$, predicted 0.5339 and realised 0.5227. Seventy-four fall in $[0.6, 0.7)$, predicted 0.6624 and realised 0.8243. Four hundred and fifteen fall in $[0.7, 0.8)$, predicted 0.7434 and realised 0.9663. Two hundred and twenty-one fall in $[0.8, 0.9)$, predicted 0.8413 and realised 0.9412. And 4,984 — 85.7% of all declaration forecasts — fall in $[0.9, 1.0)$, predicted 0.9989 and realised 0.9986.

The estimator is close to unbiased where almost all of its mass lies. In the top bin the predicted and realised values differ by 0.0003, and that bin carries 85.7% of the forecasts. The aggregate scores — expected calibration error 0.0222, Brier score 0.0155, log loss 0.0592, mean predicted 0.9575 against mean observed 97.89% — are therefore dominated by that bin, and are a weak test of behaviour anywhere else.

The three bins in $[0.6, 0.9)$ are *under*confident, and by a margin much larger than the top bin's error: 16.2 percentage points in $[0.6, 0.7)$, 22.3 in $[0.7, 0.8)$ and 10.0 in $[0.8, 0.9)$. The bin at $[0.5, 0.6)$ is close to unbiased, 1.1 points in the other direction. Underconfidence here means that $\hat{\alpha}$ assigns a lower probability to an allocation being correct than the realised frequency in the bin warrants. That is a statement about the estimator and nothing more. The declaration decision is made by (12), which consumes the fitted value model as well as $\hat{\alpha}$, and no experiment in this paper traces bin membership through to a declaration, so these bins do not say how often the policy declared, declined to declare, or would have succeeded had it declared.

The counts in those bins are small: 74, 415 and 221 forecasts respectively, out of 5,816. The three-bin block totals 710 forecasts, about one eighth of the corpus. A realised frequency computed from 74 outcomes has substantial sampling error even before the clustering of forecasts within deals is accounted for, and neither source of uncertainty is quantified here.

F.2 The FishBot v0.3 declaration forecasts for contrast

The same experiment records FishBot v0.3's declaration forecasts on the same bank. Its 4,984 forecasts occupy three bins: 403 in $[0.7, 0.8)$, predicted 0.7895 and realised 0.1935; 452 in $[0.8, 0.9)$, predicted 0.8459 and realised 0.3341; and 4,129 in $[0.9, 1.0)$, predicted 0.9722 and realised 0.9346. In aggregate the mean predicted probability is 94.59% against a mean observed 82.02%, an expected calibration error of 0.1257.

The contrast is in both magnitude and direction. FishBot v0.3's middle bins are *over*confident by 60 and 51 percentage points respectively, and its top bin by nearly 4; v0.4-Fast's middle bins are underconfident by 16.2, 22.3 and 10.0 points and its top bin is accurate to within 0.03 points. The two agents' declaration accuracies on the primary head-to-head bank, 98.55% and 82.59%, are consistent with that difference, but E6 is a separate bank from E3 and the two should not be read as one measurement.

F.3 Ask forecasts

The ask column of Table 20 covers 56,204 forecasts and is spread across all ten bins, with aggregate Brier score 0.1297, log loss 0.3790 and expected calibration error 0.0288; mean predicted 0.5338 against mean observed 0.5504. The deviations are smaller than on the declaration side and change sign across the range: the lowest bin is an over-estimate, predicting 0.0291 against a realised 0.0126; the bins from

$[0.2, 0.3)$ to $[0.6, 0.7)$ are underconfident by 4 to 6 points; and $[0.7, 0.9)$ turns over again, predicting 0.7522 against 0.7037 and 0.8496 against 0.8205. The largest bin, $[0.9, 1.0)$ with 16,141 forecasts, predicts 0.9980 against a realised 0.9963.

F.4 What these numbers are not

Every quantity in this appendix, in Table 1 and in Figure 2 is a point estimate. No clustered interval is attached to any of them. Forecasts are not independent observations: the forecasts made within one deal share a deal, and the forecasts made within one game share a game history, so the effective sample size behind every bin is smaller than its n. Computing deal-clustered intervals for the calibration metrics is straightforward with the machinery already used for the win rates and was not done; it is listed among the open items in §12.

The pooling of voluntary and forced declarations is a second limitation of the same table. Forced declarations run at well under a tenth per game in E3, so their contribution to 5,816 forecasts is small, but it is not zero, and the two populations are made under different rules — a forced declaration is made because the endgame requires one, at whatever probability the estimator reports. The artifact does not separate the two populations in any bin, so no bin of Table 20 can be attributed to one of them.

These are descriptive statistics about a forecast and its realised frequency. They do not identify why the policy declares when it does, and they should not be used to attribute the difference in declaration accuracy between v0.4-Fast and FishBot v0.3 to any particular mechanism. The ablations of §D address which components the fitted policy depends on, and they do not isolate the stopping rule.

G Reproducibility and artifact manifest

G.1 Source layout

The engine is a single C++20 translation unit assembled from headers in `engine/src/`. The rules, the card and half-suit types and the action encoding are in `engine/src/fish.hpp`. Constraint tracking (C1–C5) and the Fast posterior are in `engine/src/belief.hpp`; the exact reference engine — the block-allocation dynamic program, its partition function, its marginals and its sampler — is in `engine/src/blockdp.hpp`; the brute-force oracle that validates it is in `engine/src/oracle.hpp`. The policy is in `engine/src/v04.hpp` and the ported FishBot v0.3 population, together with the other baselines, is in `engine/src/baselines.hpp`. The game driver, which owns turn order, declaration rounds and the information-safety audit, is in `engine/src/game.hpp`; the match runner and both bootstrap procedures are in `engine/src/arena.hpp`; the cross-entropy optimiser is in `engine/src/tuner.hpp`. The policy-specification parser is in `engine/src/factory.hpp` and the command-line entry point in `engine/src/main.cpp`.

Build with `make` in `engine/`. The build produces a single binary, `engine/fish`.

G.2 Commands

The following commands are those in `engine/experiments.sh` and `engine/exploitability.sh`, with the shell's own variables expanded to the shipping policy specification. Results are written to `research/v04/results/` as JSON, JSONL or text; `engine/build_tables.py` turns them into the tables of this paper and `engine/build_manifest.py` into the manifest below.

Throughout the command-line interface `--games` counts *deals*, not games: each deal is replayed in the number of seat rotations given by `--rotations`, and `fish bench` always times each deal in two

seat orientations. The game counts in the tables are therefore the product of the deal count and the rotation count — 700 deals in six rotations give the 4,200 games of Table 14, and the three `bench` runs below give the game counts in Table 4.

```
# ---- build ------------------------------------------------------
cd engine && make

# ---- E1  engine and information-safety verification (9x9 pool) -----
./fish verify   --games=600
./fish verify   --games=300 --legacy

# ---- E2  belief-engine cross-checks -----------------------------
./fish selftest --games=40

# ---- E15 brute-force allocation oracle --------------------------
./fish oracle   --games=150 --maxdeals=200000 --samples=3000 \
                --seed=20260822

# ---- E16 declaration pre-gate false-negative audit --------------
PANEL=v03,lockout,detective,v02,diversifier,hunter,bluffer,random
./fish gateaudit --games=700 --rotations=6 --seed=90210 --panel=$PANEL

# ---- E3  primary head-to-head (repeat for each of the 8 opponents) --
./fish match --a=v04:mgate=0.008 --b=v03 --games=700 --rotations=6 \
             --seed=90210 --json

# ---- E4  round-robin matrix over the nine-policy population --------
P=v04:mgate=0.008,v03,v02,lockout,detective,diversifier
P=$P,hunter,bluffer,random
./fish matrix --policies=$P --games=200 --rotations=6 --seed=515151

# ---- E5  frozen-policy ablations (18 variant strings: Appendix D) --
./fish ablate --ref=v04:mgate=0.008 --games=500 --rotations=2 \
              --seed=606060 --panel=v03,lockout,detective,v02 \
              --variants="v04:mgate=0.008,belief=block;..."

# ---- E6  calibration --------------------------------------------
./fish calibrate --a=v04:mgate=0.008 --b=v03 --games=600 --seed=717171
./fish calibrate --a=v03 --b=v04:mgate=0.008 --games=600 --seed=717171

# ---- E7  rules dialects (repeat for v03, lockout, detective) --------
./fish match --a=v04:mgate=0.008 --b=v03 --games=400 --rotations=6 \
             --seed=828282 --legacy --json
./fish match --a=v04:mgate=0.008 --b=v03 --games=400 --rotations=6 \
             --seed=838383 --sets=8 --json
./fish match --a=v04:mgate=0.008 --b=v03 --games=400 --rotations=6 \
             --seed=848484 --no-out-of-turn --json

# ---- E8  v0.3 port validation under the v0.3 dialect ----------------
```

```
./fish match --a=v03 --b=lockout --games=1000 --rotations=2 \
             --seed=20260820 --legacy --json

# ---- E9  whole-game throughput ---------------------------------------
#      --games counts DEALS, and bench times each deal in two seat
#      orientations, so these three runs are the 400, 120 and 800
#      games reported in the throughput table.
./fish bench --a=v04:mgate=0.008 --b=v04:mgate=0.008 --games=200
./fish bench --a=v04:mgate=0.008,belief=block \
             --b=v04:mgate=0.008,belief=block --games=60
./fish bench --a=v03 --b=v03 --games=400

# ---- E12 belief substitution under an unfitted policy ---------------
#      the two full specifications are given in Appendix D
./fish match --a="v04:$GREEDY,belief=fast"  --b=v03 --games=400 \
             --rotations=6 --seed=959595 --json
./fish match --a="v04:$GREEDY,belief=block" --b=v03 --games=400 \
             --rotations=6 --seed=959595 --json

# ---- E13 action-cap incidence, including the mirror match -----------
./fish match --a=v04:mgate=0.008 --b=v04:mgate=0.008 --games=300 \
             --rotations=6 --seed=464646 --json

# ---- E14 ridge fit of the value function ----------------------------
./fish fitvalue --a=v04:mgate=0.008 --b=v04:mgate=0.008 --games=250 \
                --seed=31415

# ---- E17 simultaneous-declaration arbitration sweep -----------------
for arb in low high turn; do
  for opp in v03 lockout detective; do
    ./fish match --a=v04:mgate=0.008 --b=$opp --games=700 \
                 --rotations=6 --seed=90210 --arb=$arb --json
  done
done

# ---- policy fitting (cross-entropy method) --------------------------
# Five rounds were run; --gens and --seed differed per round, and the
# per-round generation counts are recorded in the traces under
# research/v04/runs/.  Round 5's base seed was not captured.
./fish tune --panel=v03,lockout,detective,v02,diversifier,hunter \
            --base=v04 --full --gens=<per round> --seed=<per round>

# ---- E10 local response probe: fit an exploiter, re-measure it ------
./fish tune --panel=v04:mgate=0.008 --base=v04 --full --games=180 \
            --pop=18 --elite=5 --gens=12 --beta=1 --sigma=0.7 \
            --seed=515253 --out=E10-lbr-v04.jsonl
./fish match --a="v04:allparams=$w" --b=v04:mgate=0.008 --games=600 \
             --rotations=6 --seed=6543210 --json
```

```
# ---- the whole battery ------------------------------------------------
./experiments.sh
./exploitability.sh
```

G.3 The policy-specification grammar

Every agent in every experiment is named by a string, parsed in `engine/src/factory.hpp`. The grammar is

```
spec  := name | name ":" opt ("," opt)*
opt   := key "=" value
name  := v04 | v03 | v02 | lockout | detective
       | hunter | diversifier | bluffer | random
```

Unrecognised keys are ignored, so a specification is always parseable and a misspelt key fails silently; this is worth knowing when reconstructing a run. For the `v04` base the keys used in this paper are `belief` (one of `fast`, `block`, `exact`, `exactdisj`, `sinkhorn`, `indep`, `hybrid`), `decl` (declaration threshold), `lockthr` (locked-allocation threshold), `minteam`, `vmargin`, `patient`, `askfloor`, `pool`, `force`, `oppfloor`, `gate` and `mgate` (the two cheap declaration pre-gates), `souter` and `sinner` (Sinkhorn iteration counts), `value`, `vweight`, `lweight`, `vdecl`, `ptheta` and `pphi` (the policy prior), `gmap`, `topk` (two-ply search width), `chain`, `threat`, `declare` and `gateaudit`. Individual coefficients are addressed as `w0` … `w19` for the 20 ask features and `v0` … `v15` for the 16 value features, and whole vectors as `weights=` and `vweights=`, pipe-separated.

The reconstruction key is `allparams=`. It takes the full fitted vector as a pipe-separated list. The leading 20 entries are the ask weights $w_0 \dots w_{19}$; the remaining fourteen are the decision knobs, in this order: declaration threshold, locked-allocation threshold, ask floor, patience pool, opponent-card floor, value weight, linear weight, minimum team probability, stopping margin, prior θ, prior ϕ, two-ply width K, chain weight, threat weight. That is 34 coordinates in all. Each knob is clamped to its admissible range on parse, so a vector written by the optimiser round-trips to the same configuration. The offset at which the knobs begin is derived from the ask-feature count rather than hard-coded; the hard-coded form aliased two ask weights onto the leading two knobs once the ask feature count grew to 20, and the fixed form is what produced every result reported here. `engine/freeze_config.py` asserts the 34-coordinate length when it bakes a fitted vector into the compiled defaults.

G.4 Artifact manifest

Table 21 is generated by `engine/build_manifest.py` from the artifact files themselves: the script walks `research/v04/results/`, records each file's size and SHA-256, and pairs it with the command that produced it. It is not maintained by hand.

Table 21: Artifact manifest for the experiments reported in this paper. "ID" is the experiment identifier used throughout the text, "Artifact" the file under `research/v04/results/`, "Design" the population and experimental unit recorded by the generator, and the last column the first twelve hex digits of the file's SHA-256 digest. Generated by `engine/build_manifest.py`; the full digests, together with the byte counts and the exact command line for each row, are in `research/v04/results/MANIFEST.json`.

ID	Artifact	Design	SHA-256 (first 12)
E1	`E1-verify.txt`	engine and information-safety audit, 9×9 policy cross-product, full dialect	`1cf3b1e6f994`
E1L	`E1-verify-legacy.txt`	the same audit under the v0.3 rules dialect	`4651588f2627`

ID	Artifact	Design	SHA-256 (first 12)
E2	`E2-belief-selftest.txt`	reference engine vs card-level DP vs exact rejection sampling	`7145c2b68ef8`
E3	`E3-headtohead.jsonl`	primary head-to-head; 700 deals × 6 rotations = 4,200 games per opponent	`5131b8f001fa`
E4	`E4-matrix.json`	round-robin over nine policies; 200 deals × 6 rotations per ordered pair	`8c214b0404d1`
E5	`E5-ablations.json`	frozen-policy ablations; 500 deals × 2 rotations against a four-policy panel	`32f0c3eb2fe5`
E6	`E6-calibration-v04.txt`	forecast reliability, v0.4-Fast	`a40617ce7a60`
E6b	`E6-calibration-v03.txt`	forecast reliability, FishBot v0.3	`00d15f780942`
E7	`E7-rules.jsonl`	rules-dialect sensitivity; 400 deals × 6 rotations per cell	`9e1c6f857762`
E8	`E8-v03-port.jsonl`	v0.3 port validation against the published v0.3 figures	`658945dd10b0`
E9	`E9-throughput.txt`	whole-game throughput, v0.4-Fast vs v0.4-Block vs v0.3	`d44073f72621`
E10	`E10-lbr.jsonl`	local response probe; response fitted, then re-measured on 600 deals × 6 rotations	`491111e8d214`
E11	`E11-termination.md`	termination study of four forcing rules in mirror self-play	`1260bf5fe4c9`
E12	`E12-exactness.txt`	belief substitution under a deliberately unfitted policy	`961b3aa73a49`
E13	`E13-termination.jsonl`	action-cap incidence across the population; 300 deals × 6 rotations, including the mirror match	`1a6f0dc047f9`
E14	`E14-valuefit-stats.txt`	ridge fit of the value function on self-play decision points	`8b353b8c92f9`
E15	`E15-oracle.txt`	brute-force oracle for the exact reference engine	`6ea42ed28011`
E16	`E16-gateaudit.txt`	false-negative audit of the declaration pregates over the primary bank	`719c9e7c904e`
E17	`E17-arbitration.jsonl`	sensitivity of the results to the simultaneous-declaration arbitration order	`2a4afff2422b`

`research/v04/results/MANIFEST.json` additionally records the commit the manifest was generated at, whether the working tree was clean at that moment, the shipping policy specification (`v04:mgate=0.008`), the belief mode (`Fast`), the fitted-parameter count (34), the ask- and value-feature counts (20 and 16), the recorded fitting base seeds, the selection seed and selected generation, and every evaluation seed bank used in this paper. Fitting traces are in `research/v04/runs/`; the authoritative record of which configuration was selected, and on what, is `research/v04/runs/selected.json` together with `research/v04/runs/selection.log`, which record generation 8 of the round-5 trace, chosen on validation bank 1357911.

G.5 Known reproducibility gaps

The base seed for round 5 of the fitting procedure is not recorded in any committed script or artifact; `research/v04/results/MANIFEST.json` carries it as null. Rounds 1 to 4 are recorded (20260821, 770077, 313131, 888111). The round-5 trace itself is committed, as `research/v04/runs/tune-round5.jsonl`, so the generation means from which the shipping configuration was selected can be inspected and the selection step repeated; the sampling of that round cannot be reproduced exactly.

The sixteen value-function coefficients compiled into the shipping policy are not the coefficients

in `research/v04/results/E14-valuefit.txt`. `engine/freeze_config.py` writes the 34-coordinate ask-and-knob vector but not the value vector, so the E14 fit and the shipped value function are separate objects. The E14 statistics — 405,348 rows, $R^2 = 0.2909$, RMSE = 0.3047 — describe the fit reported in that artifact and must not be read as a validation of the coefficients actually used.

Experiments E1–E14 were produced at commit `bb3bc8a` (`bb3bc8afc92c848b421af87941c1a91de3bc5300`). Experiments E15 (the brute-force oracle), E16 (the pre-gate audit) and E17 (the arbitration sweep) are newer than that commit and are regenerated by the commands in §G.2; `research/v04/results/MANIFEST.json` records the working tree as dirty at generation time for that reason. There is no single commit hash covering the whole battery.

H Extended implications for human play

v0.4-Fast maintains a 54×6 posterior and re-scores its candidate asks after every public event. A person at a table cannot. The mechanisms that the frozen-policy ablations of §D identify as the ones this configuration depends on most heavily are, however, not the expensive ones, and several of them can be run by hand. This appendix records them at length; the condensed version is in §11.3.

H.1 How to read the labels

Each item below carries one of three tags, because the recommendations differ sharply in how well they are supported.

[R] A logical consequence of the rules dialect of §2. It holds for any opponent and requires no experiment. A tag of **[R]** is never attached to a prescription about how to play; where a rules consequence suggests a course of action, the action is split off and tagged separately.

[M] A measured result. The experiment is named. In every case the measurement is of *this* fitted configuration on a specific bank against a specific panel, and in the case of an ablation it is a frozen-policy effect rather than the standalone value of the component.

[U] An untested suggestion. It is consistent with the design of the agent or with the rules, but no experiment in this paper isolates it, and it should be treated as a hypothesis.

H.2 What an ask tells the table

[R] An ask is a confession about the asker. Under this dialect a player may ask for a card only in a half-suit in which they already hold at least one card. So a player who asks for the queen of hearts has stated two things publicly: that they do not hold the queen of hearts, and that they hold at least one other card of the high hearts. The second is the one that is easy to miss. It applies to a teammate's ask exactly as much as to an opponent's.

[R] A miss is a second confession, about the target. A failed ask establishes that the named card was not in the target's hand at that moment. The exclusion stands until a public transfer moves that card, and every transfer in this game is public, so the exclusion is never silently invalidated.

[R] The content of both events is an exclusion, not an impression. What an ask and a miss add to the public record is a collection of statements of the form "this card is not held by this player" together with a disjunction over the asker's own hand. "Somebody wanted low clubs" is not among them; "the seven of clubs is not held by seat 2 or seat 4, and seat 3 holds at least one of the other five low clubs" is.

[U] Write them down in that form. Exclusions and disjunctions combine with hand counts, and impressions do not, so a person keeping a record by hand is better served by the first. No experiment in this paper evaluates any human bookkeeping scheme.

[M] The certificates that encode these two deductions are removed by the largest frozen-policy ablations in the study. In E5, dropping the ask-legality certificates — while keeping capacity conditioning, keeping every exclusion from a miss, and keeping the entire decision policy unchanged — moved the frozen policy from 77.50% to 24.73%. Dropping capacity conditioning as well moved it further, to 21.48%. These are the largest decreases in the study, but none of them isolates a single constraint: the implementation applies the fitted policy prior only along the path these rows replace, so each removes the prior too (§10.4). These are effects on this frozen parameter vector; they do not say what a policy refitted without those constraints would score.

H.3 Counting

[R] Hand sizes are public, and they close deductions that exclusions alone leave open. If a player holds four cards and you have already located four specific cards in their hand, every other unresolved card is excluded from them at once. Conversely, if a player holds four cards and only four unresolved cards can possibly be theirs, they hold all four.

[U] Run the check after every transfer, on the two players whose counts changed. That is where a new certainty is most likely to appear. The recommendation about *when* to run the check is a suggestion about human bookkeeping; no experiment here evaluates it.

H.4 Locked half-suits, and what waiting does and does not buy

[R] If your team holds all six cards of a half-suit, no opponent can take any of them. An opponent needs a card of a half-suit to ask in it, and an opponent has none. Ownership of that half-suit is therefore frozen (Theorem 3), and an opponent who claims it must claim it incorrectly, which awards it to you.

[R] Waiting therefore carries no risk to ownership (Corollary 4). No sequence of opponent asks can remove a card of a half-suit your side holds in full, so the passage of time cannot cost your team that half-suit.

[U] Spend the intervening turns resolving the split. If you are certain your side holds all six but unsure which teammate holds what, there is no time pressure from the opponents. Whether turns are better spent that way than on anything else available is not measured here.

[R] The rules push declaration timing in both directions. One consequence favours holding. A locked half-suit scores only when it is claimed, and claiming resolves six cards that were, from an opponent's point of view, part of the unresolved pool U; by (1) that tightens every opponent's capacity constraint on the cards that remain, so holding the half-suit keeps six slots absorbing probability mass the opponents cannot allocate elsewhere. The other favours claiming. A card of a locked half-suit licenses only asks that are certain to fail, so a hand composed largely of locked cards yields little from a turn, whereas a player who claims and empties a hand leaves the asking cycle and passes the turn to a teammate whose asks can succeed. A third channel runs alongside both: asks inside a locked half-suit remain legal for the owning team and still carry ask-legality certificates, so information about the split continues to change while ownership does not (§6.3). Develin [3] records that channel from the human side — such asks are available to the owning team and to nobody else — and the fitted v0.4-Fast configuration was given no term that uses it (§12).

[U] Which of the two dominates is a question about the position in front of you. It is quantitative, it depends on the opponents, and Theorem 3 does not settle it. Nothing in this paper measures the trade-off directly.

[R] If a half-suit is not yet fully yours, the ownership guarantee does not apply. An opponent holding a card of the half-suit has a legal ask in it, so a card you hold can still be taken away.

[U] In that case prefer the earlier claim. Weigh the risk that an opponent removes one of your cards against the chance that the split is not yet resolved. No experiment here isolates the value of claiming earlier in an unlocked half-suit.

[R] A hand made entirely of locked cards licenses only asks that are certain to fail. Every possible target of such an ask is void in the half-suit, so the ask transfers nothing and the turn produces no card movement.

[U] So claim the half-suit, empty the hand, and pass the turn to a teammate who can use it. That is a suggestion about what to do with a turn that cannot produce a transfer; the rules consequence above is what is established, and the fitted configuration was not given a term that targets this case.

[M] No experiment here isolates the value of waiting. The "cash locked half-suits immediately" switch is inert in the shipped configuration — the value-based stopping rule decides first, so the switch is never consulted — and its E5 row is exactly zero by construction. What is observed, not manipulated, is that on the primary bank the fitted v0.4-Fast configuration held a locked half-suit for 5.16 public events before a correct declaration, against 14.82 for FishBot v0.3 (§10); that is a description of two policies' behaviour, not evidence that either duration is better. What the rules guarantee is the safety of waiting with respect to ownership, not its profit.

H.5 Ranking candidate asks

The ordering below is roughly what the fitted weights produce. The support behind the individual steps is very uneven, and the tags say so. Every interval quoted in this subsection is a paired percentile bootstrap over deals, the construction described in §D.1.

1. **[R] An ask for a card you have located cannot fail.** The card is in the named hand, so the ask succeeds and the turn stays with you. **[U] Take every such ask before anything speculative.** The ordering is a suggestion; no experiment here compares it against alternatives.
2. **[R] A hit retains the turn, so an ask can start a sequence.** The value of an ask is therefore not confined to the single card it names. **[U] Prefer a likely card that leaves another likely card behind it over an equally likely card that leaves nothing.** No experiment here isolates this ordering.
3. **[M] Then the ask that completes a lock.** An ask that would give your team the sixth card of a half-suit converts a contested half-suit into a frozen one. In E5, zeroing the lock-completion weight cost +3.20 points (95% paired percentile-bootstrap interval +1.55–+4.85) on the frozen configuration — one of the few individual ask weights whose interval excludes zero.
4. **[U] Then consider who receives the turn on a miss.** A failed ask hands the turn to the player you named, so you are choosing a successor as much as a card; avoid handing it to a player visibly strong in a half-suit where your side is exposed. In E5, zeroing the reply-threat weight changed the win rate by -0.35 points with a paired interval of -1.98–+1.27, so this study does not establish the effect.
5. **[U] Then consider what the ask gives away.** Your first ask in a half-suit announces that you hold one of its cards. Between two near-equal asks, the one in a half-suit you have already revealed leaks less. In E5, zeroing both information-leak weights changed the win rate by +0.12 points with a paired interval of -1.38–+1.65; again not established.

H.6 Checking a declaration before making it

[R] Two different claims are involved, and only the stronger is sufficient. "Our side holds all six of these cards" and "I know which of us holds each one" are not the same statement: a declaration names a card-to-player allocation, and an allocation that is right about the team and wrong about

the split fails.

[U] Before claiming, name all six cards and a holder for each, explicitly. If the team-level claim is solid but the split is not, and the half-suit is already locked, wait and resolve the split. If it is not locked, weigh the incomplete claim against the chance that an opponent removes one of your cards first. The procedure is a suggestion about how a person might make the check; nothing in this paper evaluates it.

[M] The measured difference between the two agents is in declaration accuracy; which half of the mechanism produces it is not resolved. On the primary bank v0.4-Fast declared correctly 98.55% of the time against FishBot v0.3's 82.59% in the same games (§10). In E5, replacing the fitted stopping rule with a fixed confidence threshold changed the win rate by +0.12 points (95% paired percentile-bootstrap interval -1.23–+1.47), and moving the declaration confidence threshold between 0.80 and 0.99 also changes nothing the experiment can resolve. That stopping-rule ablation is unresolved from zero, so these experiments do not attribute the declaration-accuracy difference to the probability estimate rather than to the rule that consumes it. The calibration detail in §F is a description of that estimate, not an explanation of the behaviour.

H.7 The forced endgame

[R] Running out of cards is not a defeat. A player with no cards cannot be asked, so nothing can be taken from them. A whole team with no cards forces the opponents to claim every remaining half-suit without asking, and some of those claims will be wrong, which awards those half-suits to the cardless team.

[U] Emptying a hand deliberately is an option worth knowing, not a measured recommendation. The preceding paragraph follows from the rules; no experiment in this paper isolates the value of steering towards a cardless hand on purpose, and the fitted configuration was not given a term that rewards it.

[R] A correct claim is informative to everyone; an incorrect one is informative and costly. A correct claim announces the exact split of six cards, which changes every player's count of what remains and can resolve a half-suit that was previously unresolved. An incorrect claim supplies the same information to the opponents and gives them the half-suit as well.

[U] When it is your team that must claim everything, claim the surest half-suit first. Ordering the claims by confidence makes the later claims easier, on the argument above; the reverse order forgoes that help. The engine's forced endgame uses a fixed willingness ladder rather than a confidence ranking (§7.5), and no experiment here compares claim orders.

H.8 Deliberate misdirection

[M] Nothing in this study supports paying for theatre. The v0.3 study reported no reliable benefit from deliberate misdirection, and this study does not change that. v0.4-Fast contains no bluffing rule, and on the primary bank it won 99.95% of games against the misdirection artist, the archetype in the panel built around deceptive asking (Table 14). That is the second largest margin in the panel, behind the random legal control at 100.00% and ahead of the focused hunter at 97.64%, the adaptive diversifier at 92.14% and FishBot v0.2 at 84.24%: the deception archetype is not separated from the panel's other weak baselines.

That is a result about one archetype in one panel, and it is not an isolation experiment: the misdirection artist differs from the other baselines in more than its use of deception, so the margin cannot be attributed to the deception alone. The rules-level observation behind the negative result is narrower and firmer: an ask is legal only from a hand that holds another card of the half-suit, so the information an ask reveals is constrained by what the asker actually holds. Behaviour that changes

neither what the other players can deduce nor who holds the turn changes nothing that the game scores.

I Extended related work

This appendix records background that informed the design but that the main argument does not depend on. Nothing here is used to support a result in §10; readers who want only the load-bearing literature should read §3 and stop there.

I.1 Further informal sources on the game

Beyond Pagat [2] and Develin [3], the descriptive corpus is thin. Pagat's tactics list covers deliberate misses that inform a partner, locking out a dangerous opponent by never asking them anything, and exhausting one rank across all opponents before switching rank; it also records a partner-signalling convention and a "Forced Claims" rule, both attributed to named contributors. Develin's chapter contains the corpus's only quantitative endgame discussion, in which the alternative to a coin-flip declaration would need a success probability greater than one. The Wikipedia entry [6] names the "stalemate breaker", and secondary strategy pages [4, 5] restate blackballing, asking the asker, and an endgame delegation heuristic. We use these as evidence about how the game is played and talked about, not as sources of quantitative claims. The two substantial sources also disagree with each other on this point: Develin's chapter forbids pre-agreed conventions outright, where Pagat records a partner-signalling convention as an accepted tactic.

On Somani's agent [7, 8]: it is a Python implementation with a complete rules engine and a sound deduction fixed point, and the four-player, 48-card variant it implements makes the declaration problem structurally easier than the one studied here. With a single teammate, naming the allocation of a six-card half-suit ranges over 2^6 possibilities and is frequently forced by the deduction fixed point; with two teammates it ranges over $3^6 = 729$, and the allocation question stops being a formality. The published learner regresses a hand-written per-outcome score rather than a terminal game result, which penalises the deliberate miss that both Pagat and Develin recommend, and its move generator excludes known misses except as a fallback. Reading the published source, the terminal comparison that would supply the win/loss signal tests an integer team identity against an enumeration member, so it never succeeds; on that reading every move in every training game receives the losing reward and the released model never saw a win/loss gradient, which would make it a greedy maximum-likelihood asker rather than a learner of the game's objective. We record this as a reading of the source rather than as a reproduction: we did not run the training. No strength numbers have been published for it, so it is not usable as a baseline here in any case.

One methodological caveat from the local-best-response literature bears directly on the probe of §10.6 and is recorded here because the probe does not address it. Lisý and Bowling report that restricting the point at which the response is allowed to deviate changes the measured exploitability substantially, so a single entry point understates it and the entry point has to be swept [71]. Our response is fitted over the whole game with no sweep over where it may begin deviating, which is one of the reasons §10.6's figure is a lower bound rather than a bound.

I.2 Equilibrium computation

The equilibrium line runs from counterfactual regret minimisation [27] through outcome- and external-sampling variants [28], CFR+ [29] and discounted CFR [30] to function approximation [31, 32]. Two properties of this family bear on Fish. CFR+ performs relatively poorly in games where some actions are very costly mistakes [30], and an incorrect declaration is exactly such an action; and DREAM's

exploration term grows with depth [32], while a game in our dialect runs to 101.4 public events on average against FishBot v0.3 in E3. Student of Games extends the belief-conditioned search line and identifies Scotland Yard as the closest published public-observation domain [34]. We did not attempt any of this.

I.3 Team-game complexity results

§3 names the target concept for a Fish team and states why we do not compute it; this subsection records the machinery that would be needed if one did. Celli and Gatti establish the complexity of the team-maxmin equilibrium with correlation and the worst-case cost of forgoing correlation [36]. Farina and coauthors connect ex-ante team collusion to extensive-form correlation [37]; the team belief DAG and the two-player conversion give a zero-sum game whose equilibria are realization-equivalent to a team-maxmin equilibrium with correlation [38, 39], at a representation cost exponential in how much private information distinguishes teammates inside a public state. Reductions to a continuous-state game over public beliefs [33], building on the common-information reduction of decentralised control [35], are the other standard route. Learning-based attacks on team games exist [40, 41]; the fictitious-cross-play analysis proves that self-play converges to *local* team equilibria [41], which is the formal version of the informal complaint that a self-played agent's asks stop carrying meaning outside its own training population. Together with the information-set size quoted in §3, these results are why this report makes no equilibrium claim of any kind.

I.4 Cooperative play through public actions

Hanabi is the reference benchmark for coordination through observable actions [42], and is relevant because Fish likewise has no side channel. The Bayesian Action Decoder made the public belief an explicit object of the policy [43]; the Simplified Action Decoder showed that letting teammates condition on the greedy rather than the exploratory action is a prerequisite for conventions to survive ε-greedy training [44]; Other-Play demonstrated how much of a self-play score is convention rather than skill, with the same agents scoring far worse when paired across independent training runs [45]; and Off-Belief Learning provides an explicit dial on convention depth together with a uniqueness result that makes independent runs interoperable [46]. Test-time search over a fixed blueprint provides a further improvement [47]. Bridge bidding is the other model system, an auction that is literally a learned language: bidding networks improved on a strong commercial program [53], joint policy search — which scores simultaneous changes at several information sets, which is what inventing a convention requires — improved a system further [54], and training with human partners improved results again [55].

The asymmetry that separates Fish from both is the audience. Hanabi has no adversary, so every bit of signal is pure profit, and in bridge the harm done by a leaked auction is diffuse. In Fish a leaked card location is directly executable: once it is public that a player holds a card, any opponent holding another card of that half-suit can take it and keep the turn. The Hanabi construction that most clearly fails to port is the modular hat-guessing code, which works because each receiver sees every other receiver's hand and can subtract it [48]; in Fish no player sees any hand. Any future claim that an agent has learned a convention would also have to survive the scramble ablation of [52], in which high speaker–listener consistency coexisted with no causal effect.

I.5 An informal secrecy argument

The following is an argument, not a theorem; we state it because it is the design rationale for building the agent with no private channel at all, and we have not proved it. Teammate and opponent hands are exchangeable given the public history. An absolute codebook — one whose meaning is fixed by public

information alone — is therefore decoded identically by all five listeners, so with two teammates against three opponents the Wyner-style secrecy rate of such a convention is non-positive [50, 51]. Only a receiver-relative codebook, whose semantics depend on the receiver's own holdings, would have positive rate, and it pays for that with ambiguity at the encoder. The justification here is structural and informal: we give no proof, and none is claimed. Public-signalling economics predicts a similar outcome, with cheap talk separating before two audiences only when it is credible against both and otherwise collapsing toward pooling [49].

I.6 Remaining determinization detail

§3 states that averaging perfect-information values over deals sampled from the public history collapses to a hit-probability heuristic in this game; the collapse is the following. Almost every reachable position carries the same perfect-information value, because the team on turn takes everything it can reach, so with t ranging over opponents and c over askable cards and h the public history,

$$\arg\max_{(t,c)} \mathbb{E}[V^{\mathrm{PI}}(t,c)] \;\approx\; \arg\max_{(t,c)} \Pr[\,t \text{ holds } c \mid h\,].$$

The right-hand side is one ply deep and blind to the information an ask leaks, the information a refusal supplies and the option value of postponing a declaration, which is why the hit probability enters §7 as one feature among several rather than as an engine.

Determinization for bridge was proposed well before it was made to work [11], and constrained deal generation in practice is done by rejection: deal against suit-length restrictions, then discard deals whose auction the program's own bidding module would not have produced [12, 16]. Frank and Basin named the two mechanisms that make the method unsound however many worlds are drawn: *strategy fusion*, in which the searcher plays a different strategy in each sampled world although a real player must commit to one strategy per information set, and *non-locality*, in which a node's value depends on parts of the tree outside its own subtree [13]. Long, Sturtevant, Buro and Furtak characterise when determinization nevertheless performs well, through leaf correlation, bias and a disambiguation factor [15], and Information Set Monte Carlo Tree Search replaces the independent trees with a single tree over information sets, redeterminizing each iteration [17]. The repairs that work are those attacking non-locality as well as strategy fusion [14]. Information Set Monte Carlo Tree Search has been extended by re-determinizing at non-root seats and by self-determinization, which allows a form of bluffing to emerge [20, 18]; it is nevertheless unsound, and its measured exploitability can *increase* with additional computation [21]. A Spades post-mortem traced blunders to a sampler that placed a card of true probability 1/3 into 87% of its sampled worlds [19], which is the failure mode our oracle test (E15) is designed to detect. One smaller result survives into our design: Go Fish sits at the maximum of Zuckerman, Felner and Kraus's Opponent Impact measure [22], so the choice of which opponent to ask is a first-class decision rather than a tie-break, and our ask features treat it as one.

I.7 Sampling and history filtering

The matrix-scaling approximation §3 adopts for the Fast path carries a guarantee: the deterministic strongly polynomial analysis of Linial, Samorodnitsky and Wigderson bounds the permanent within a factor e^n [60]. Of the permanent methods §3 sets aside, Ryser's inclusion–exclusion formula is exact in $O(2^n n)$ time [56], which is unusable at $n = 54$, and Glynn's sign-vector formula is the other exact method [57]; the Jerrum–Sinclair–Vigoda chain is a fully polynomial randomised approximation scheme [58] whose constants make it impractical at this scale, and a practical study measured it at tens of thousands of seconds on instances where Ryser's formula takes milliseconds [59]. Sequential importance sampling is the standard contingency-table alternative [61], with the documented warning

that it can underestimate by an exponential factor under any row or column ordering while appearing to have converged [62]. This is one reason §5 uses a direct construction rather than an importance-weighted one. More generally, constructing a history consistent with a public state is FNP-complete in the worst case, and enumeration is polynomial only for sparse public trees [26]; Fish escapes this because its public record determines every card movement, so the only object to be reconstructed is the initial deal.

I.8 Variance reduction, and why we do not use it

Poker supplies a family of control variates: advantage-sum estimators [65], importance sampling over imaginary observations [66], MIVAT's least-squares value function [67], and AIVAT [68, 70]. The reported gains are large in two-player settings and smaller in six-player ones, which is the closer analogue to our setting. We deliberately do not use AIVAT or MIVAT. AIVAT is unbiased for *any* value function, which is a constraint on its expectation rather than on any realised estimate; [69] tuned such a heuristic on a published six-player dataset until all fourteen players simultaneously appeared to be large winners, which the zero-sum structure makes impossible. A learned control variate fitted by us, on our own agent, on our own evaluation data, is the exact pathology this literature warns about, and the duplicate-block design of §9 already removes the largest part of deal variance without introducing a fitted estimator. That design is the standard remedy rather than a novel one: duplicate bridge replays each board with the same cards in the same seats, and the computer poker competition added common seeds across pairings for the same reason [64].

I.9 Rating systems and sequential testing

Bradley–Terry-style rating models [73, 74] have two documented failure modes when the rated population is a training run's own checkpoints: they are not redundancy-invariant, so duplicating one agent redistributes every rating, and they cannot represent cyclic dominance. Maximum-entropy Nash averaging over the antisymmetric logit matrix [75] and empirical-game response graphs [76] are the proposed alternatives. We report Bradley–Terry ratings in §E.2 as a compact summary of a fixed round-robin, not as a claim about a strategy space.

Two further traps shaped §9. Monitoring an interval computed for a fixed sample size and stopping when it looks favourable is not a test; in simulation this produced a false-positive rate far above nominal under the null [70], which is why sequential designs on normalised Elo exist [78]. Our seed banks are fixed in advance and reported at their planned size. Finally, refining an abstraction does not monotonically reduce exploitability [72], which is the general form of the observation in §10.4 that substituting a more accurate belief into a policy fitted around a less accurate one does not improve it.